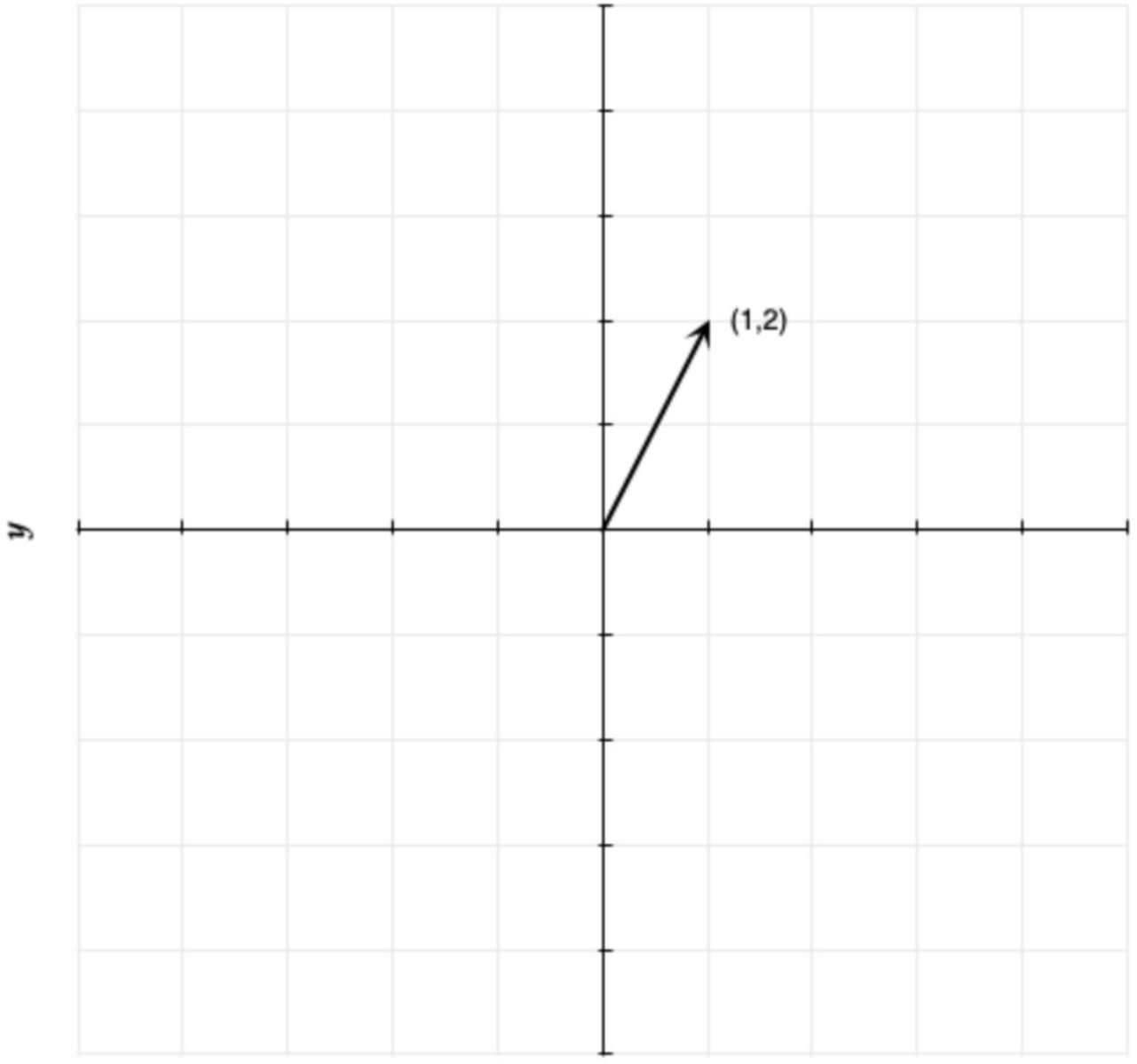
Language Models

From Vectors to ChatGPT

Vectors



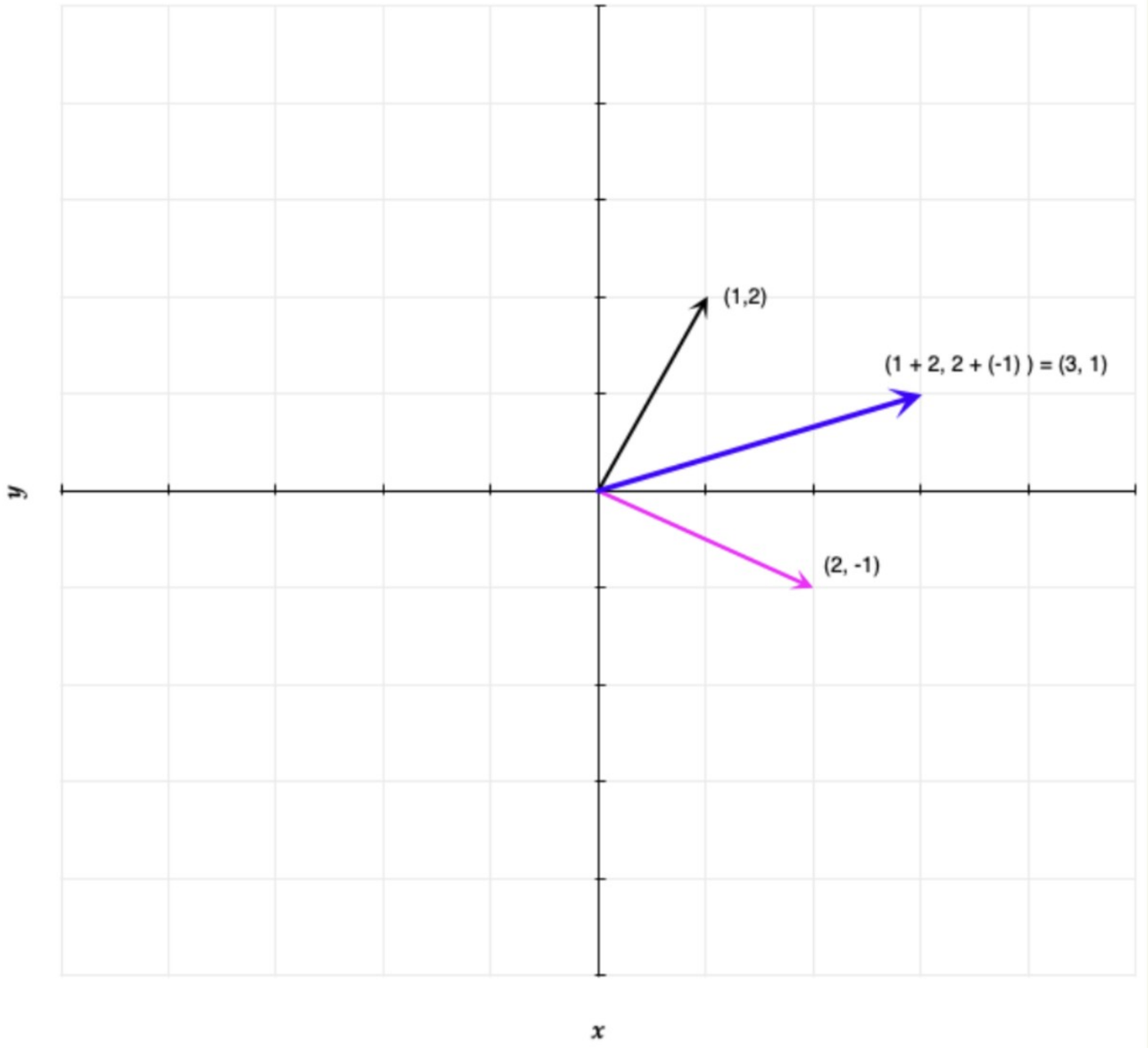
Geometric Interpretation of a vector

Vectors are arrows pointing in space.

For $\vec{v} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, imagine moving 1 step to the right (x-axis) and 2 steps up (y-axis) from the origin.

Every pair of numbers corresponds to one arrow, and every such arrow from the origin, corresponds to a unique pair of numbers.

Thus, a vector has a visual *length* and *direction* in space.



Vector addition

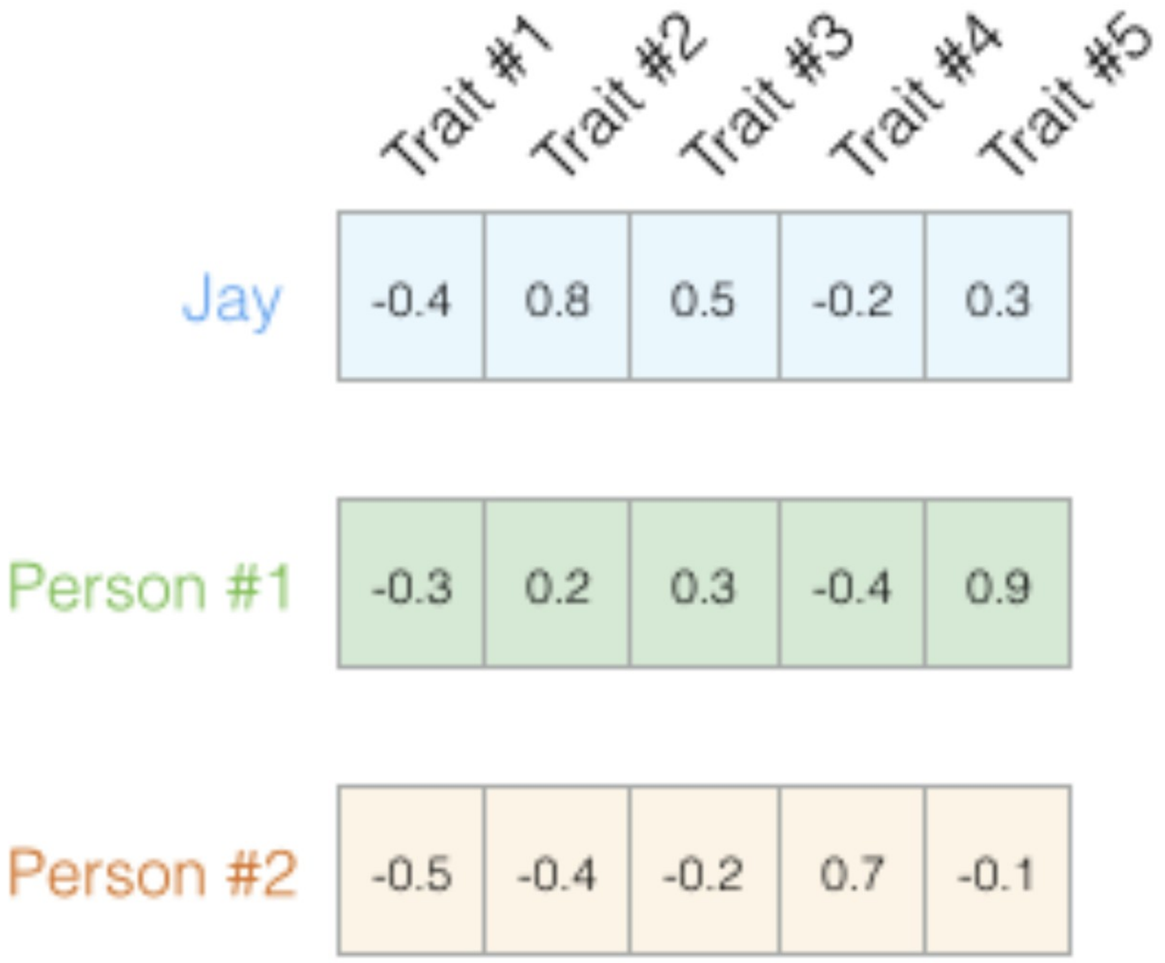
We can add vectors by adding the individual components using the geometric interpretation from previous. Here we first move one step right and 2 steps up, and then 2 steps right and one step *down* (-1). The net is 3 steps right and 1 step up. This can be seen as:

$$\vec{u} = \vec{v} + \vec{w} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 2 \\ -1 \end{pmatrix} = \begin{pmatrix} 1+2 \\ 2+(-1) \end{pmatrix} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}$$

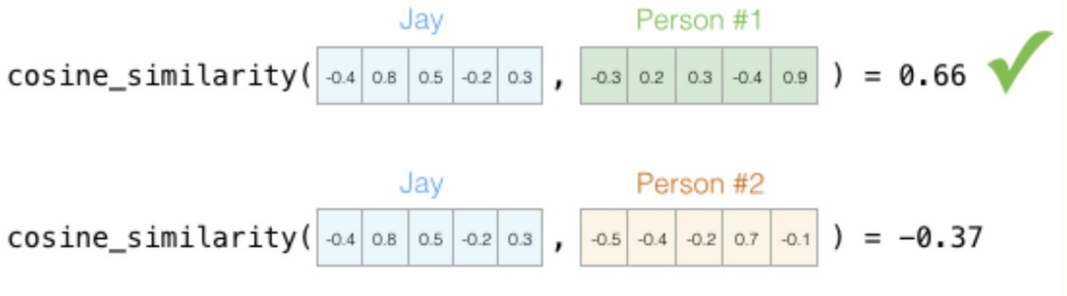
We are adding the x components in the column vectors to get the x component of the new column vector $\vec{u}$. We do the same for the y components.

Embeddings

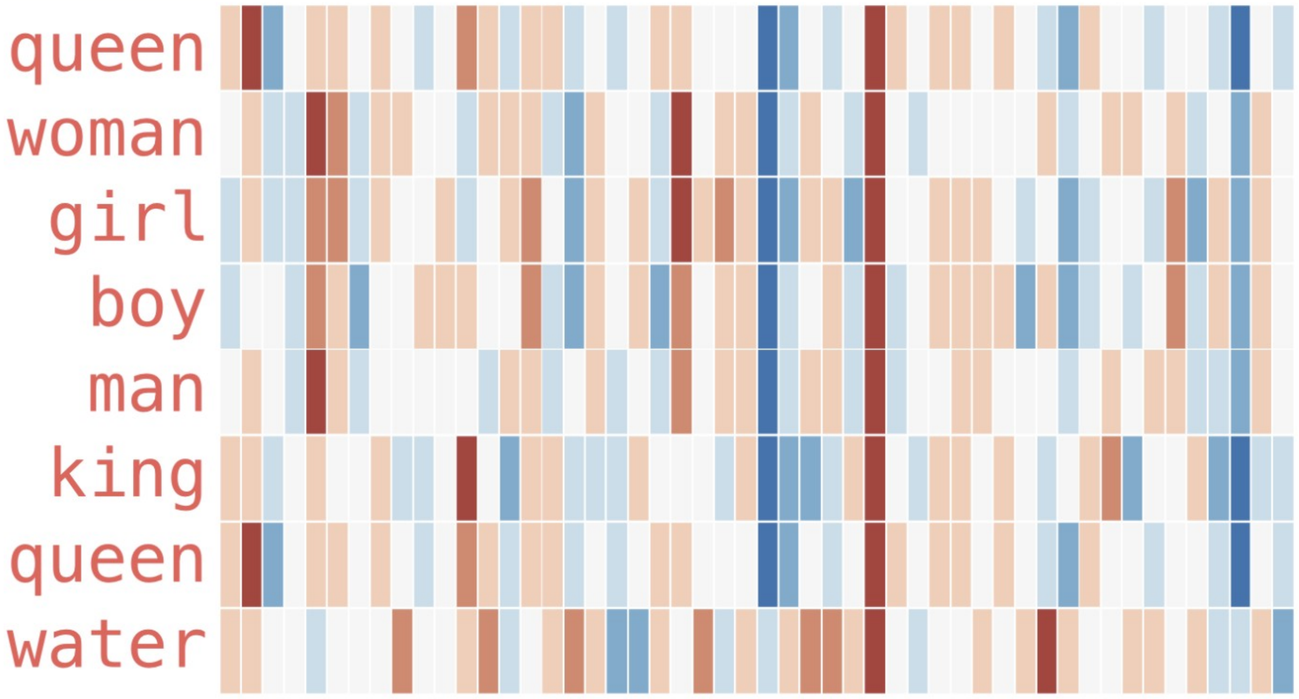
The Basic Idea



- Observe a bunch of people
- **Infer** Personality traits from them
- Vector of traits is called an **Embedding**
- Who is more similar? Jay and who?
- Use Cosine Similarity of the vectors

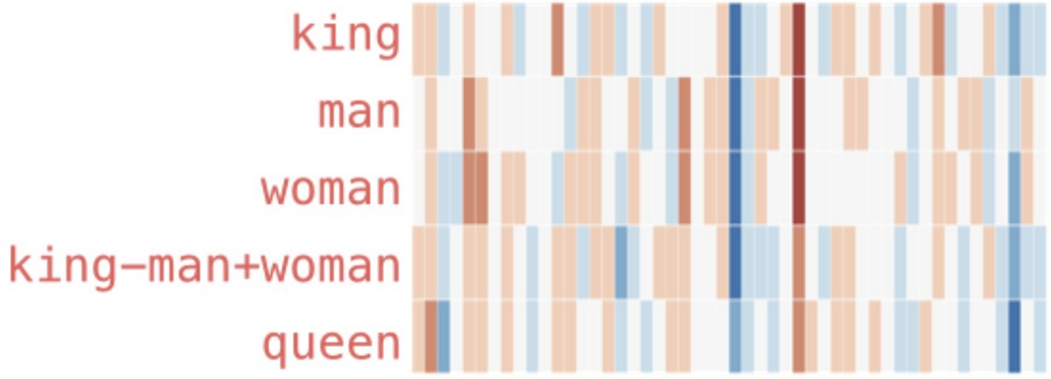


Obligatory example

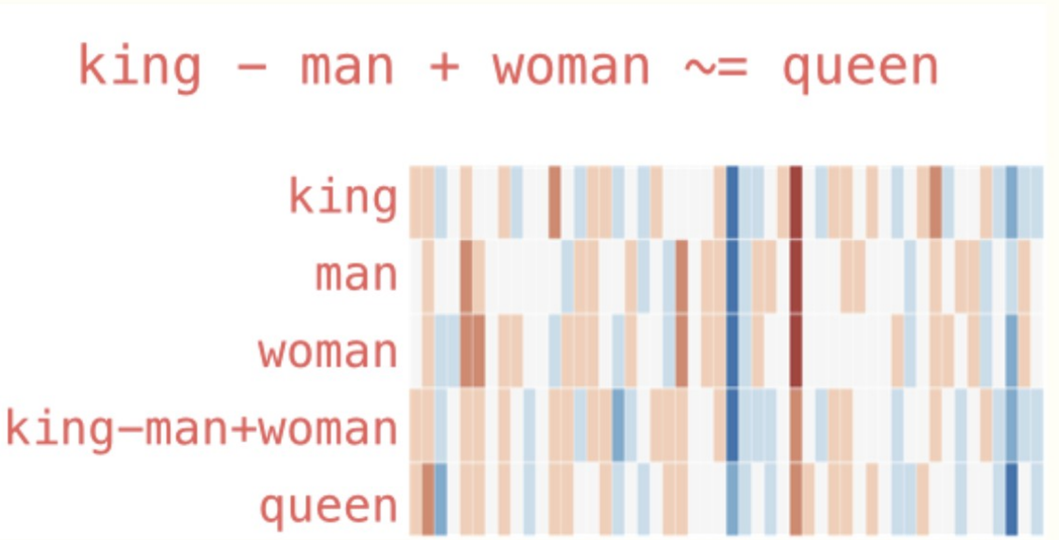


See man->boy as woman->girl, similarities of king and queen, for eg.
These are lower dimensional GloVe embedding vectors

king − man + woman ~= queen



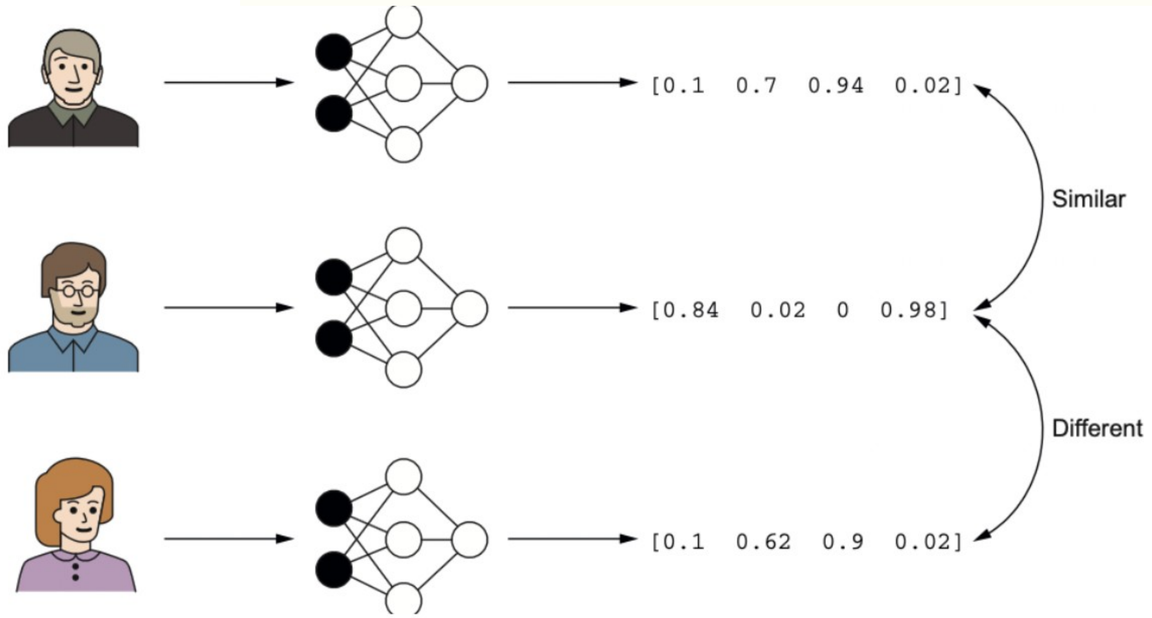




Embeddings map similar concepts to similar vectors

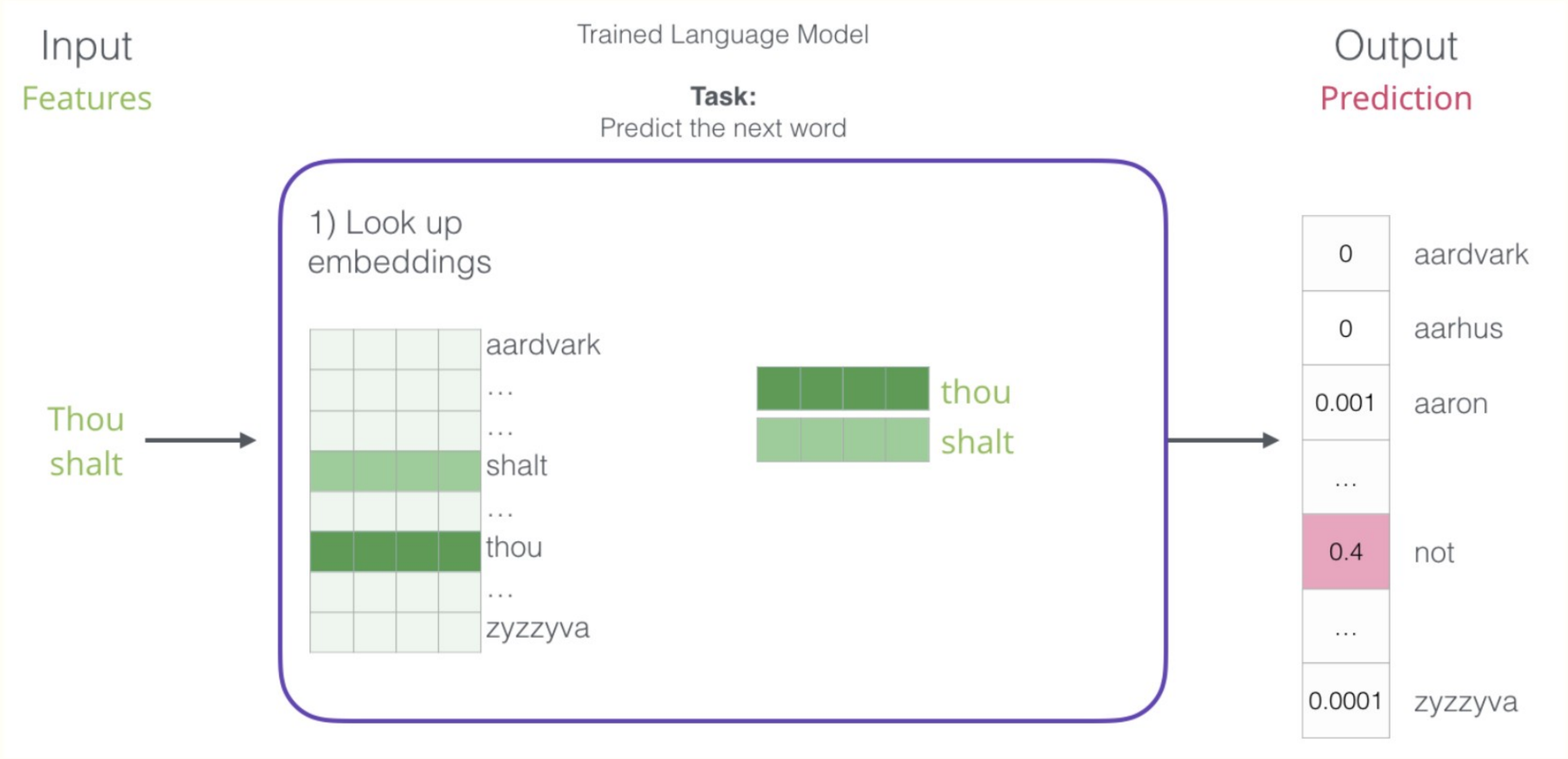
These concepts can be from different spaces, like users and movies

Any model can be used to produce embeddings: Apple face recognition.



How do we train word embeddings?

We need to choose a downstream task. We could choose **Language Modeling**: predict the next word. We'll start with random "weights" for the embeddings and other parameters and run SGD. A trained model+embeddings would look like this:



Why is language modeling a great task?

How do we set up a training set? All the labels are provided for us!!! This is **self-supervised** learning!

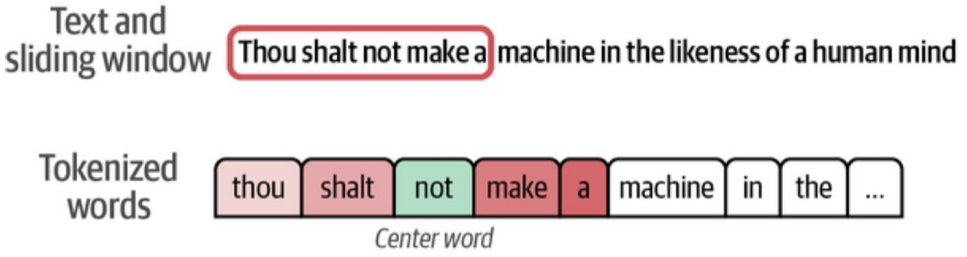
Thou shalt not make a machine in the likeness of a human mind

Sliding window across running text

thou	shalt	not	make	a	machine	in	the	…
thou	shalt	not	make	a	machine	in	the	
thou	shalt	not	make	a	machine	in	the	
thou	shalt	not	make	a	machine	in	the	
thou	shalt	not	make	a	machine	in	the	

Dataset

input 1	input 2	output
thou	shalt	not
shalt	not	make
not	make	a
make	a	machine
a	machine	in

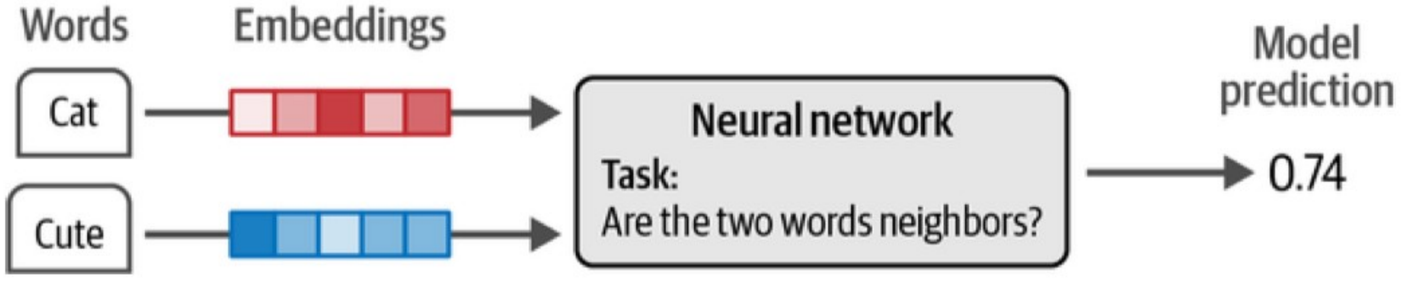


Word 1	Word 2	Target
not	thou	1
not	shalt	1
not	make	1
not	a	1
thou	apothecary	0
not	sublime	0
make	def	0
a	playback	0

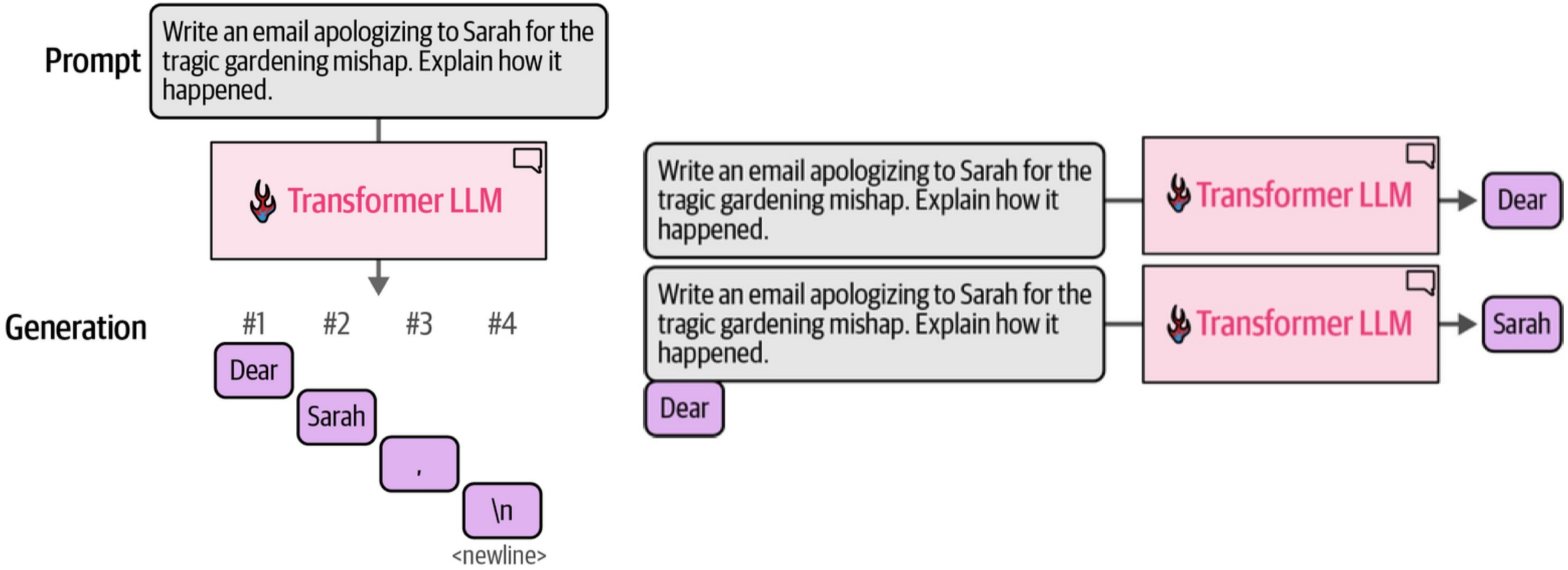
Positive examples

Negative examples

It is not enough to have just positive samples, you need negative samples too. These can be obtained by random sampling.

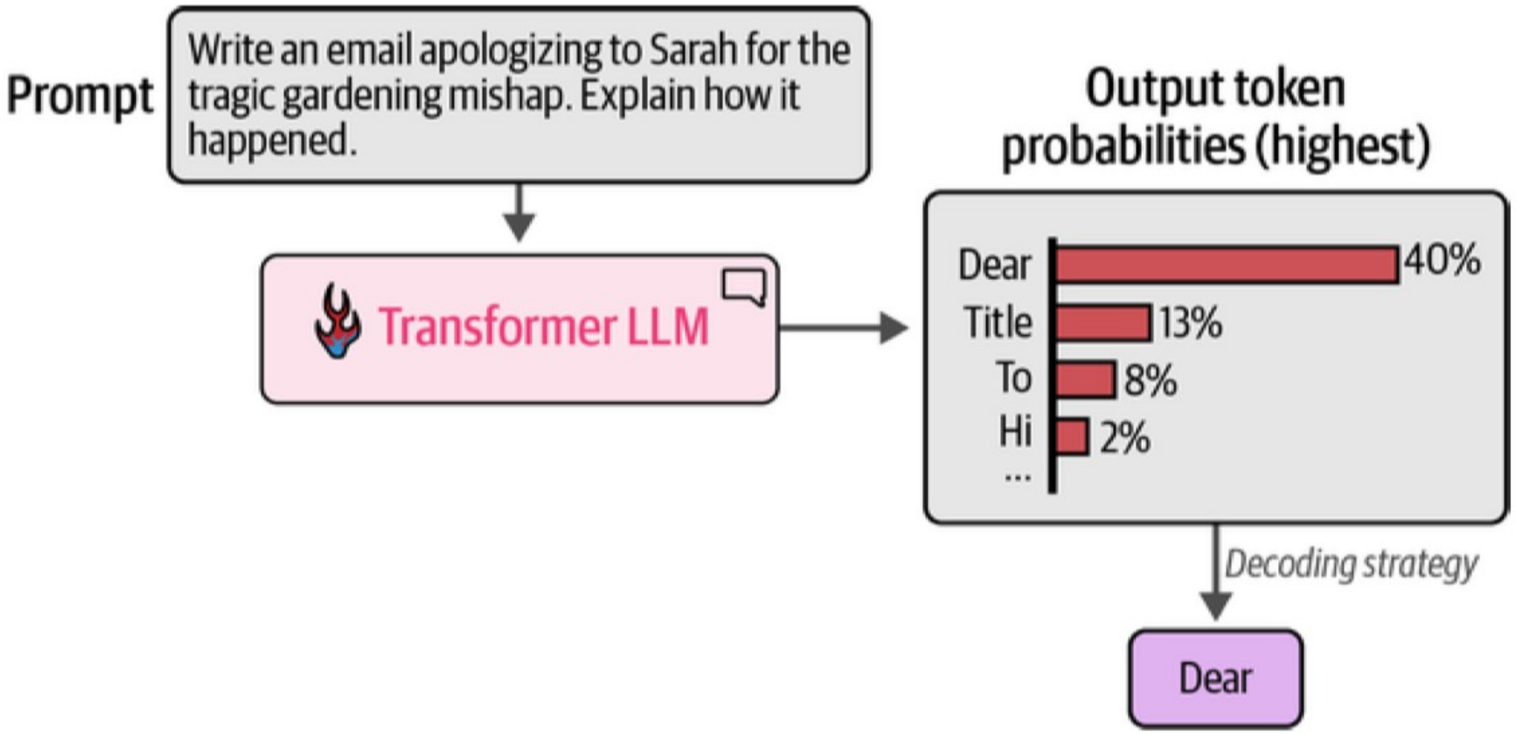


Generation happens token by token



Probabilities drive everything

We sample from probabilities to get the next word

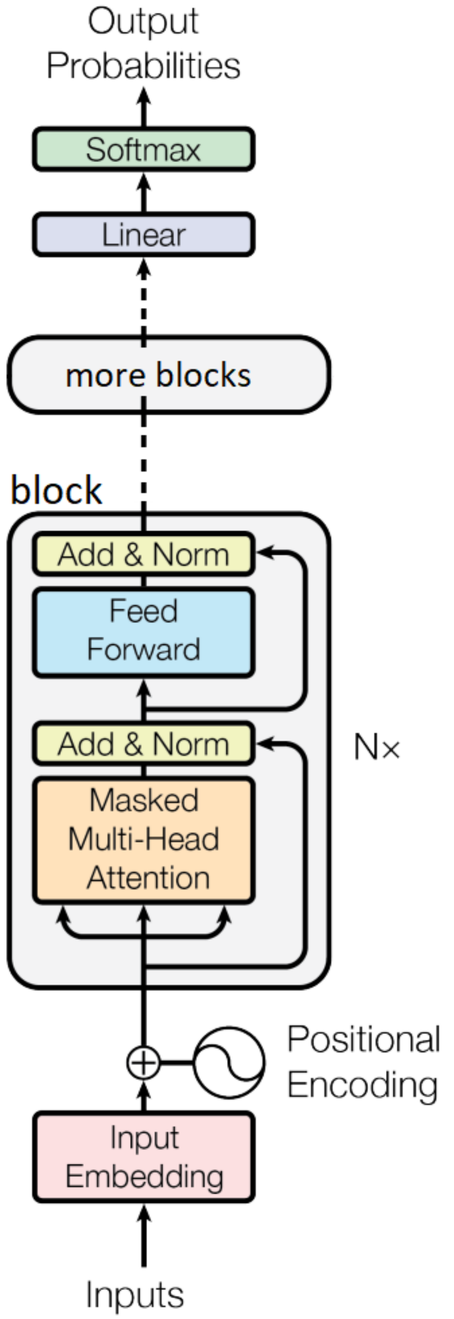


Why use a transformer architecture?

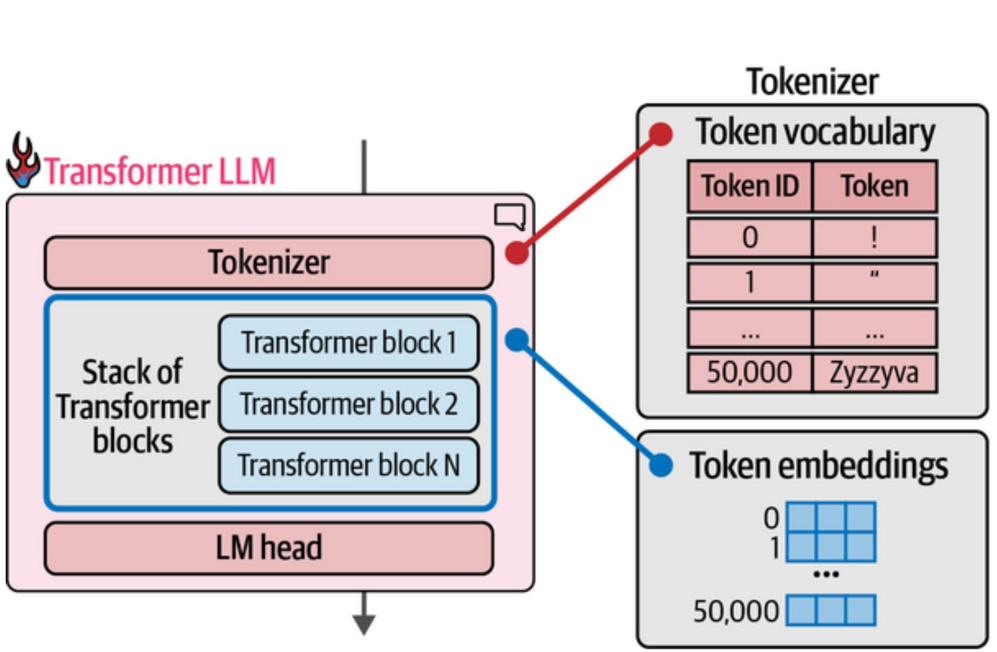
- **ATTENTION**: The transformer has a mechanism to figure out which words in a context are important to understanding the meaning of a given word and thus impacting its embedding

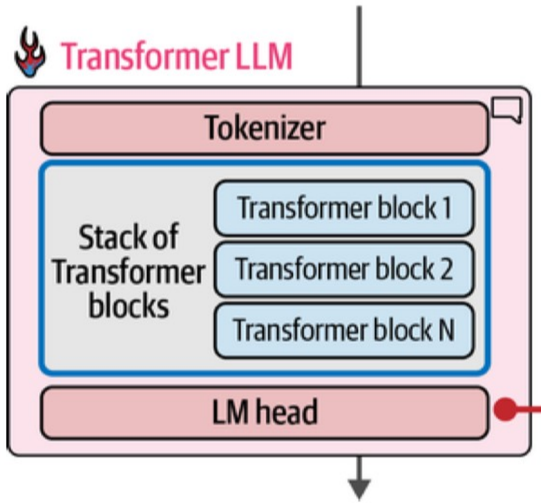
- **MEMORY**: The transformer has a MLP mechanism to associate with a word and its embedding facts that are pulled out from a kind of memory database.

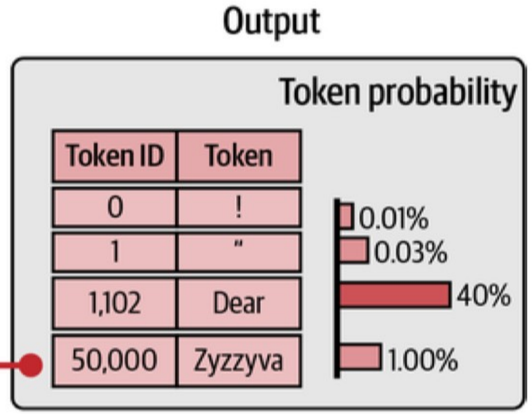
- **PARALLELISM**: At the level of parallel paths through tokens, parallel attention heads, and parallel paths through the memory bank (MLP) there are multiple opportunities to exploit the parallelism inherent in GPUs. This is in contrast to RNN mechanisms.

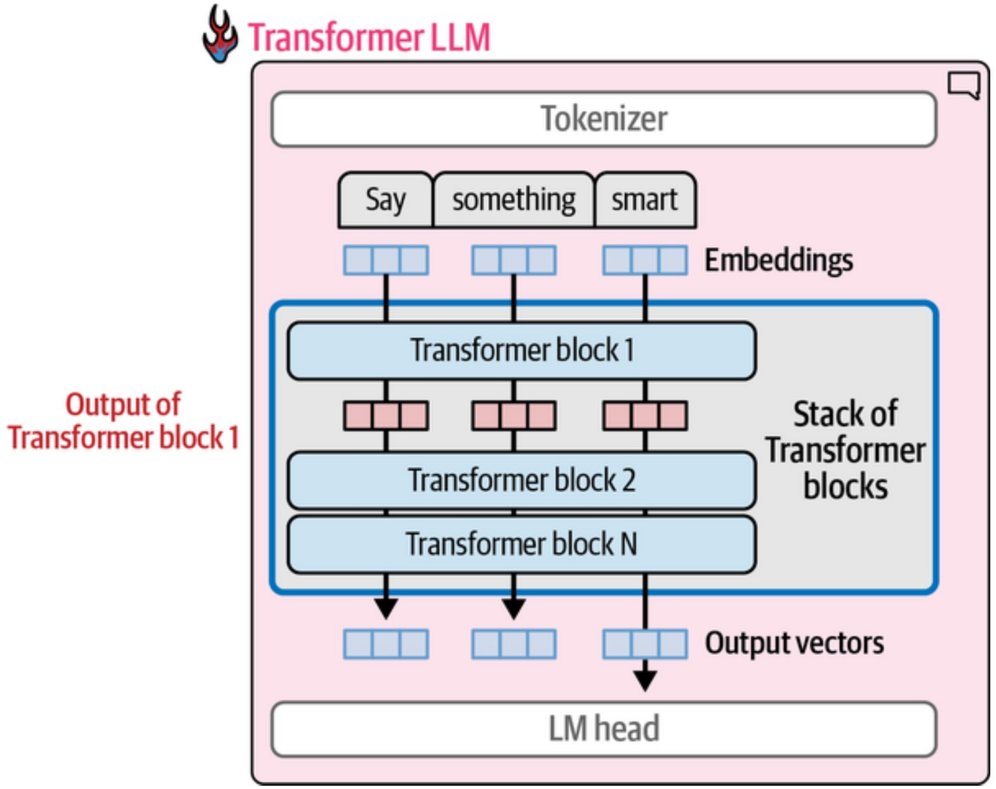


The GPT Architecture



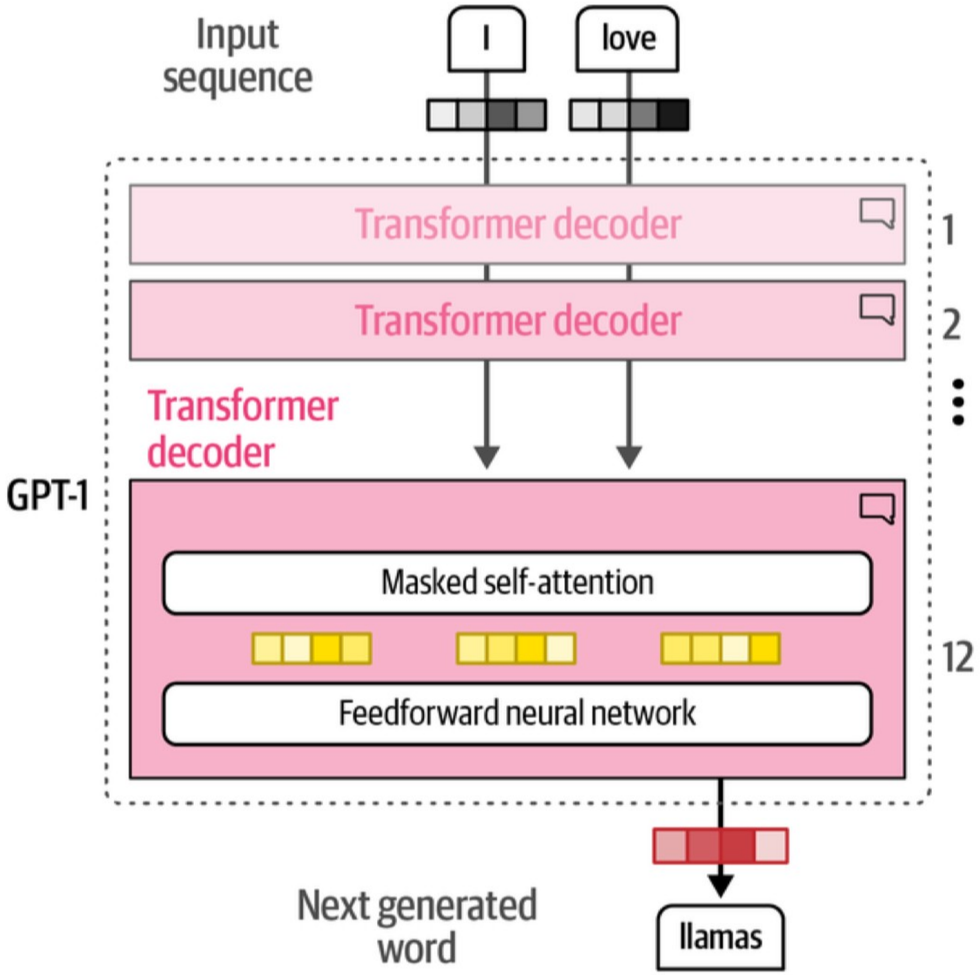


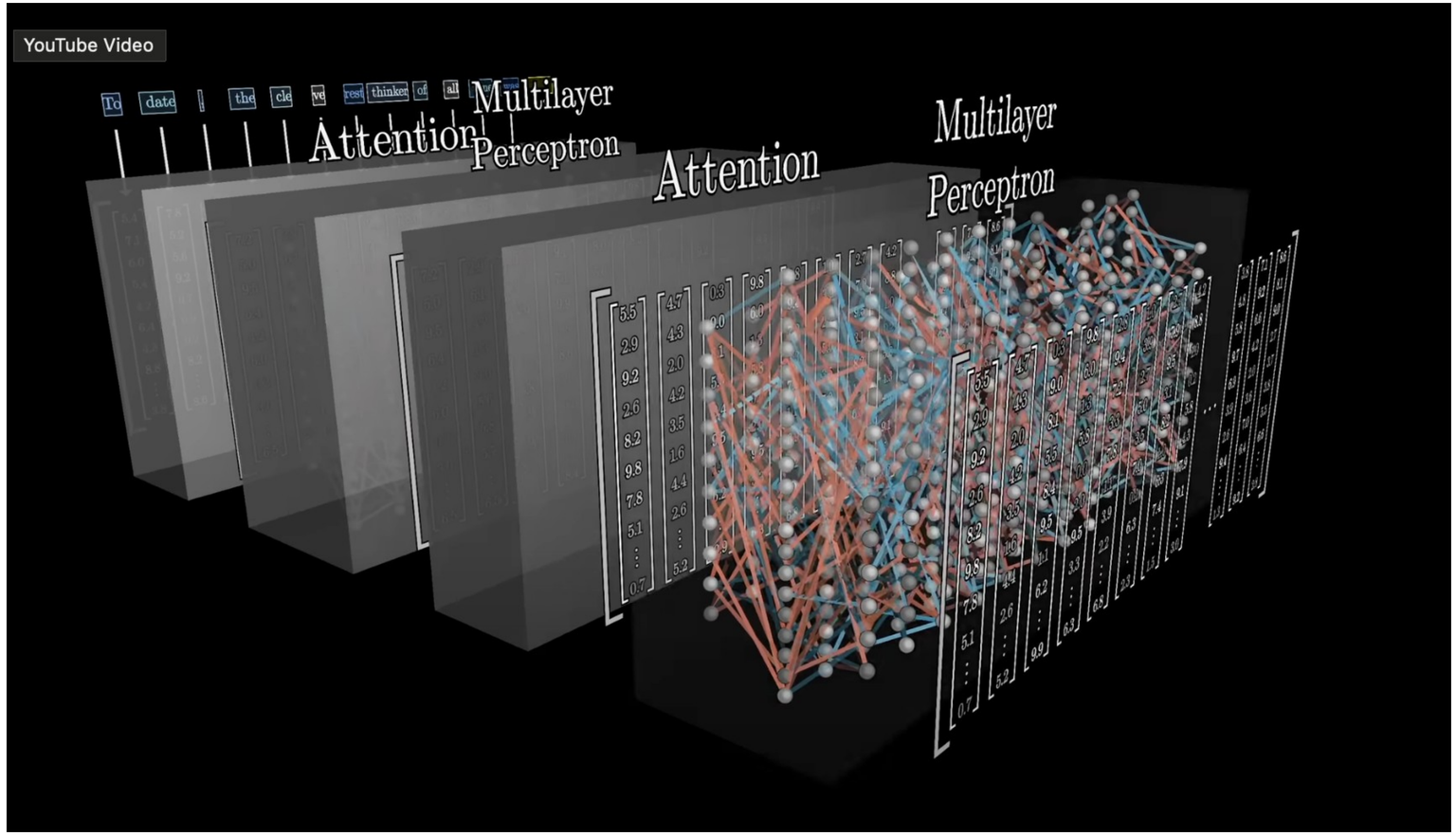


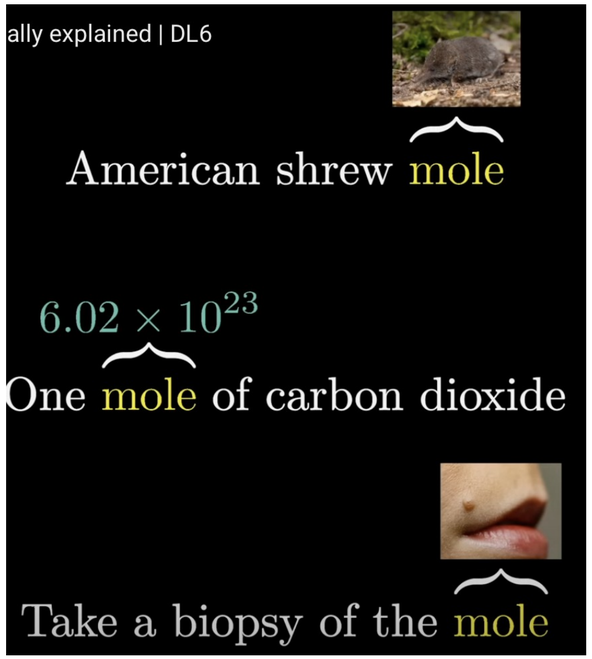


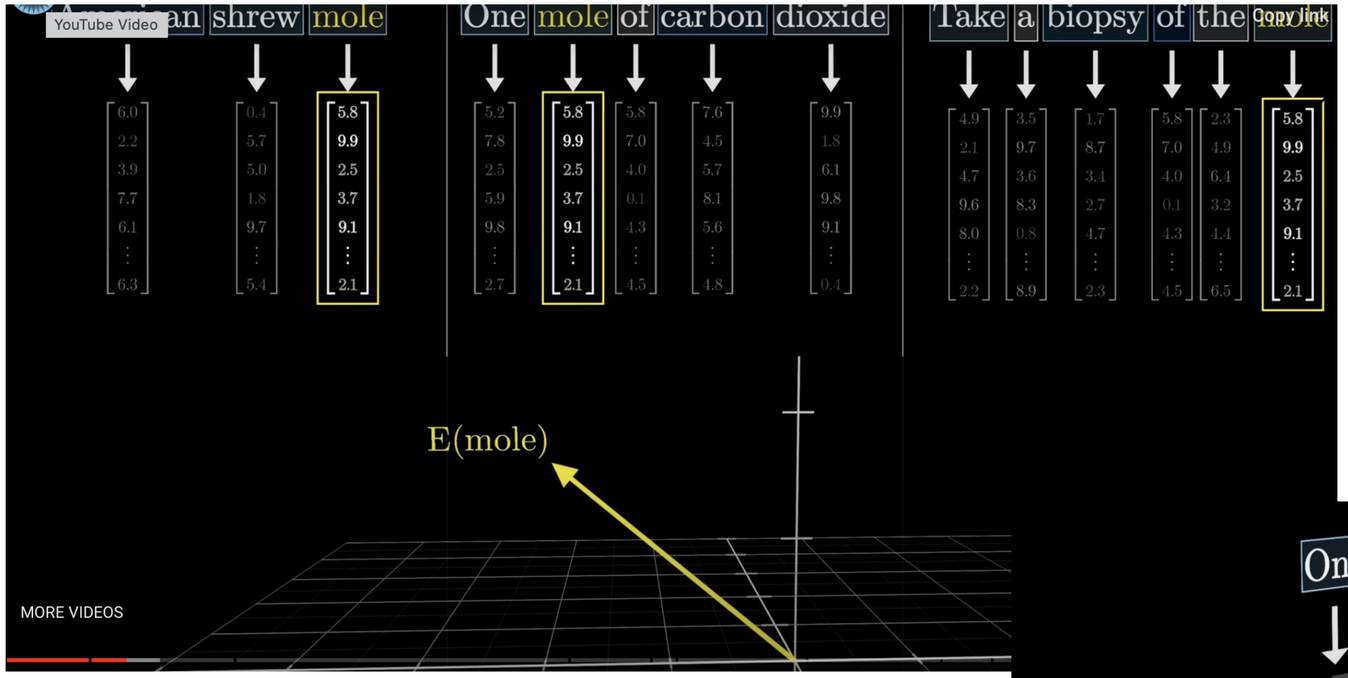
Output of Transformer block 1

What does the Transformer Block look like?



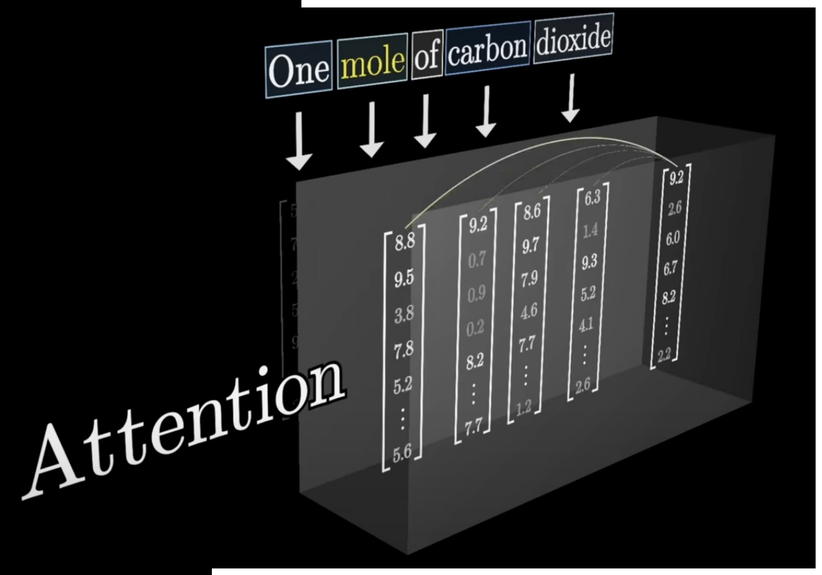


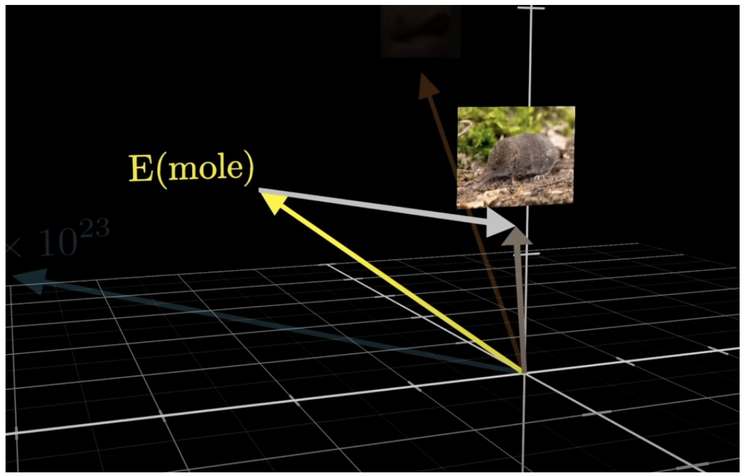


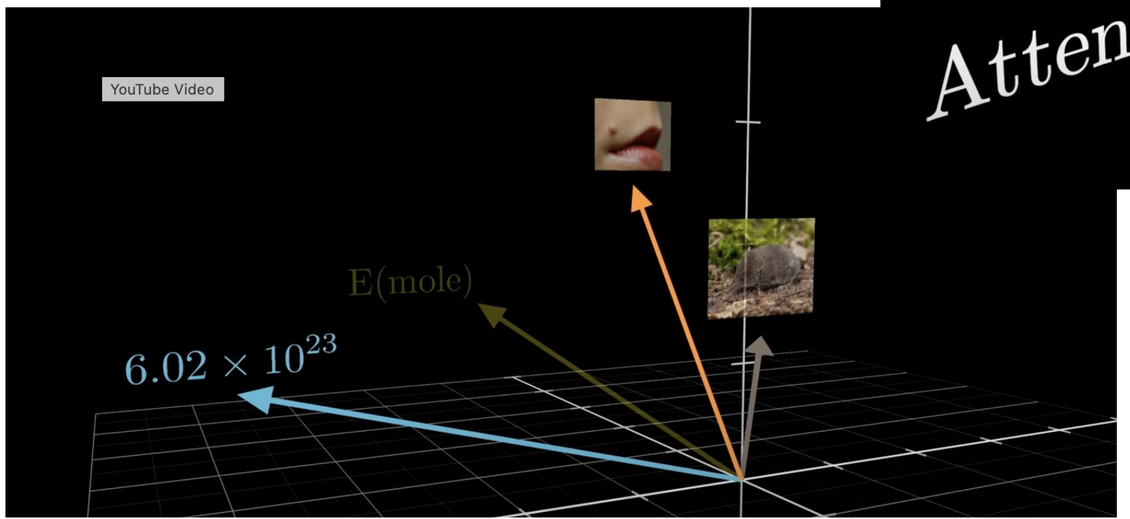


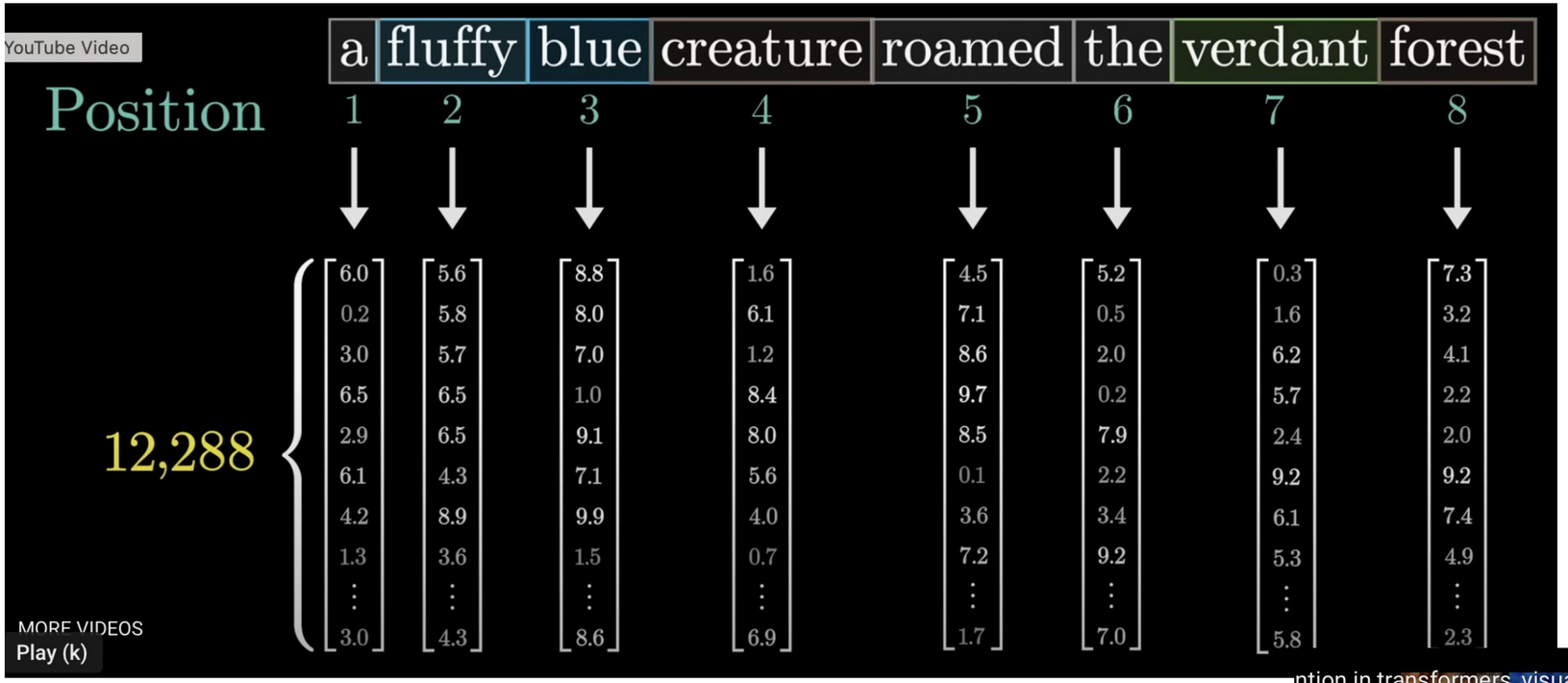
Attention Contextualizes

We start with un-contextualized embeddings for mole. The attention process produces vectors to add to the original mole embedding to contextualize them from their surrounding context.

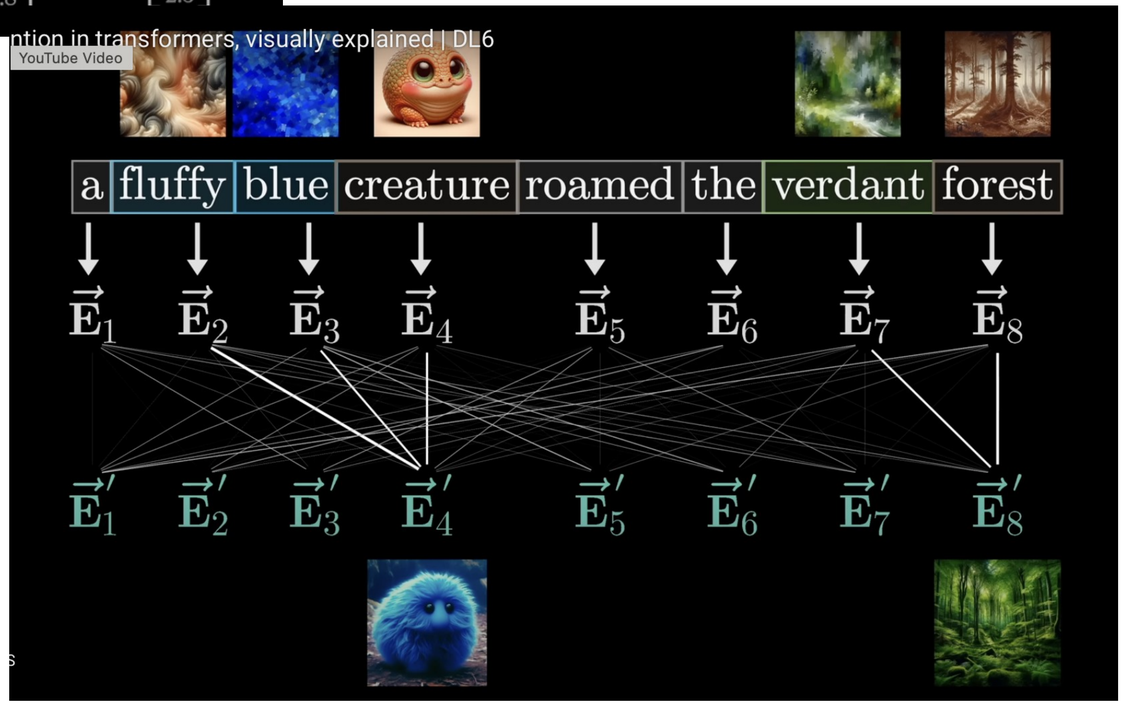


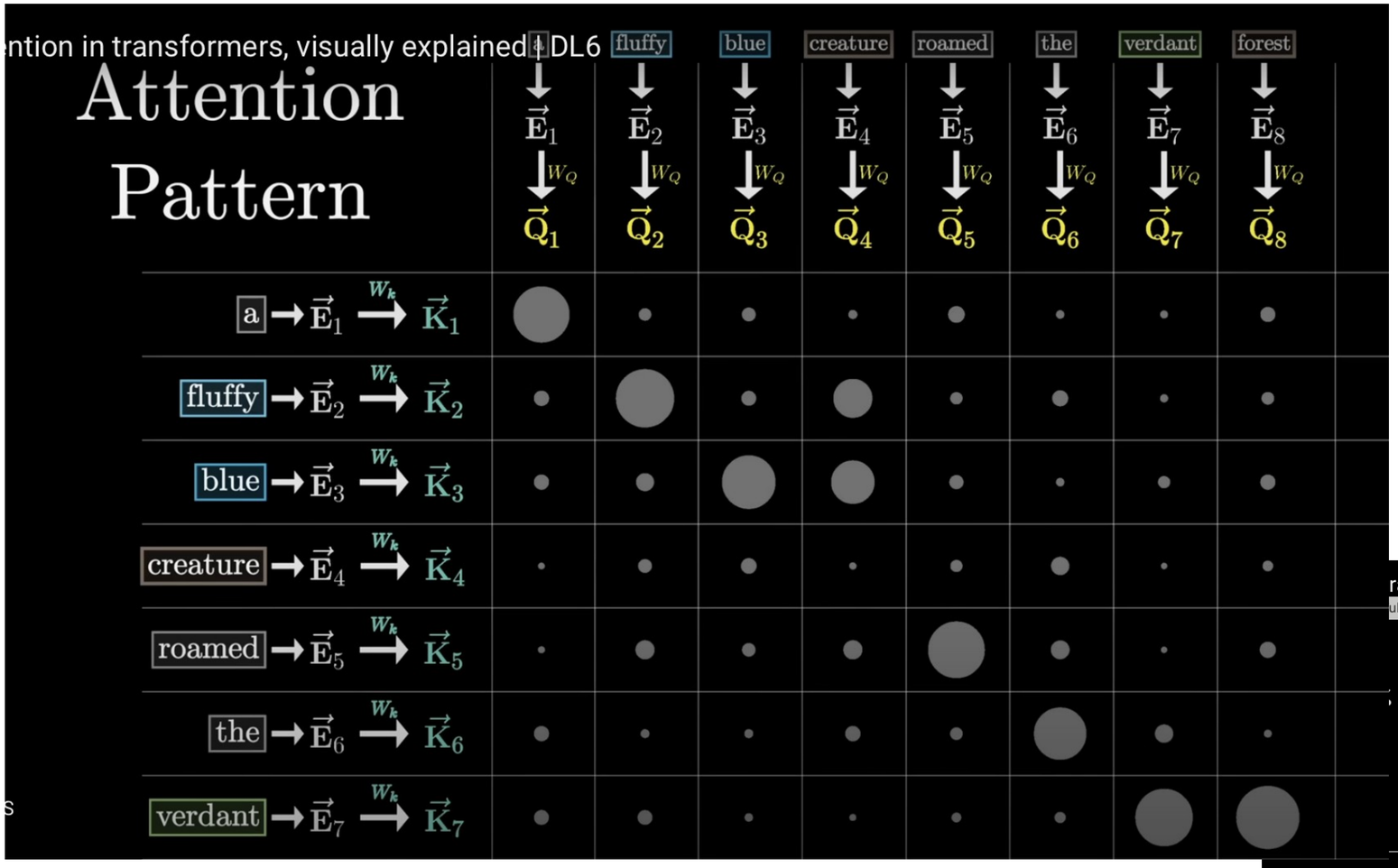


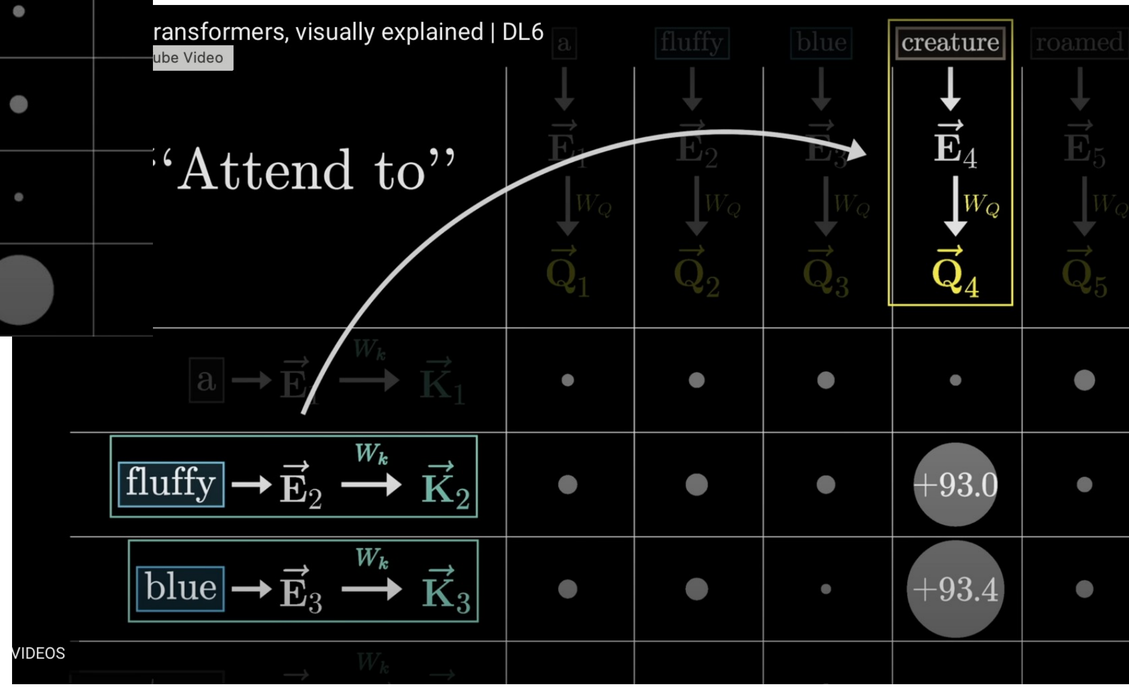




As an example consider the sentence above. You want the adjectives fluffy and blue to impact the word creature, and verdant to impact forest. In the image on the right, these are shown to have strong connections, or high attention values

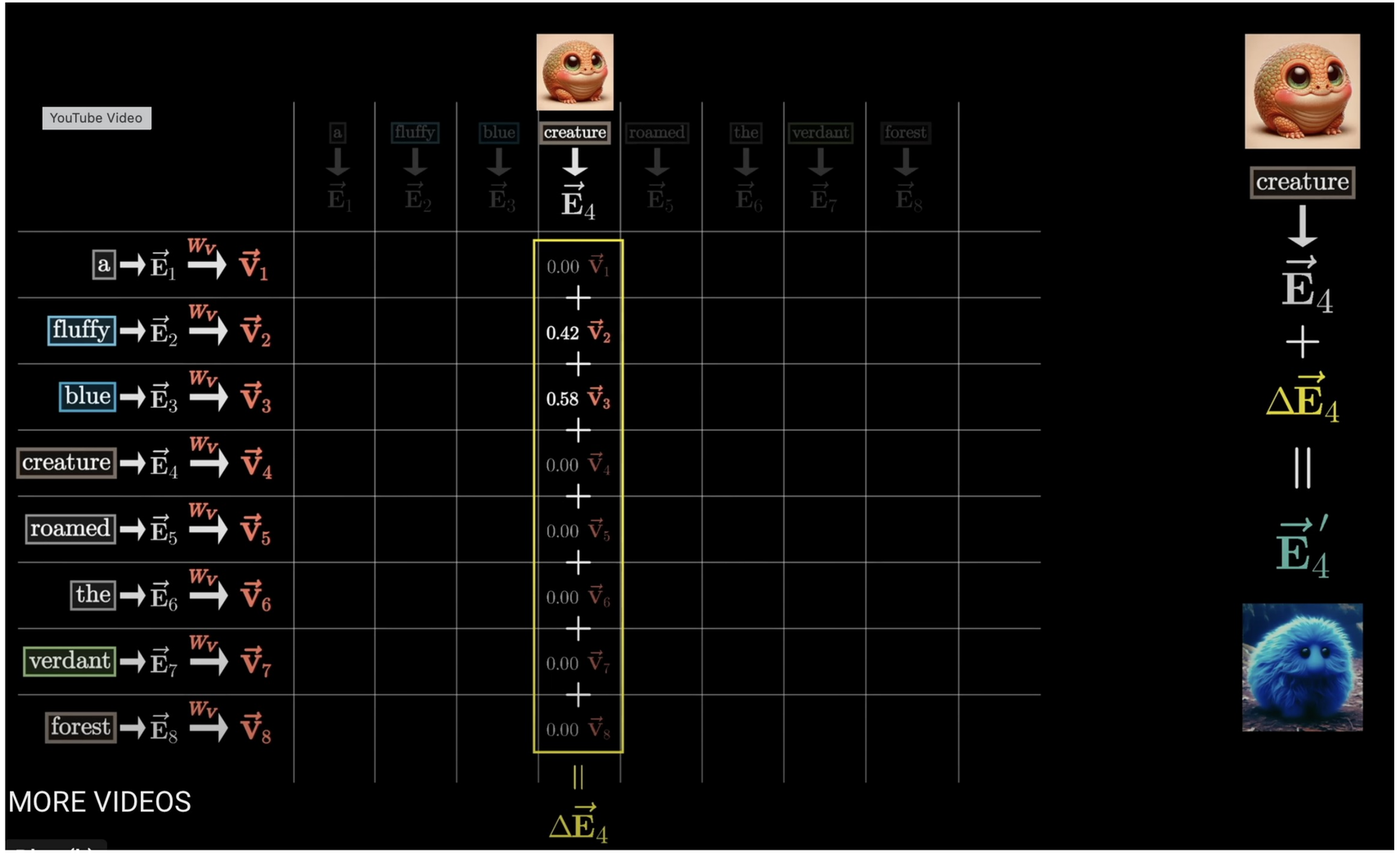






Attention

Dot Product



Now we multiply the for each of the context tokens by its attention based relevance score to combine the information from all the context in proportion to what is more important

Copy link

Multi-headed attention

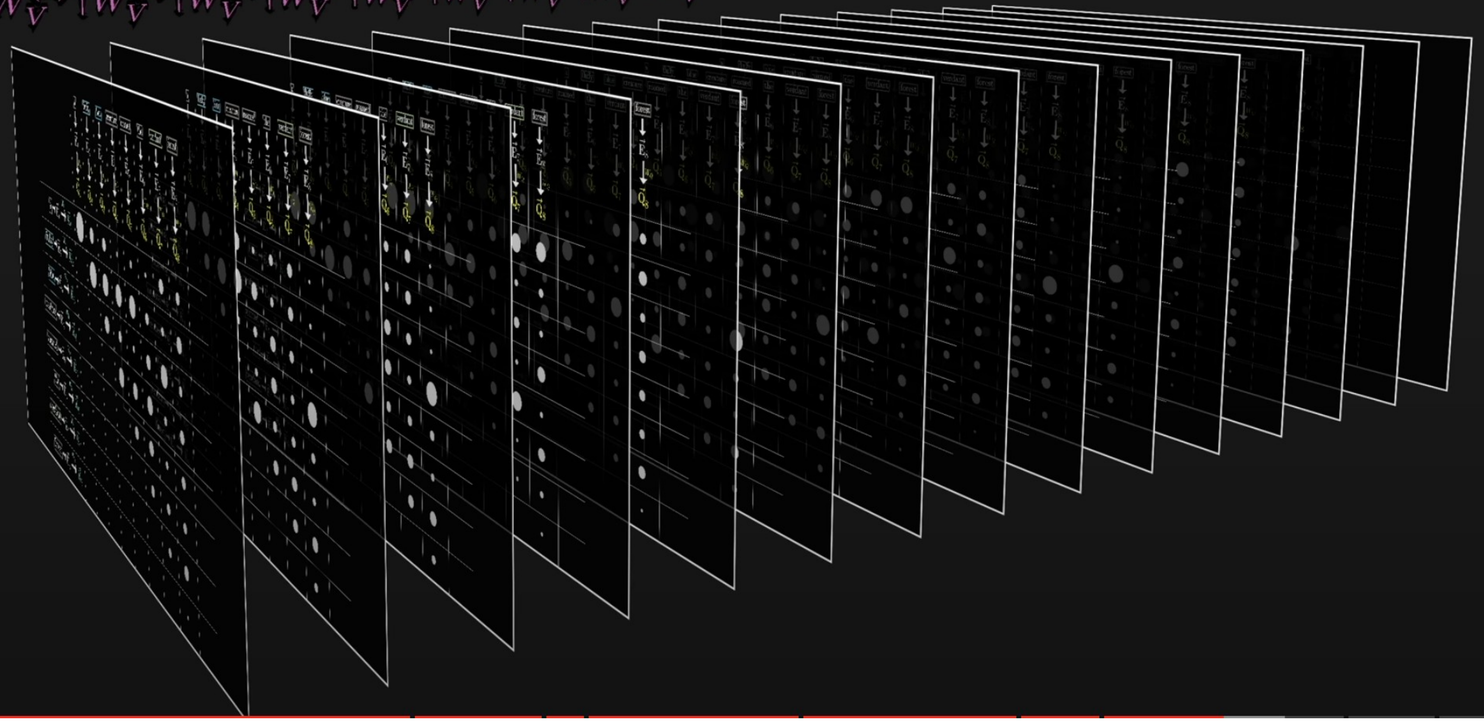
Video

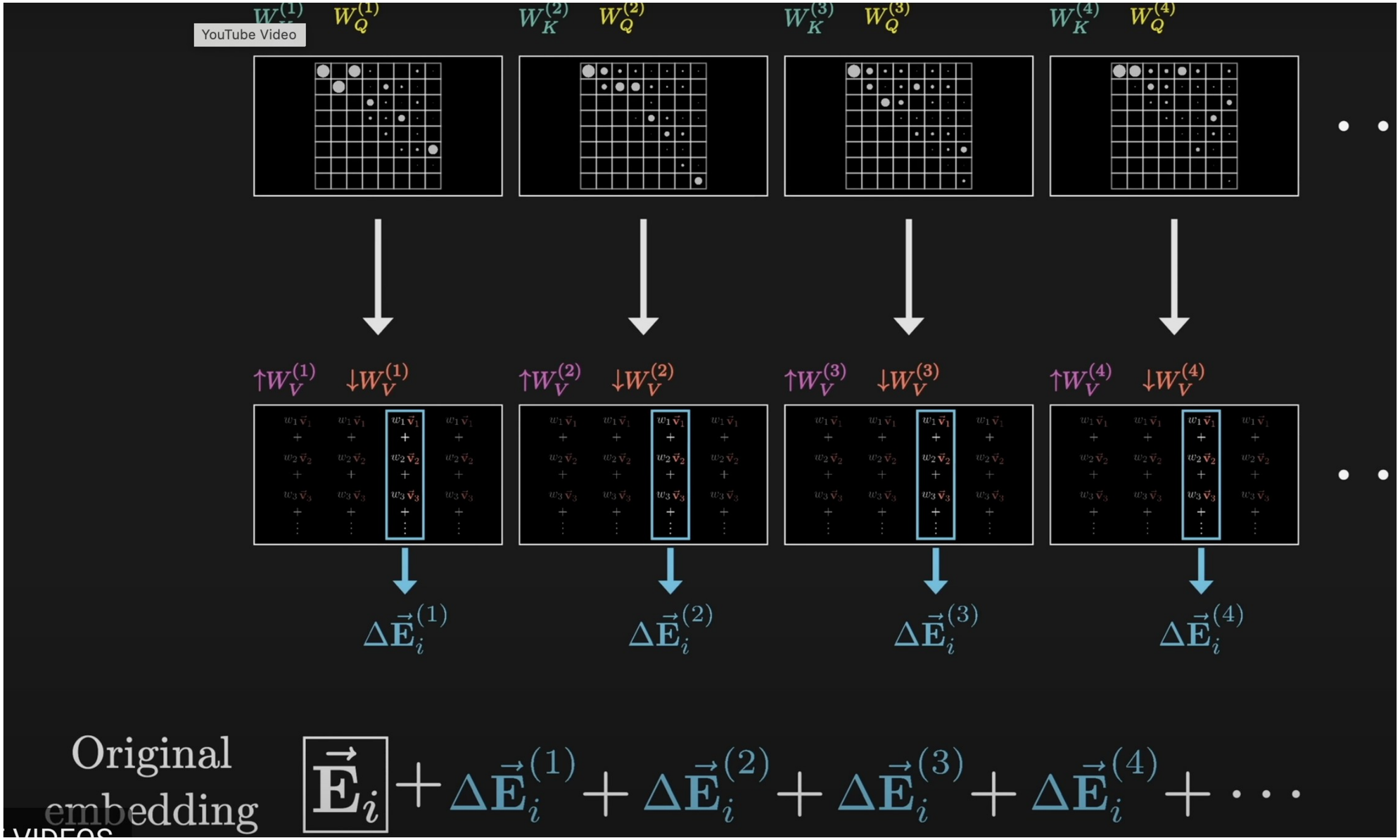
$$W_Q^{(1)}\ W_Q^{(2)}\ W_Q^{(3)}\ W_Q^{(4)}\ W_Q^{(5)}\ W_Q^{(6)}\ W_Q^{(7)}\ W_Q^{(8)}\ W_Q^{(9)}\ \cdots$$

$$W_K^{(1)}\ W_K^{(2)}\ W_K^{(3)}\ W_K^{(4)}\ W_K^{(5)}\ W_K^{(6)}\ W_K^{(7)}\ W_K^{(8)}\ W_K^{(9)}\ \cdots$$

$$\downarrow W_V^{(1)}\ \downarrow W_V^{(2)}\ \downarrow W_V^{(3)}\ \downarrow W_V^{(4)}\ \downarrow W_V^{(5)}\ \downarrow W_V^{(6)}\ \downarrow W_V^{(7)}\ \downarrow W_V^{(8)}\ \downarrow W_V^{(9)}\ \cdots$$

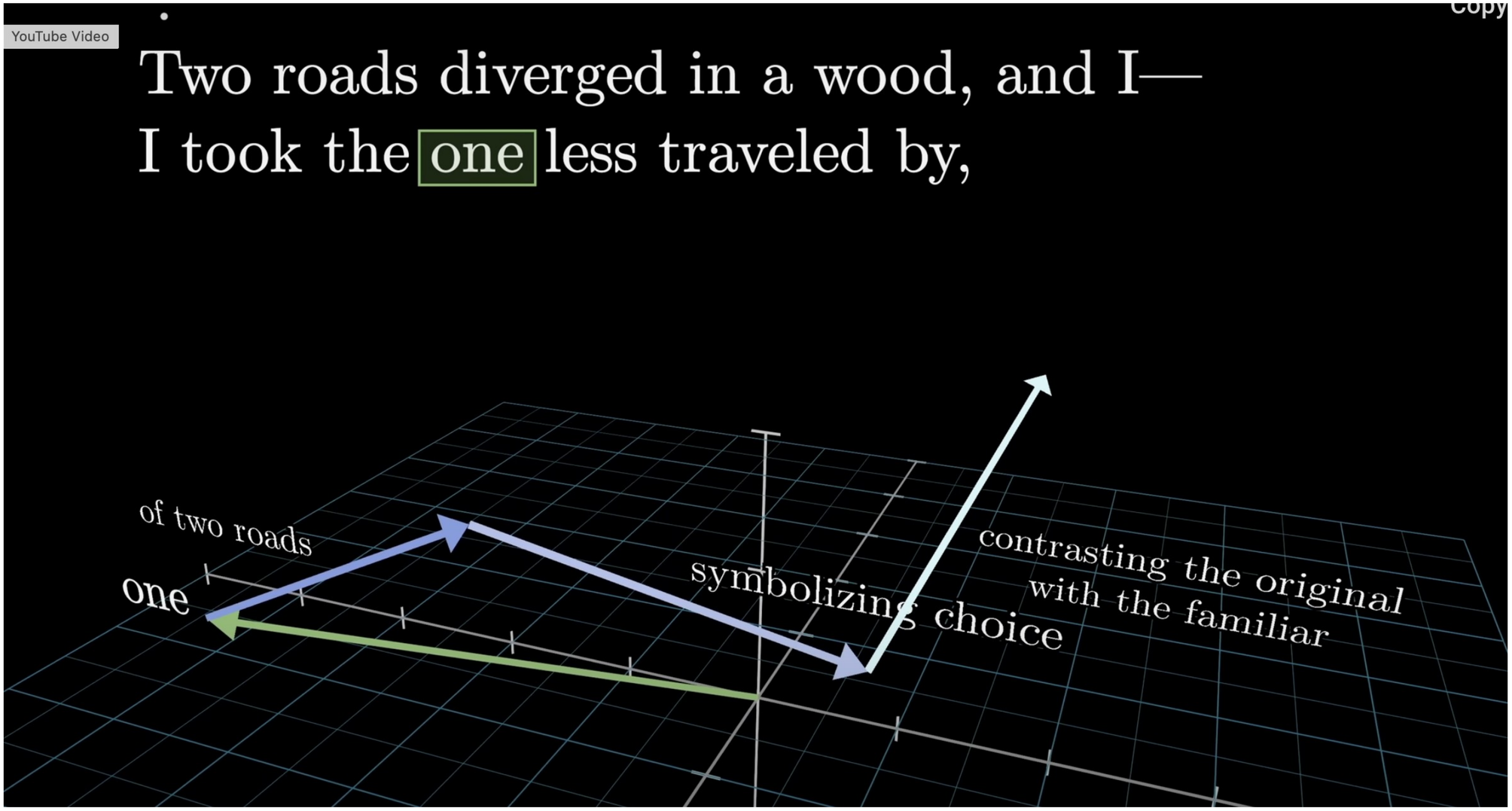
$$\uparrow W_V^{(1)}\ \uparrow W_V^{(2)}\ \uparrow W_V^{(3)}\ \uparrow W_V^{(4)}\ \uparrow W_V^{(5)}\ \uparrow W_V^{(6)}\ \uparrow W_V^{(7)}\ \uparrow W_V^{(8)}\ \uparrow W_V^{(9)}\ \cdots$$

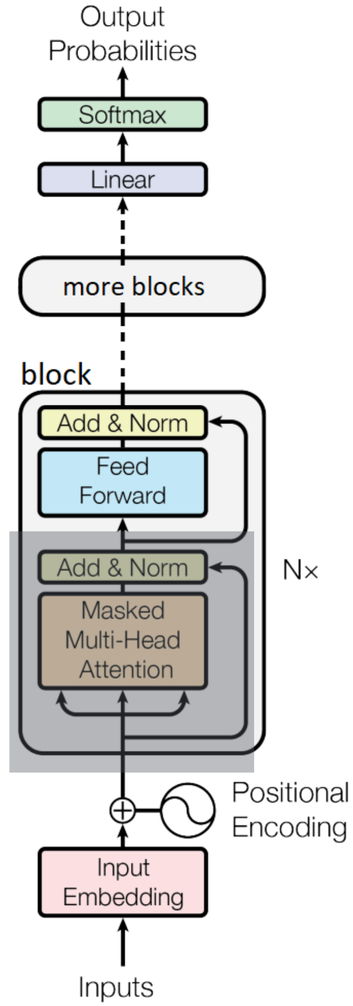




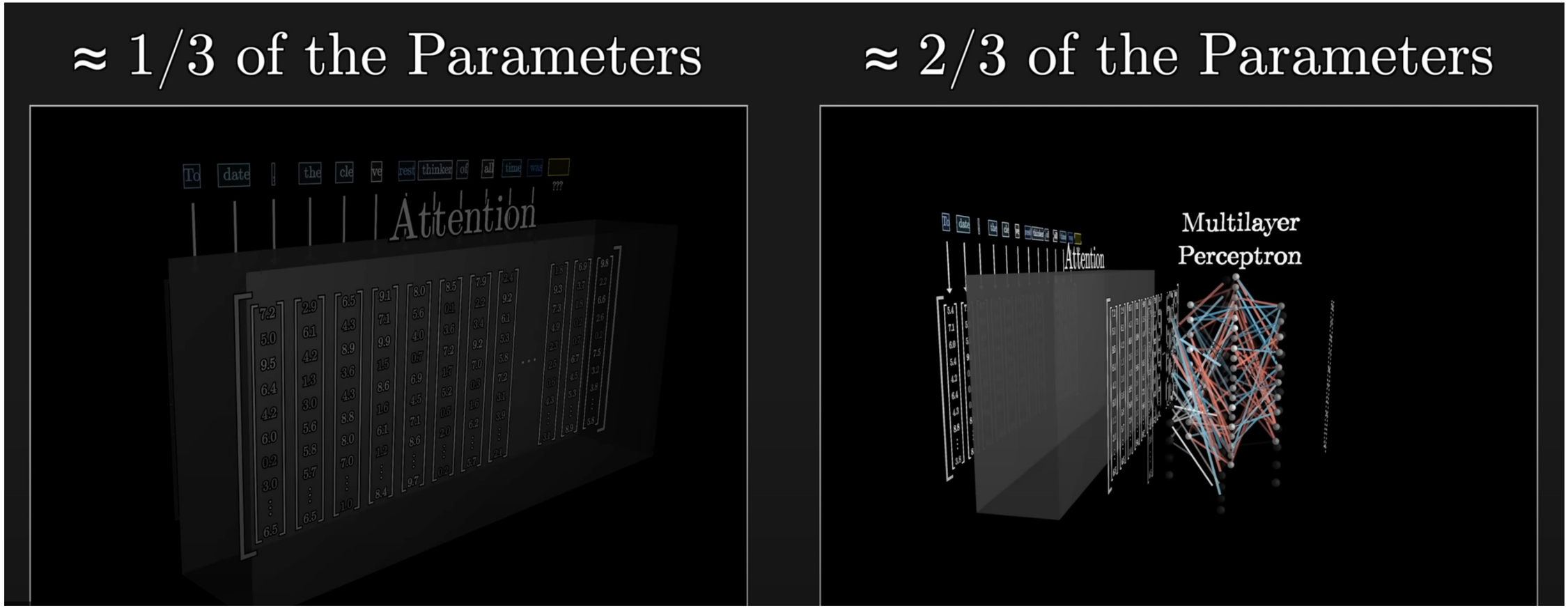
Now, in parallel, we calculate the nudge for each and every token

Remember: we are always adding to the original vector to nudge it

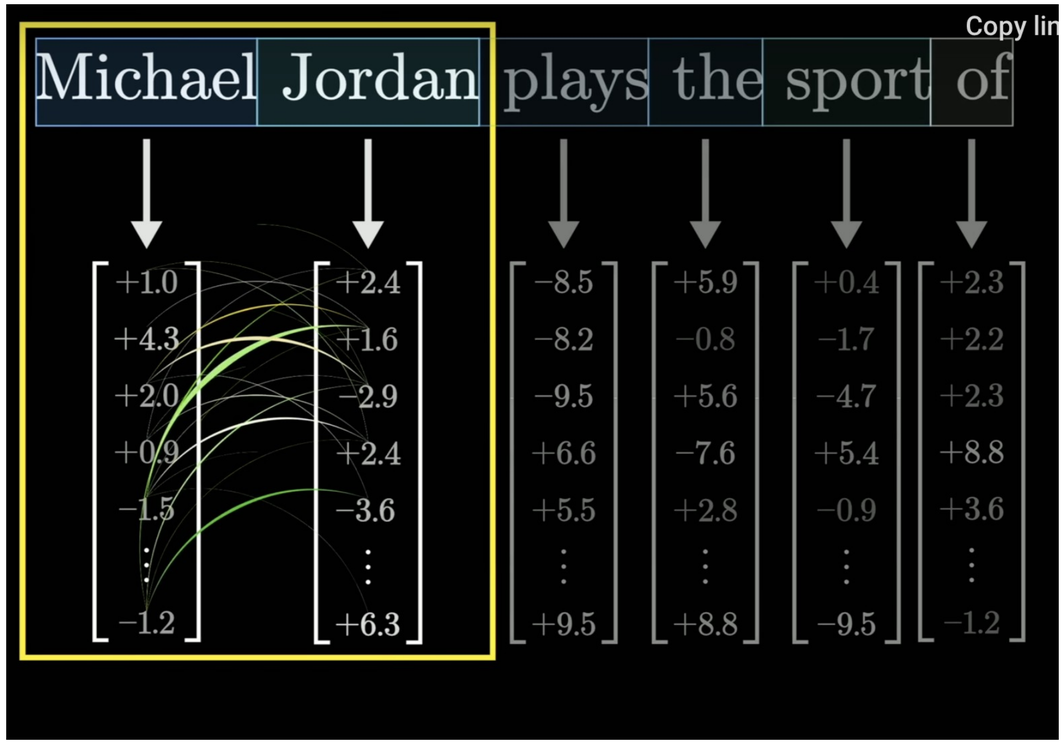


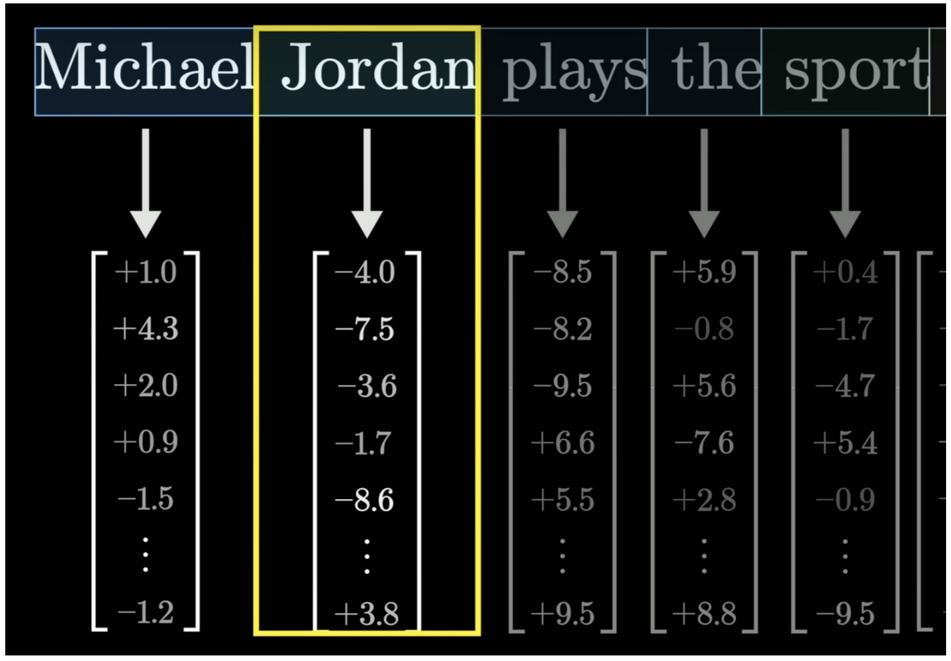


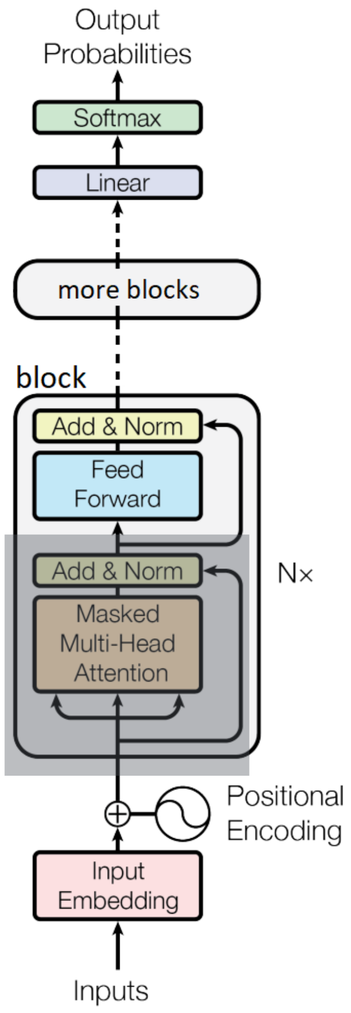
MLP provides the memory

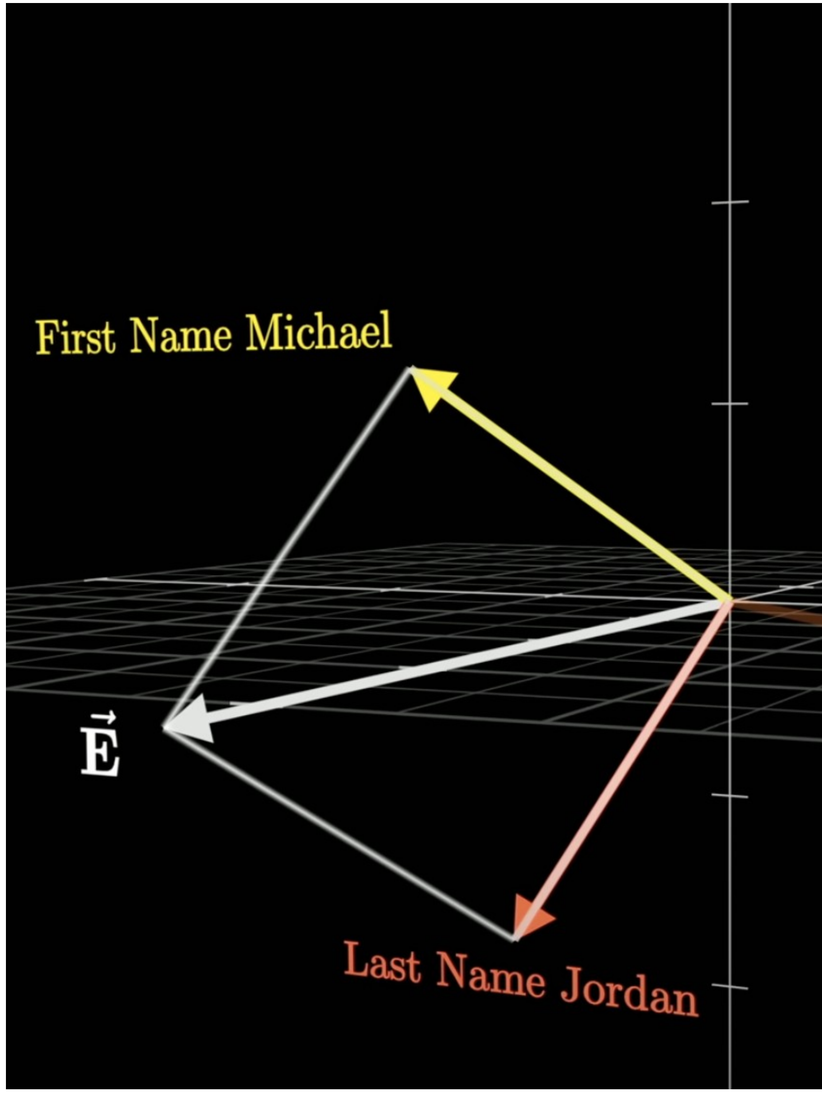


Now that we have the context-nudged token, we want to see what equipping our model with a memory does to that token.

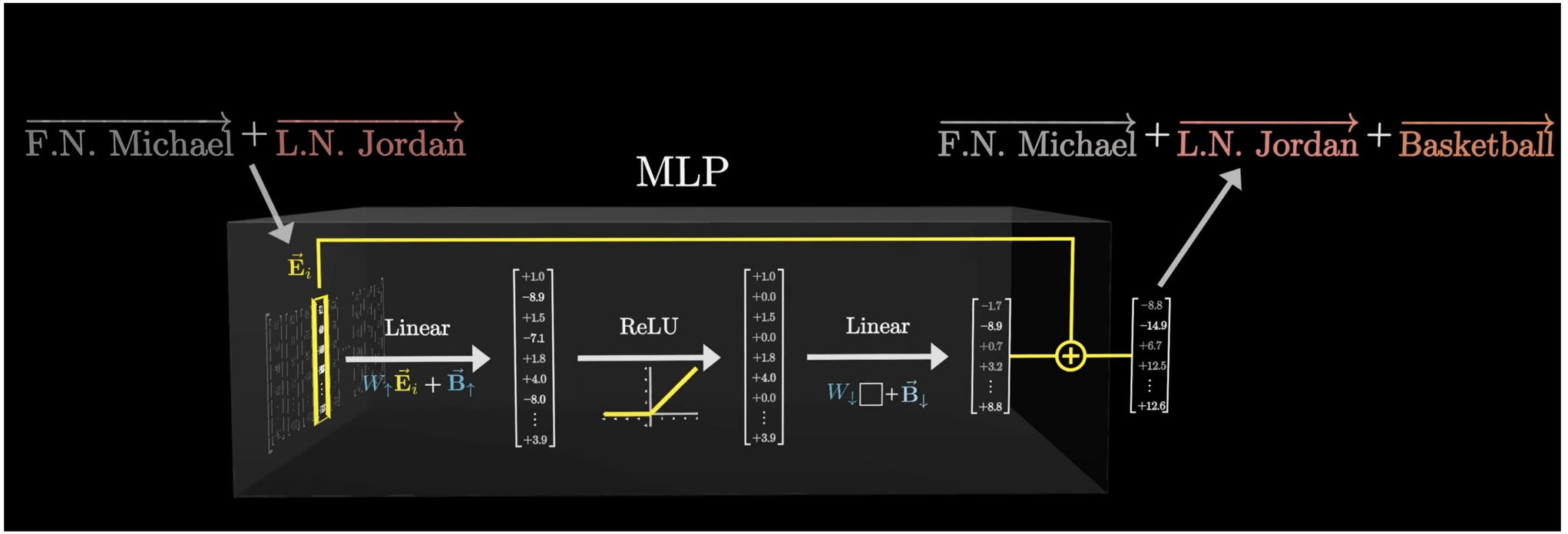


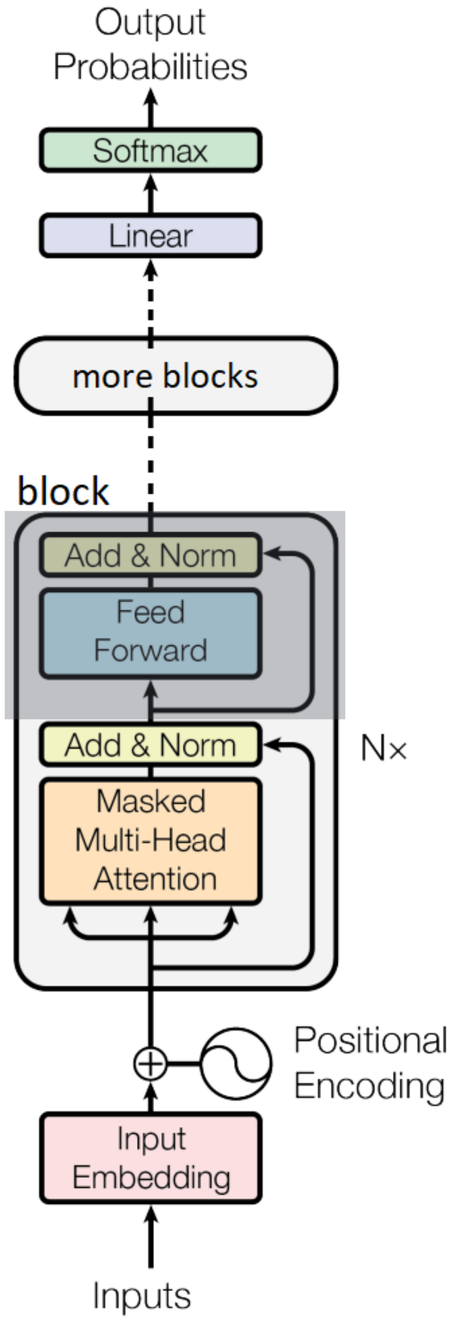




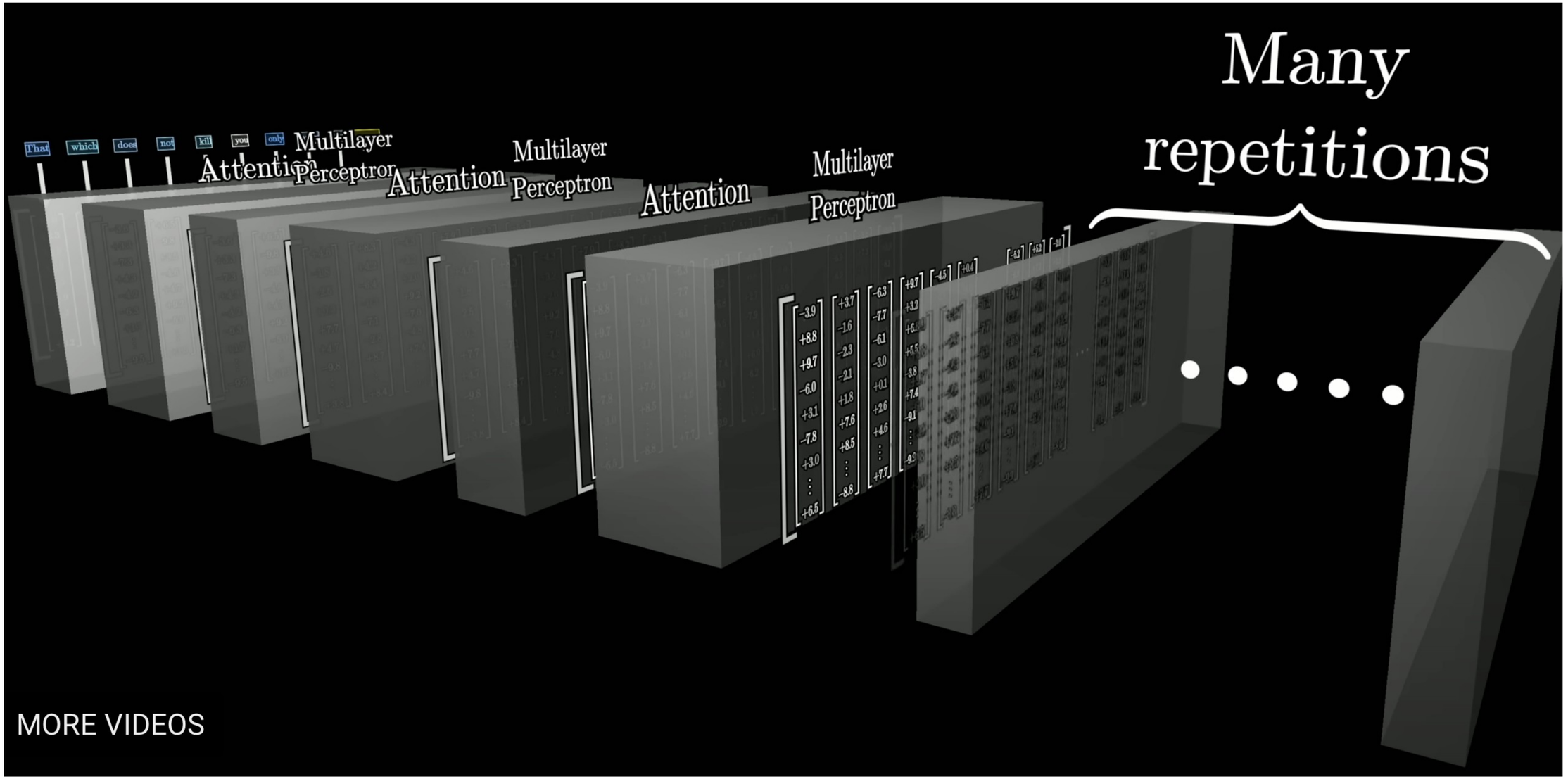


Lets say we want to complete the sentence “Michael Jordan plays the sport of …”. Lets say we went to the attention process, and nudged the embedding of “Jordan”by something that captures the first name “Michael”





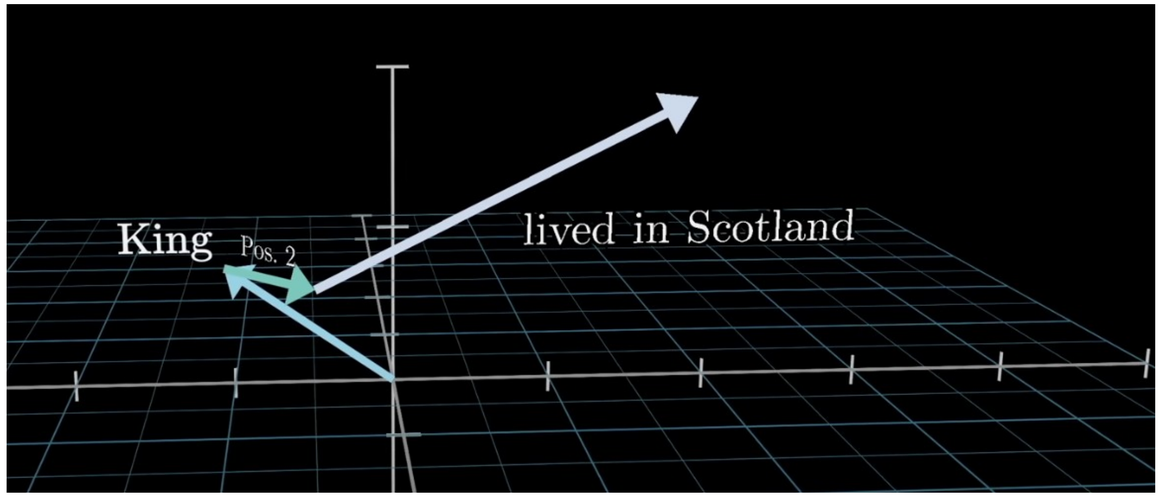
We started with “Jordan” at the second token in the sequence. The attention mechanism added the “Michael” vector onto it. Now the memory mechanism adds the “basketball” o to it, so that the “Jordan” embedding has been nudged into both having Michael picked u via the attention mechanism and basketball picked up via the memory mechanism.



The process of context and memory repeats many times in the transformer, allowing multiple contaxts and memories to be added. This is how gpt like models can seem to be reasoning: they are building in a lot of memory, and the interaction of multiple contexts with this memory seems like they can interpolate between multiple knowledge bases,

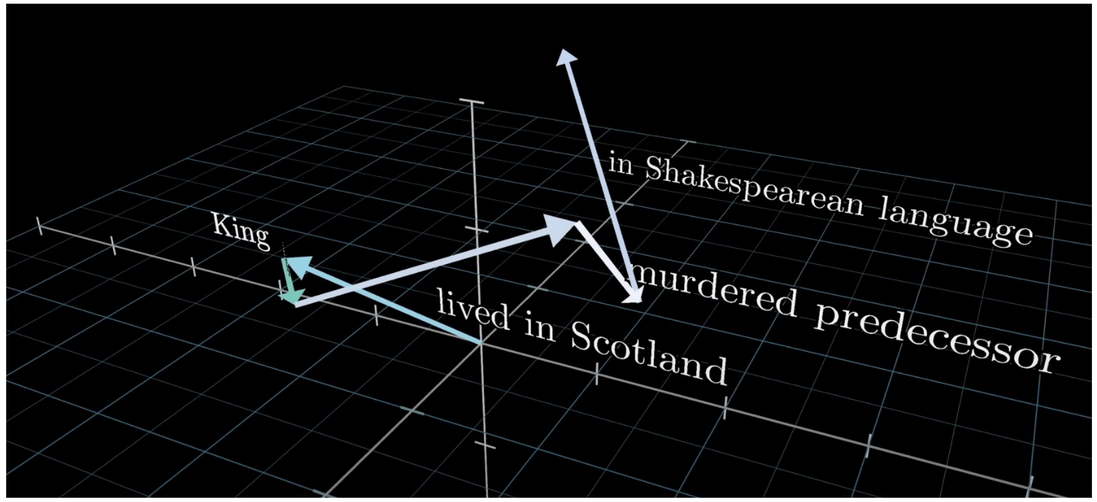
What are we trying to do with embeddings?

The King doth wake tonight and takes his rouse ...



1. We want information to be picked from the current context: drinking, excesses

ATTENTION



2. We want information to come with a memory: Hamlet, Shakespeare, Claudius

MLP

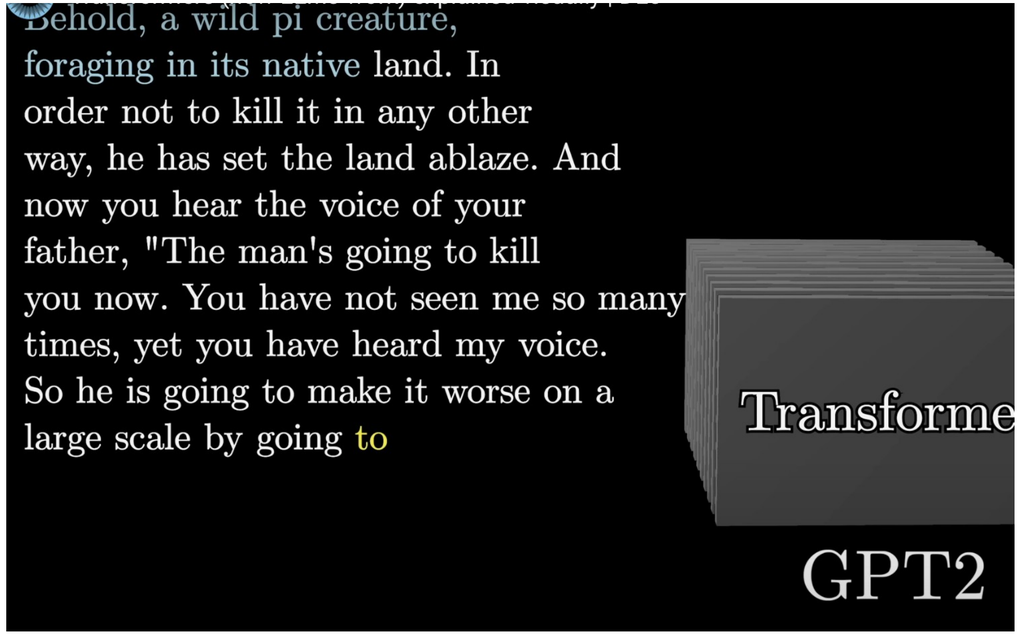
3. We want information to come from multiple contexts.

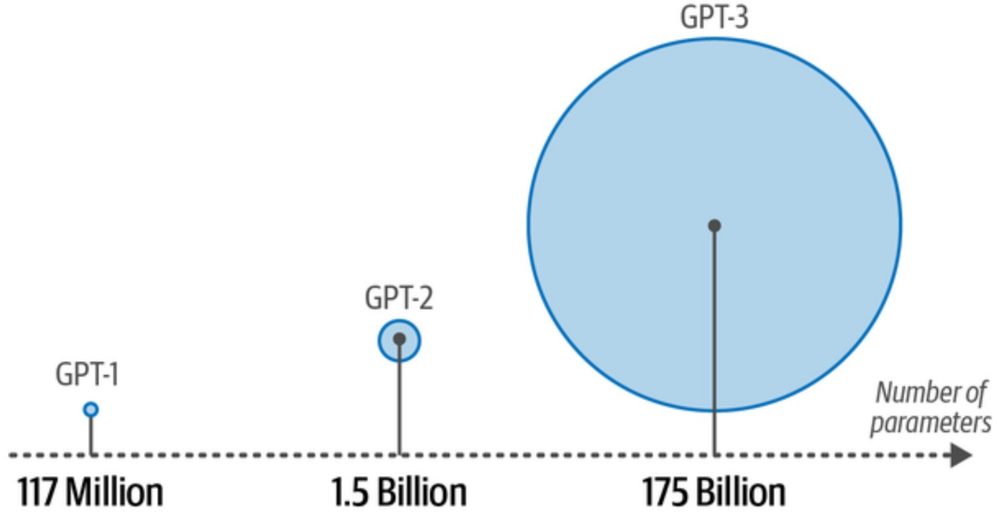
MULTIPLE HEADS

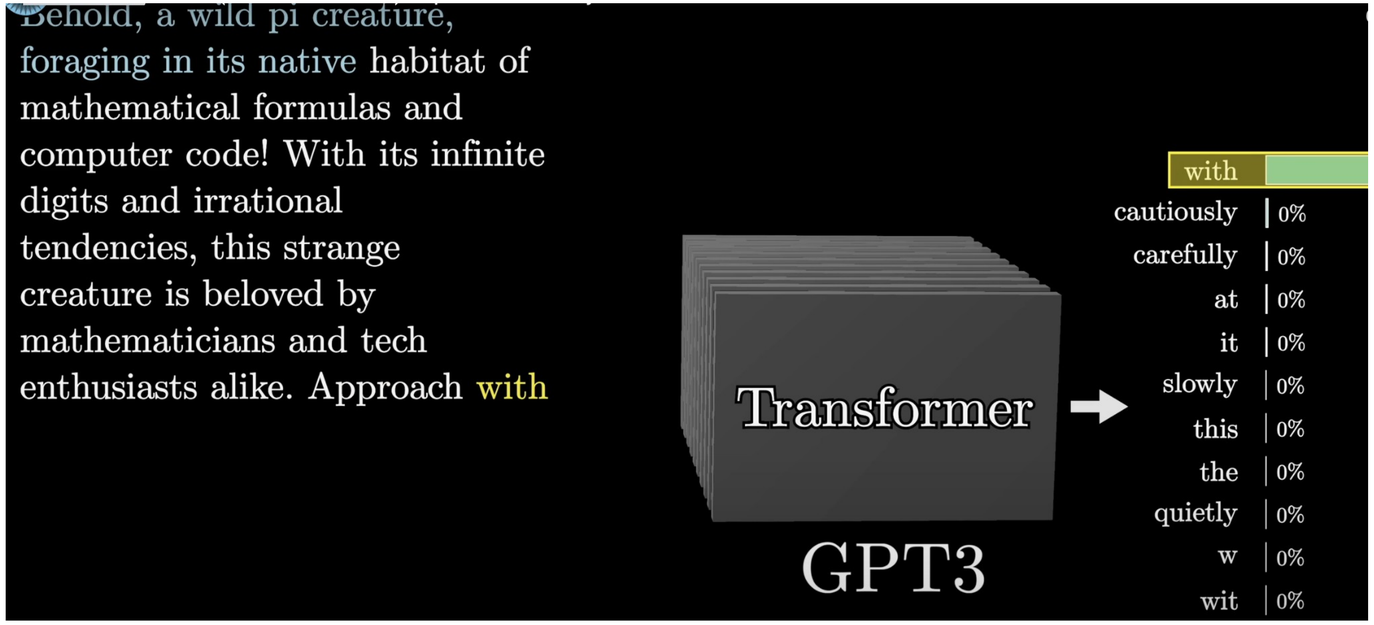
Intelligence?

And so, between the attention and MLP layers, a transformer forms a building block that entwines context and memory, and gives us the notion of an intelligent being interpolating between different gobs of memory in the context of the current discussion. This explains why gpt models seem intelligent, and also hints at why they go off the rails with hallucinations at times.

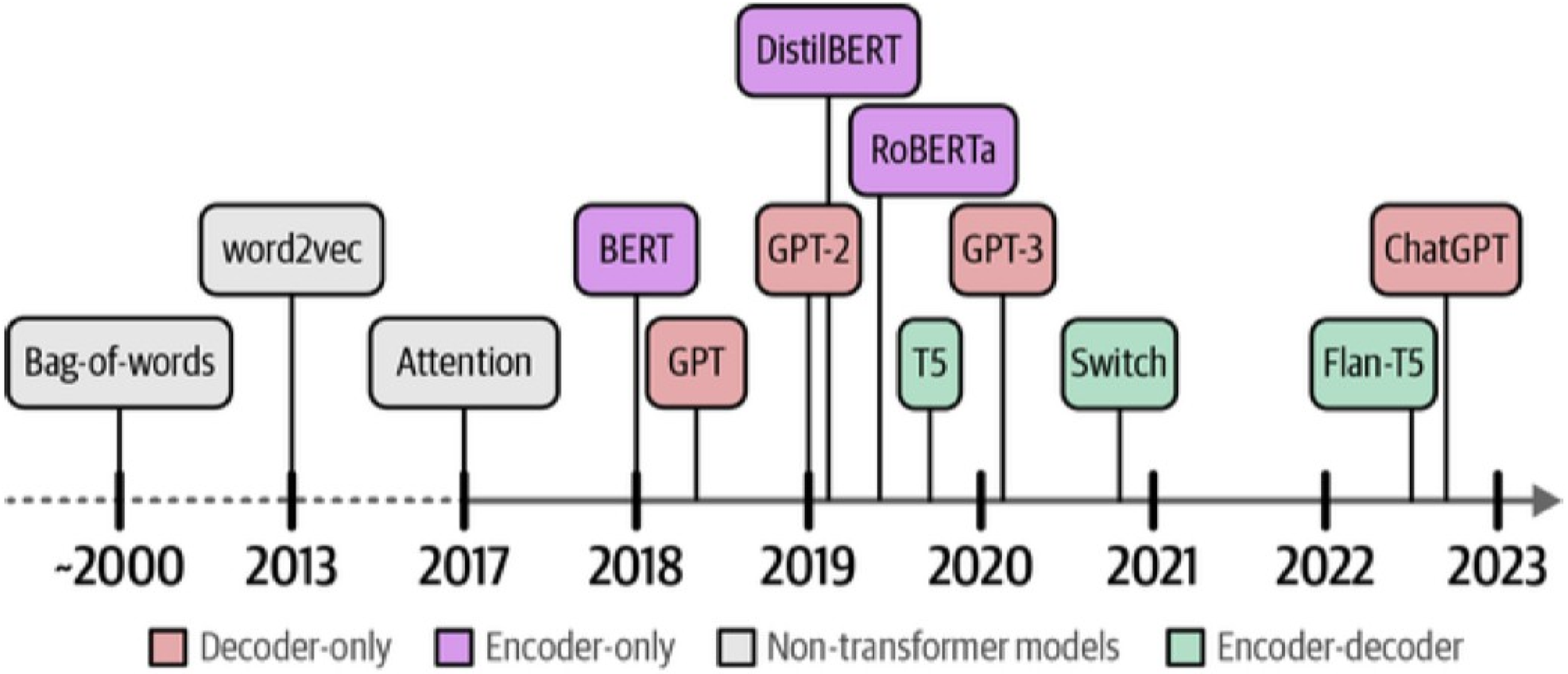
From GPT-2 to GPT-3

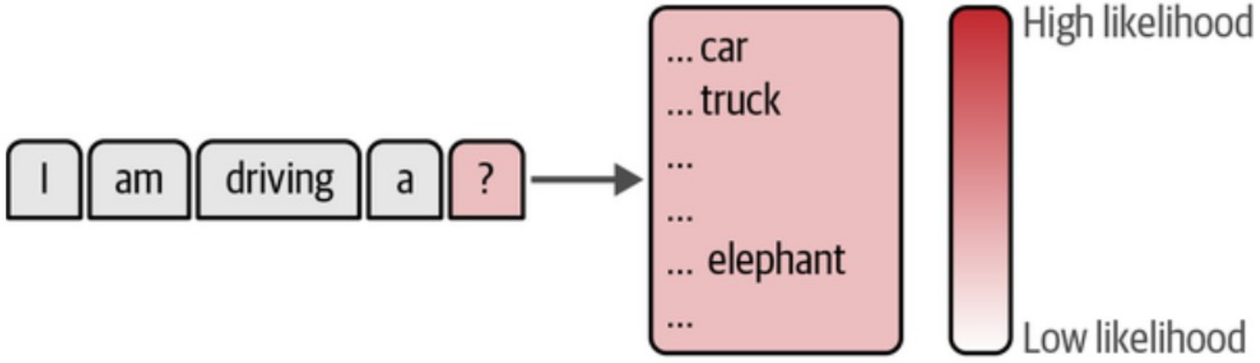


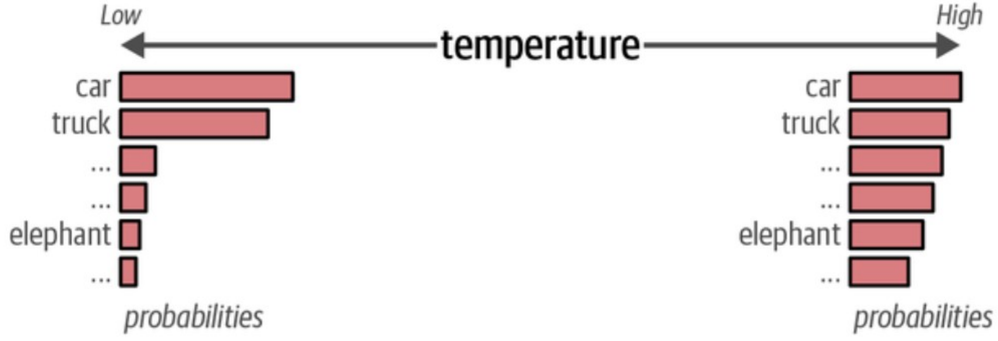


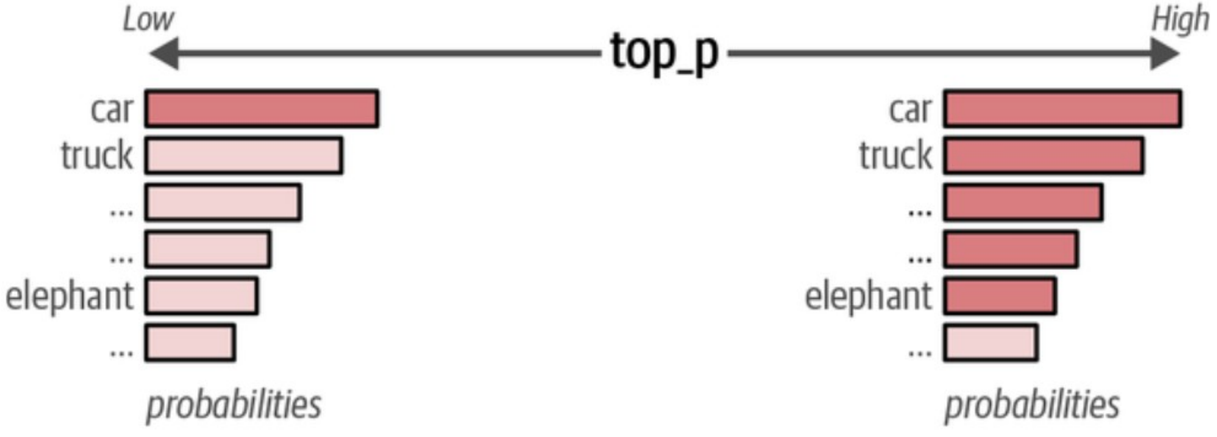


The history of language modeling



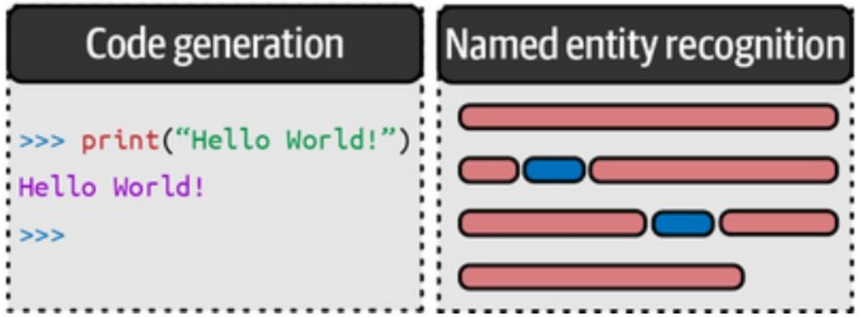




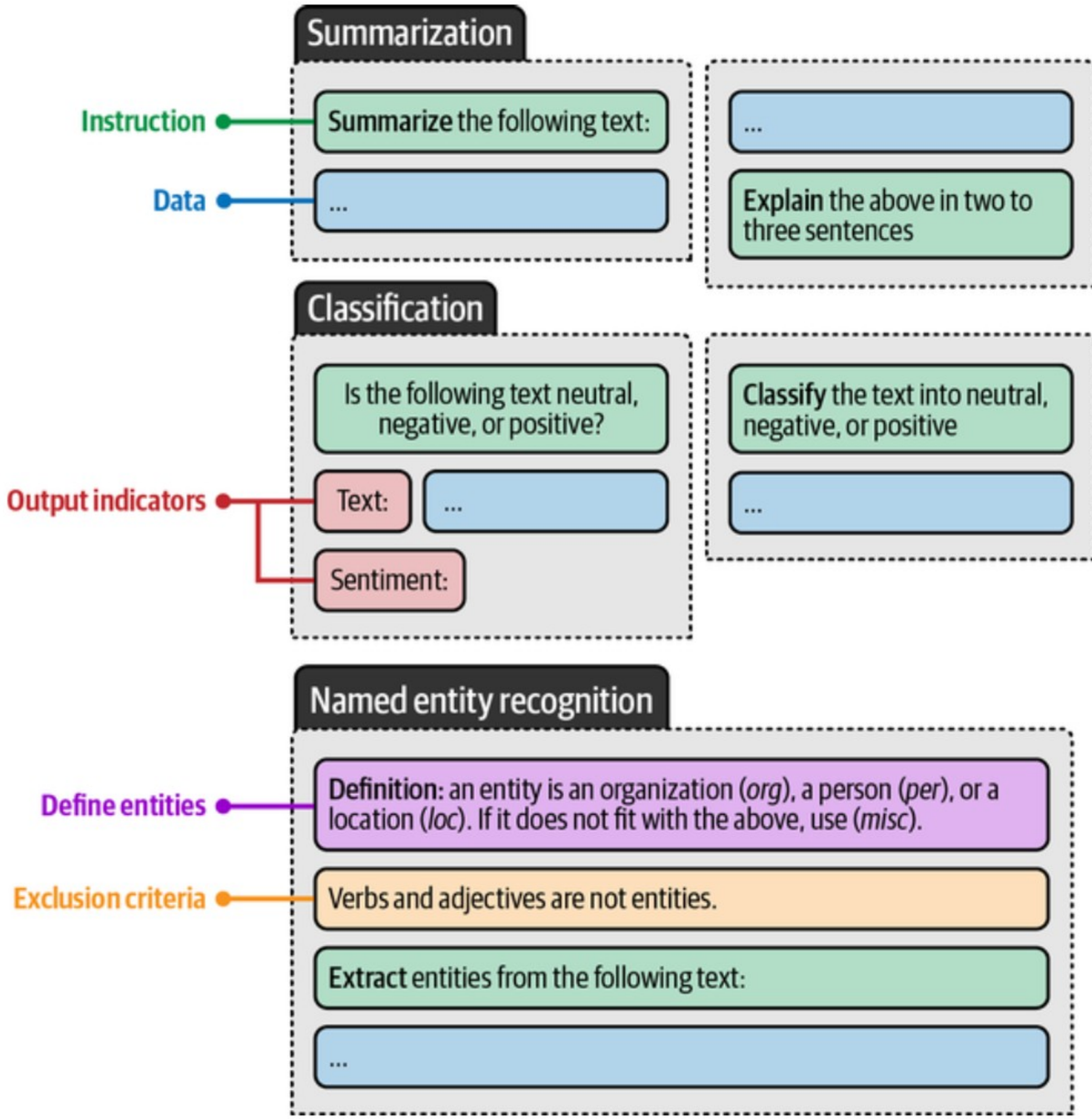


Example use case	`Temperature`	`top_p`	Description
Brainstorming session	High	High	High randomness with large pool of potential tokens. The results will be highly diverse, often leading to very creative and unexpected results.
Email generation	Low	Low	Deterministic output with high probable predicted tokens. This results in predictable, focused, and conservative outputs.
Creative writing	High	Low	High randomness with a small pool of potential tokens. This combination produces creative outputs but still remains coherent.
Translation	Low	High	Deterministic output with high probable predicted tokens. Produces coherent output with a wider

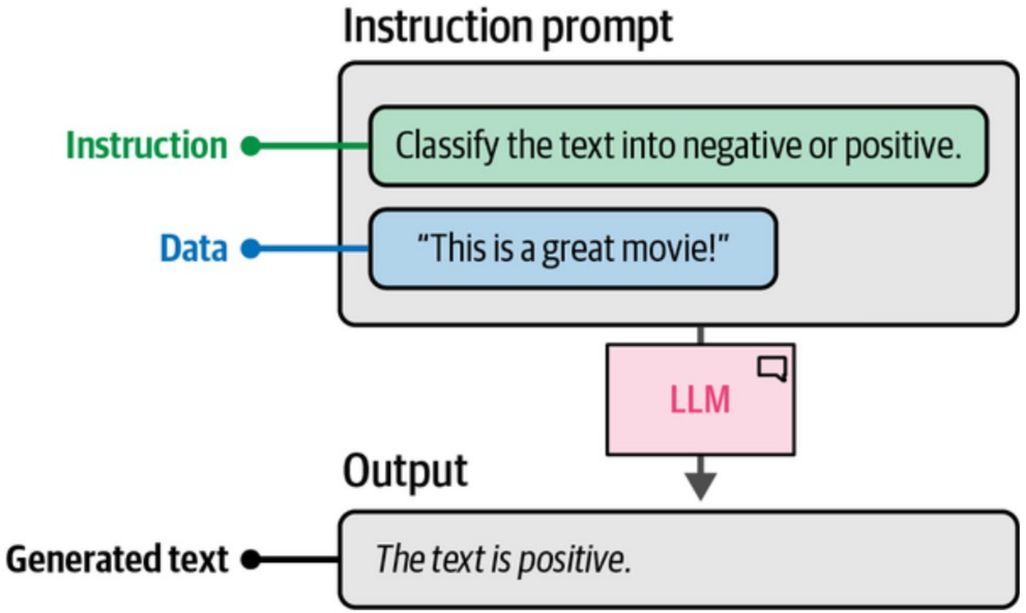


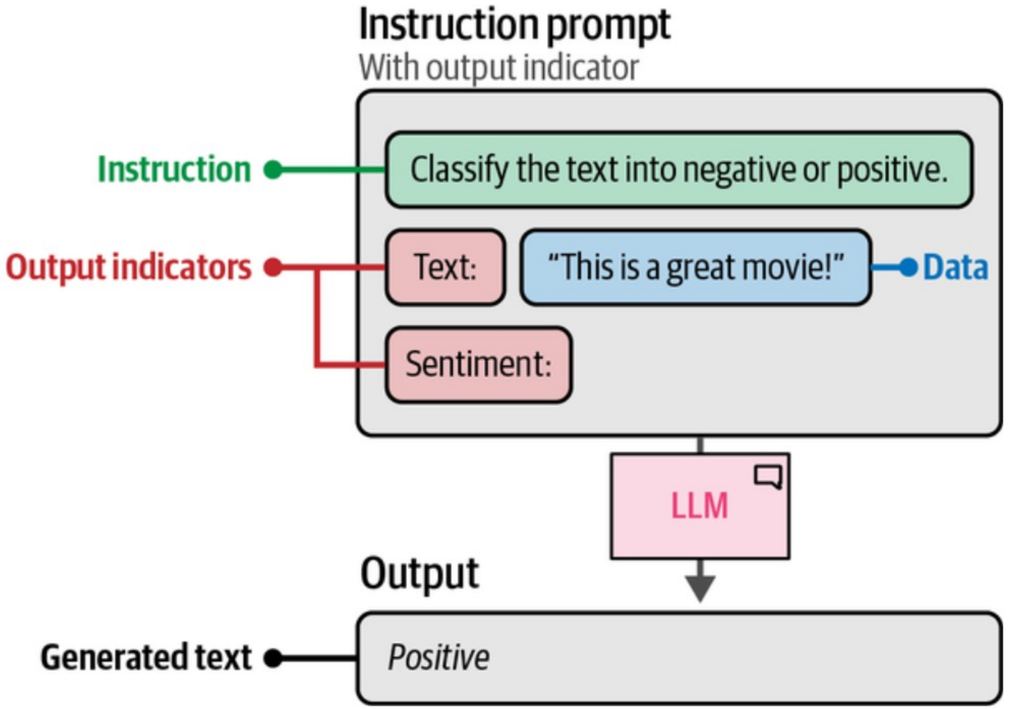


What can you do?

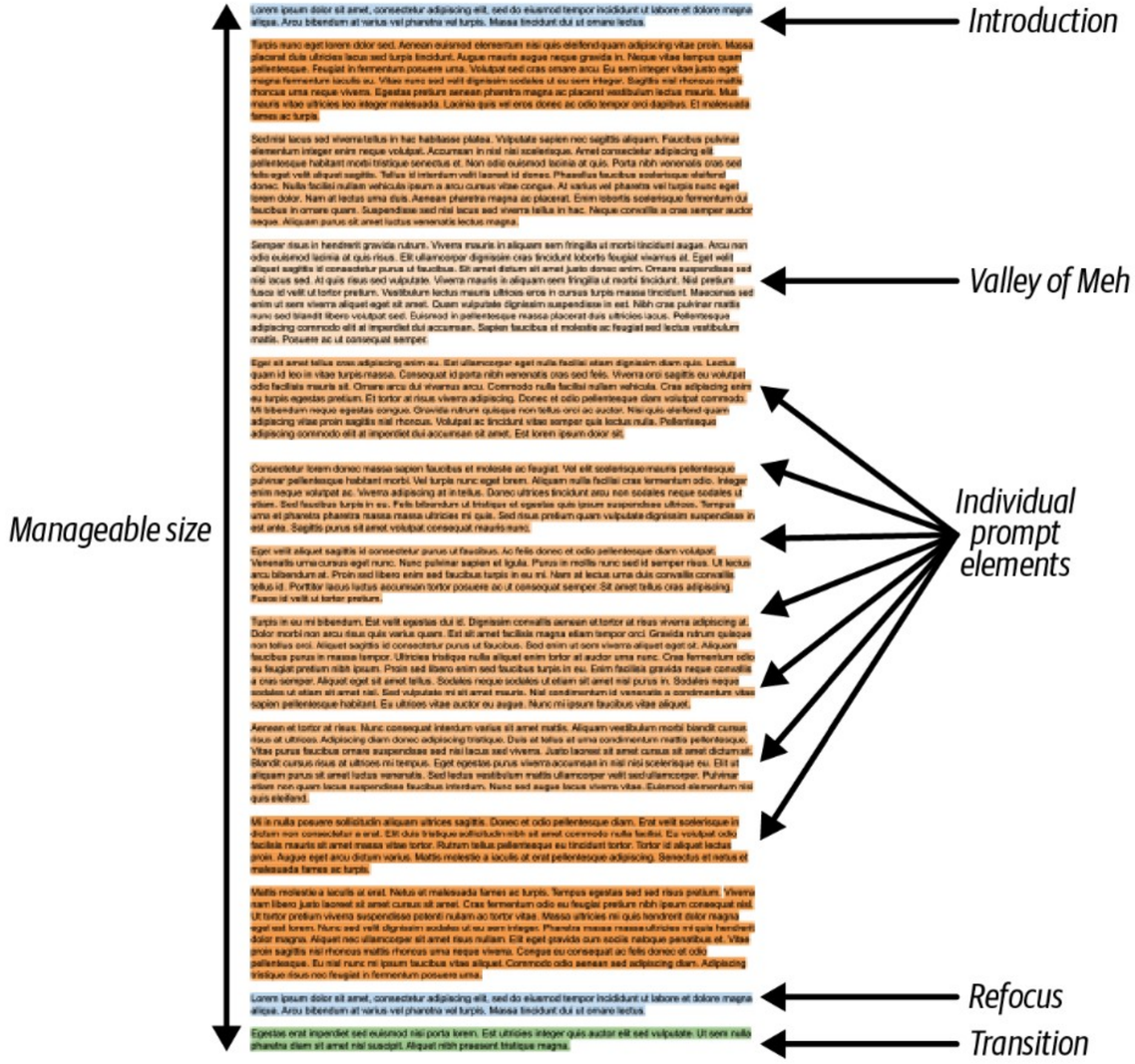


From simple to structured





Prompt Structure



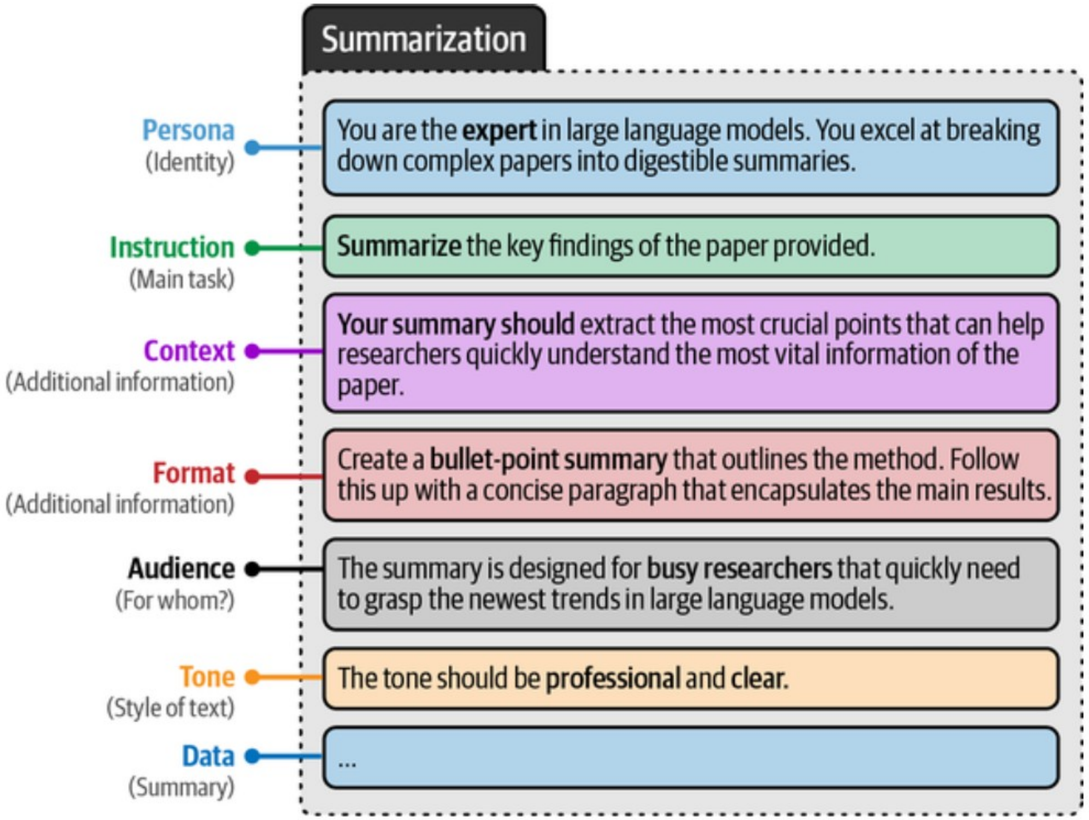
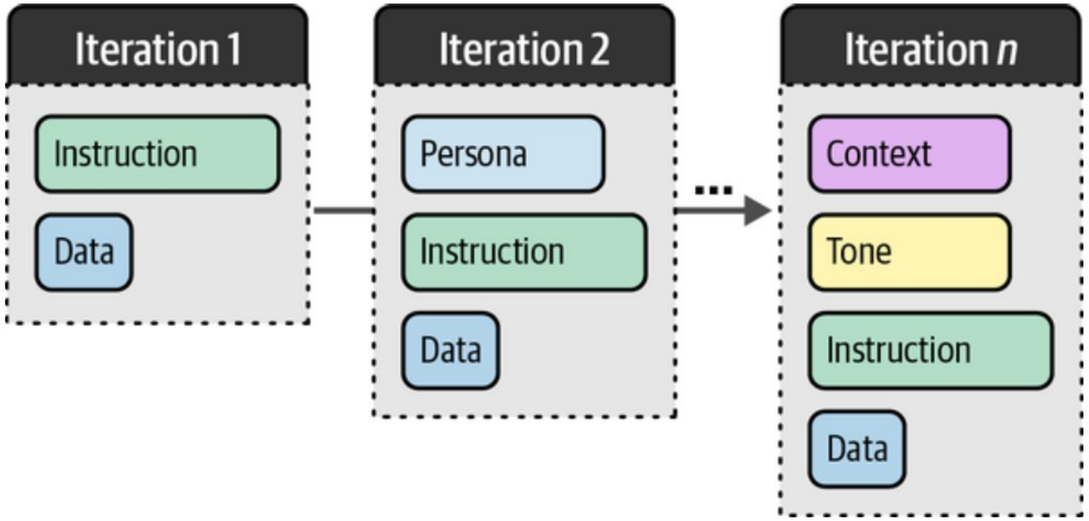


Figure 6-11. An example of a complex prompt with many components.



Zero-shot prompt

Prompting without examples

Classify the text into neutral, negative, or positive.

Text: I think the food was okay.
Sentiment: ...

One-shot prompt

Prompting with a single example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.
Sentiment: **Neutral**

Text: I think the food was okay.
Sentiment:

Few-shot prompt

Prompting with more than one example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.
Sentiment: **Neutral.**

Text: I think the food was great!
Sentiment: **Positive.**

Text: I think the food was horrible...
Sentiment: **Negative.**

Text: I think the food was okay.
Sentiment:

Prompt Structure

Editing: Leverage Ratio Name And Value (prompt_leverage_ratio_name_and_value.txt)

```
{prompt_financial_covenant}

Now, finally, on the last line of your response, outside of the python block, you MUST return to
the user the maximum value of any leverage ratios (not coverage ratios, ie variables with the
substring leverage_ratio in them) in the python block. The format is `leverage_ratio_value=
<value>`. There may be multiple name-value pairs, or a list of dictionaries each having a period
and a ratio, from which you will need to compute the maximum. If one of them is None, consider
that value to be 0.0. Put a comment (#) sign after the answer, and put the name of the python
variable, or dictionary period there that had the max value. This is the format:
`leverage_ration_name="<value>". Then, after another comment (#) sign, in a python list, put the
python variables and values you used, or put in there the list of dictionaries you used for this
maximization.

Here is one example with variables:

leverage_ratio_value = 10.0 # leverage_ratio_name="consolidated_leverage_ratio_2" #
[consolidated_leverage_ratio_1=5.0/1 , consolidated_leverage_ratio_2=10.0/1.0]

Here is another with the periods list of dictionaries:

leverage_ratio_value = 6.0 # leverage_ratio_name="First Lien Leverage Ratio" # [{"period": "on or
prior to April 2, 2022", "ratio": "6.00:1.00"}, {"period": "after April 2, 2022 through and
including April 1, 2023", "ratio": "5.75:1.00"}, {"period": "after April 1, 2023 through and
including March 30, 2024", "ratio": "5.50:1.00"}, {"period": "after March 30, 2024 through and
including March 29, 2025", "ratio": "5.25:1.00"}, {"period": "after March 29, 2025", "ratio":
"5.00:1.00"}]

Make sure this value is on the LAST LINE, OUTSIDE of the python block.
```

Editing: Financial Covenant (prompt_financial_covenant.txt)

```
{content}
Extract multiple values from the above text. Each extraction is a variable equal to something else in python.

{include}

Extract the following quantities:
- Leverage Ratio name as `leverage_ratio_name` and value as `leverage_ratio_value`(extract as separate variables, keep
value of `leverage_ratio_name` as string). If there is more than one value because the values are different in
different time periods, the value of `leverage_ration_value` should be a list with each element a dictionary with keys
`period` and `ratio`.
- Coverage Ratio name and value (extract as separate variables, keep name as string)
- Minimum Liquidity
- Minimum EBITDA
- Maximum Capital Expenditure
- Amount of Letter of Credit Excluded from Springing Trigger Calculation
- Springing Trigger - Threshold in Percent
- Springing Trigger Condition
- Cash Applicable to Liquidity/Cash Netting
- Cash Netting Maximum
- Cov-lite Deal
```

Editing: Include (include.txt)

```
In the terms to be extracted, the first writing in each term is the name of the term. In the brackets I provide alternate ways that
this term may have been represented in the text. Use the bracketed text to search using this alternate language but define the
variable based on the term. For example `- Equity Cure - Annual Usages (or maximum number of cure rights allowed within a year)`
would lead to variable `equity_cure_annual_usages`, even if the extraction you search for in the document is different, ie "maximum
number of cure rights allowed within a year".

For example let us consider "Assumed Indebtedness Ratio Test". This will usually be given as a quantity followed by a ratio. Write
the quantity as a python programming language lowercase variable with underscore between its components and the ratio in a fraction.
For example `assumed_indebtedness_ratio_test = "fixed_charge_coverage_ratio >= ratio`. Convert superlatives such as 'no less than' to
their corresponding mathematical operator...in this example '>='. If there are no comparion simply say `variable = number`. If there
is an or in the definition, reflect that into the python programming language.

Use standard python functions such as max, min when needed. Here is an example of how you should return, consistent with everything
being in legal python: general_limit_debt = max(125000000, 0.5 * four_quarter_consolidated_ebitda). Dont do something like
assumed_indebtedness_basket = max(820000000, 0.60 * 'Consolidated EBITDA' but rather do assumed_indebtedness_basket = max(820000000,
0.60 * 'consolidated_ebitda'. All variables must be lower cased and spaces converted to underscores.

As another example consider '$45 million in any calendar year, with unused amounts in any calendar year being permitted to be carried
over to succeeding calendar years up to a maximum of $60 million in any calendar year'. This would translate to
equity_repurchase_basket_rp(t) = max(60_000_000, equity_repurchase_basket_rp(t) + remaining_equity_repurchase_basket(t-1)) and
remaining_equity_repurchase_basket(t) = 45_000_000 - equity_repurchase_basket(t) . Do not be afraid of using two formulas to capture
the logic if needed.

If you cannot convert into a pythonic form, just return english. For example, dont EVER do:
subordinated_unsecured_debt_basket =
"indebtedness_or_disqualified_stock_to_satisfy_and_discharge_indebtedness_permitted_to_be_incurred_hereunder_in_the_form_of_senior_un
secured_notes", but rather, return an english string as the value: subordinated_unsecured_debt_basket = "Indebtedness or Disqualified
Stock to satisfy and discharge Indebtedness permitted to be incurred hereunder in the form of senior unsecured notes"

I will repeat the previous para because its important. If you are choosing text as a right hand side value, save it as a python
string, not as an underscore jointed variable (even if you put quotes around it).

Where there is a maximum or minimum of a dollar amount and a fraction of another quantity like ebitda, further extract the dollar
amount as a variable of the form `variablename_dollar_amount` and the fraction as a variable of the form `variablename_fraction`.
This should be only the numerical value of the function, not the thing it modifies.

If you cannot find the value for a variable, return 'variable = None'

Where your RHS is a comparison, make sure its proper python and dont make it a string.

If there are multiple values for one extraction, put them in a python list.

Do not return anything outside ```python and ``` tags, unless I explicitly ask you to in text below.
```

Prompt Versioning and Feedback

Extracto Primo | Runs | Prompt Templates | Prompt Editor | 2 runs | Reset Filters | Drill Down

Filter by Prompt Version: All Versions

Filter by Extraction Name: Tearsheet_Extraction/leverage_ratio_nam

Filter by Input File: HLLY - Amendment no 1.data_point.json.p

Filter by Model: azure-gpt-4o

ID	PROMPT VERSION	MODEL	PROMPT	FILENAME	RESPONSE	ACCEP
76	1	azure-gpt-4o	**Tearsheet_Extraction/leverage_ratio_** Leverage Ratio name and value: Maximum Total Leverage Ratio ≤	HLLY - Amendment no 1.data_point.json.processed	```python leverage_ratio_name = "Maximum Total Leverage Ratio" leverage_ratio_value = [{"period":	Selec
83	2	azure-gpt-4o	**Tearsheet_Extraction/lever** Leverage Ratio name and val Maximum Total Leverage Rat			

← Back | Run #83 | **Filters:** EXTRACTION: Tearsheet_Extraction/leverage_ratio_name_and_value • FILE: HLLY - Amendment no 1.data_point.json.processed • MODEL: azure-gpt-4o | ← Prev 2 of 2 | Next →

Extraction Name

Tearsheet_Extraction/leverage_ratio_name_and_value

Input File

HLLY - Amendment no 1.data_point.json.processed

Response: `leverage_ratio_value = 5.50 # leverage_ratio_name="Maximum Total Leverage Ratio" [{"period": None, "ratio": "5.50:1.00"}]`

```
```python
leverage_ratio_name = "Maximum Total Leverage Ratio"
leverage_ratio_value = [{"period": None, "ratio": "5.50:1.00"}]

coverage_ratio_name = "Interest Coverage Ratio"
coverage_ratio_value = "1.65:1.00"

minimum_liquidity = 45_000_000
minimum_ebitda = None
maximum_capital_expenditure = None
amount_of_letter_of_credit_excluded_from_springing_trigger_calculation = None
springing_trigger_threshold_in_percent = None
springing_trigger_condition = None
```

Editing: Leverage Ratio Name And Value (prompt_leverage_ratio_name_and_value.txt)

```
{prompt_financial_covenant}

Now, finally, on the last line of your response, outside of the python block, you MU
the user the maximum value of any leverage ratios (not coverage ratios, ie variable
substring leverage_ratio in them) in the python block. The format is `leverage_ratio
<value>`. There may be multiple name-value pairs, or a list of dictionaries each ha
and a ratio, from which you will need to compute the maximum. If one of them is Non
that value to be 0.0. Put a comment (#) sign after the answer, and put the name of
variable, or dictionary period there that had the max value. This is the format:
`leverage_ration_name="<value>". Then, after another comment (#) sign, in a python
python variables and values you used, or put in there the list of dictionaries you
maximization.

Here is one example with variables:

leverage_ratio_value = 10.0 # leverage_ratio_name="consolidated_leverage_ratio_2" #
[consolidated_leverage_ratio_1=5.0/1 , consolidated_leverage_ratio_2=10.0/1.0]

Here is another with the periods list of dictionaries:

leverage_ratio_value = 6.0 # leverage_ratio_name="First Lien Leverage Ratio" # [{"p
prior to April 2, 2022", "ratio": "6.00:1.00"}, {"period": "after April 2, 2022 thr
including April 1, 2023", "ratio": "5.75:1.00"}, {"period": "after April 1, 2023 th
including March 30, 2024", "ratio": "5.50:1.00"}, {"period": "after March 30, 2024
including March 29, 2025", "ratio": "5.25:1.00"}, {"period": "after March 29, 2025"
"5.00:1.00"}]

Make sure this value is on the LAST LINE, OUTSIDE of the python block.
```

Run Information

Prompt Version: 2

Model: azure-gpt-4o

Command Line:

```
uv run erun.py llm -t Tearsheet_Extraction --prompt-file Prompts/Tearsheet_Extraction/prompt_leverage_ratio_name_and_value.txt -e "https://cc-openai-dev-1.openai.azure.com/openai/deployments/gpt-4o/chat/completions?api-version=2024-10-21" -d "data/tsai-field-lookup-dictionary.json" -m azure "Leverage Ratio name and value"
```

Timestamp: 4/13/2025, 7:52:47 PM

Evaluation

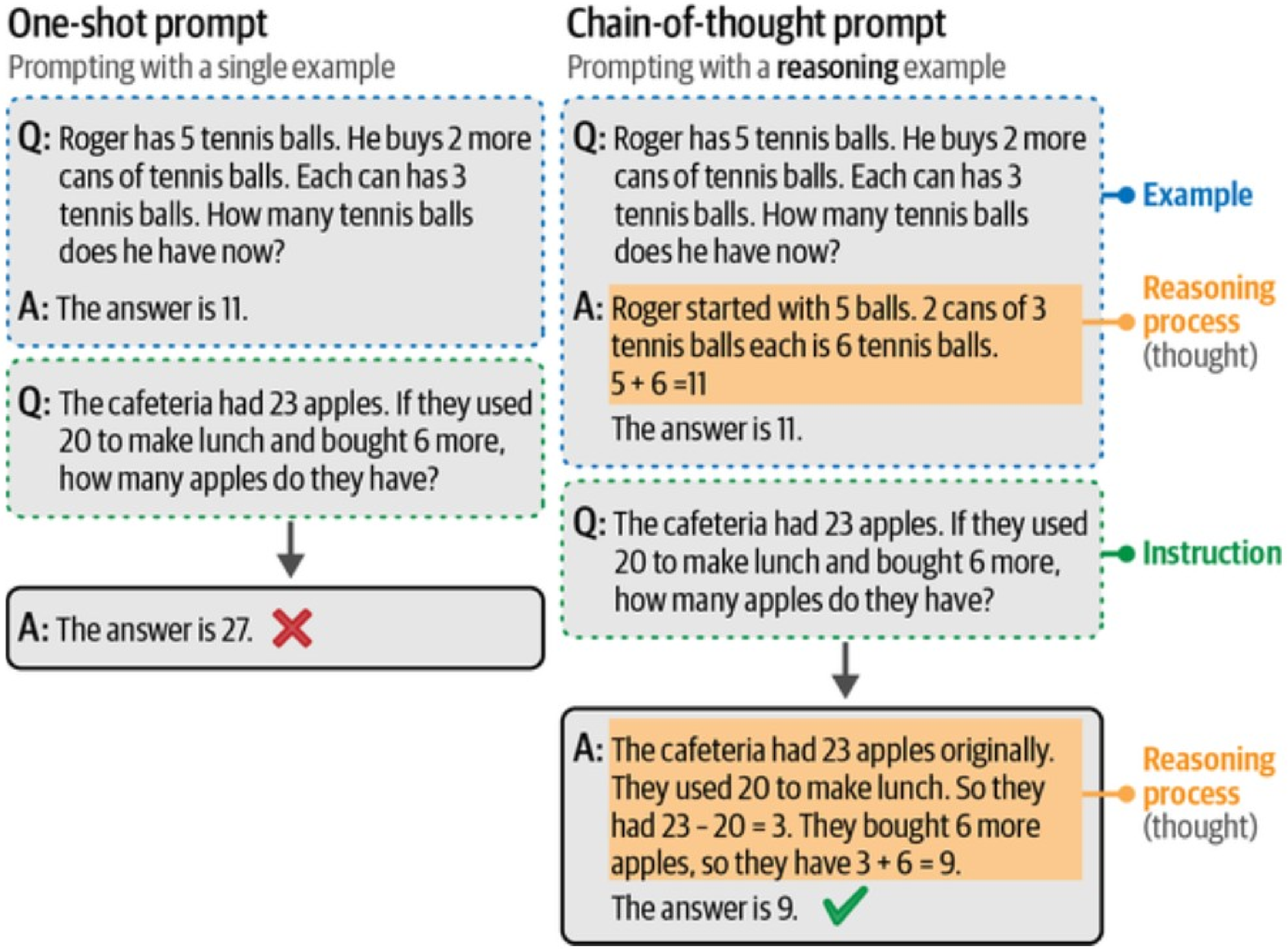
Acceptable: Select...

Explanation: Enter explanation...

Save Changes

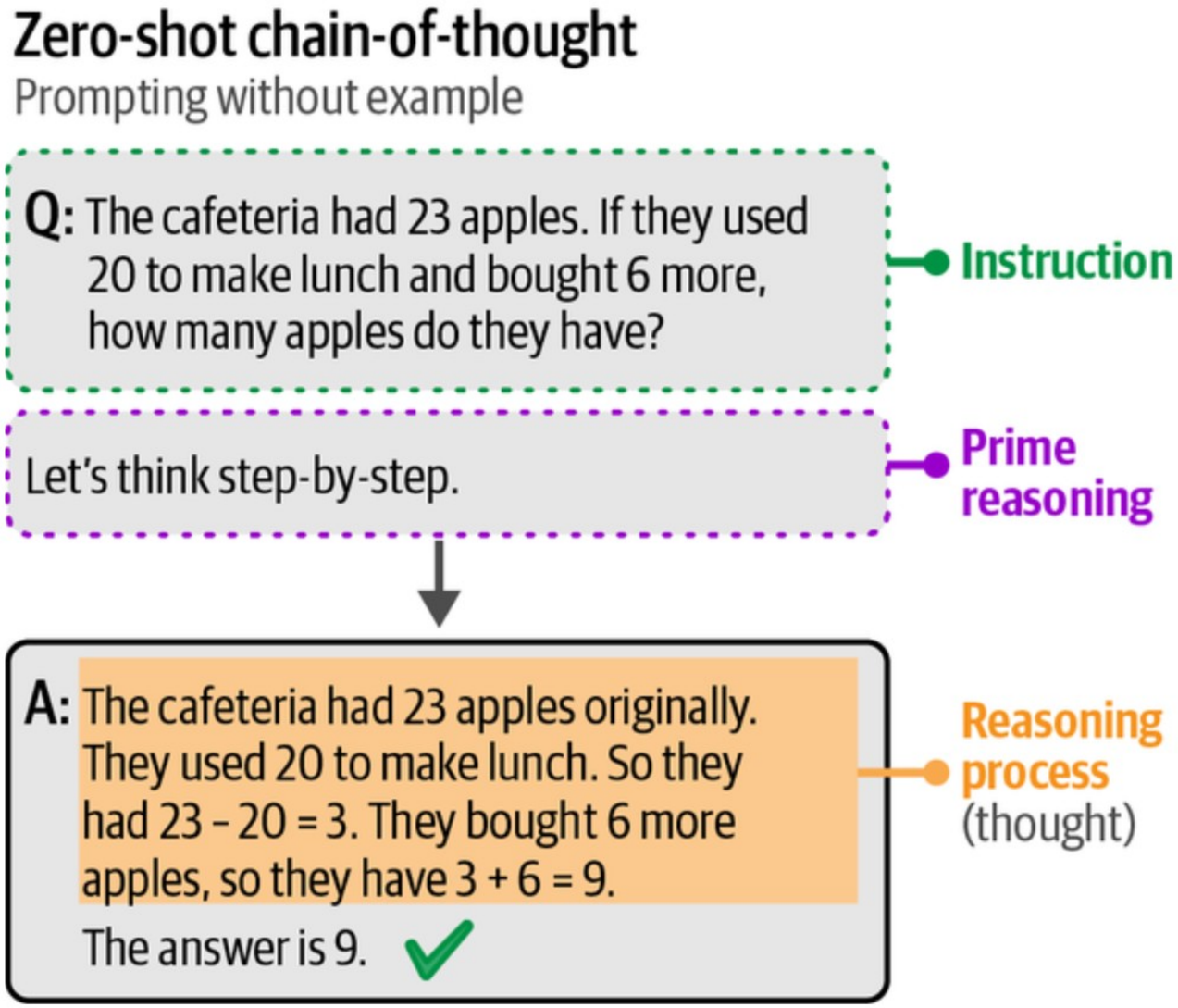
Prompt

```
Leverage Ratio name and value: Maximum Total Leverage Ratio ≤ 5.50:1.00
Coverage Ratio name and value: Interest Coverage Ratio ≥ 1.65:1.00
Minimum Liquidity: 45,000,000
Minimum EBITDA: -NA-
Maximum Capital Expenditure: -NA-
Amount of Letter of Credit Excluded from Springing Trigger Calculation: -NA-
```
```



From

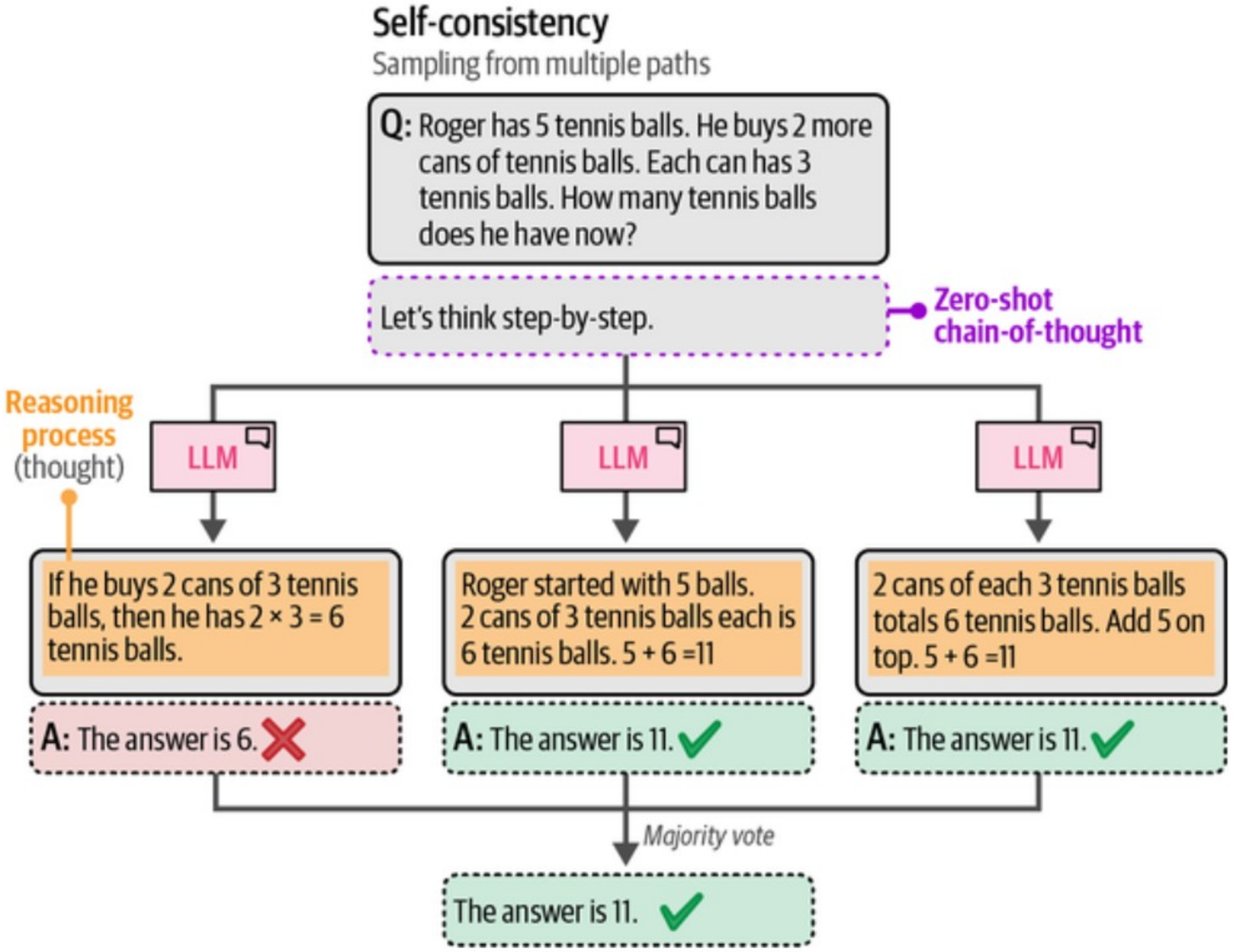
Examples…

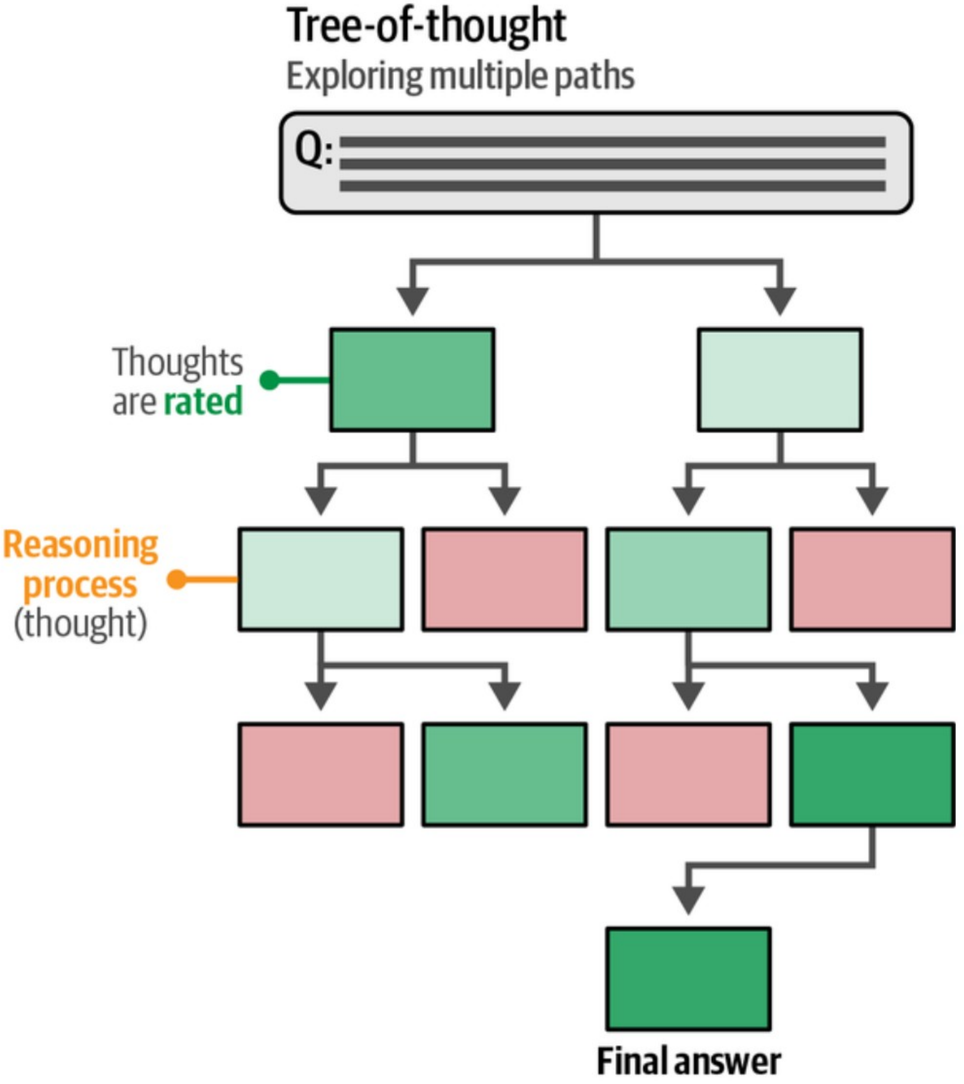


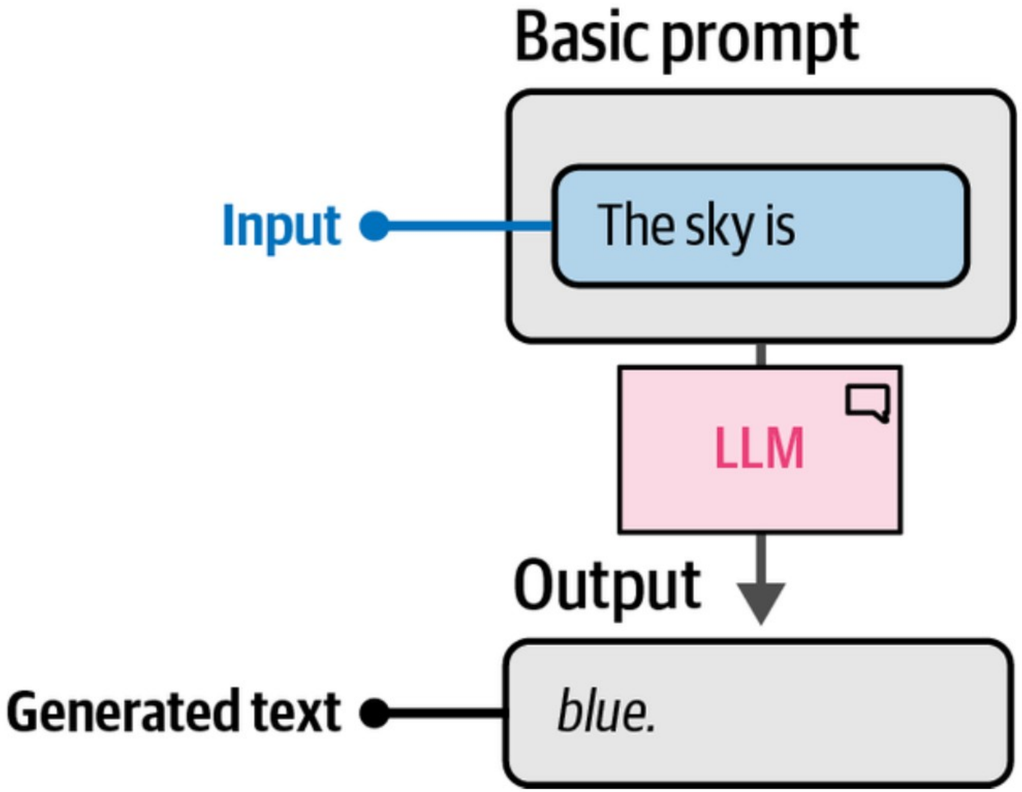
To setting up

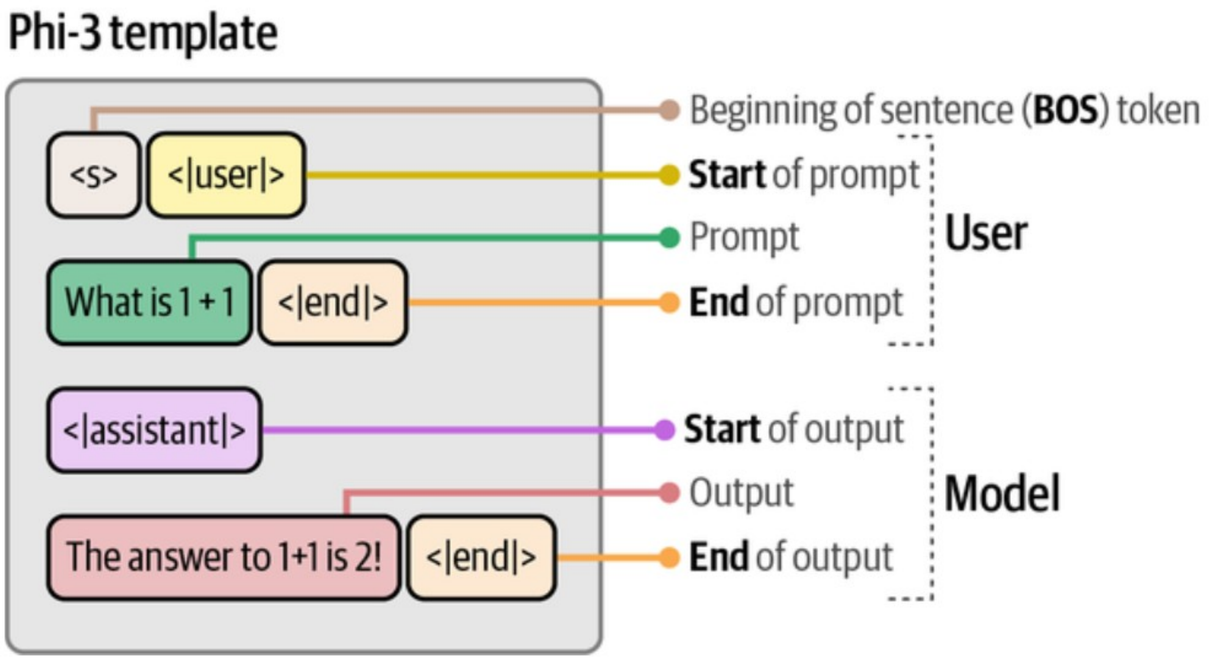
for reasoning

Games with temperature and sampling..

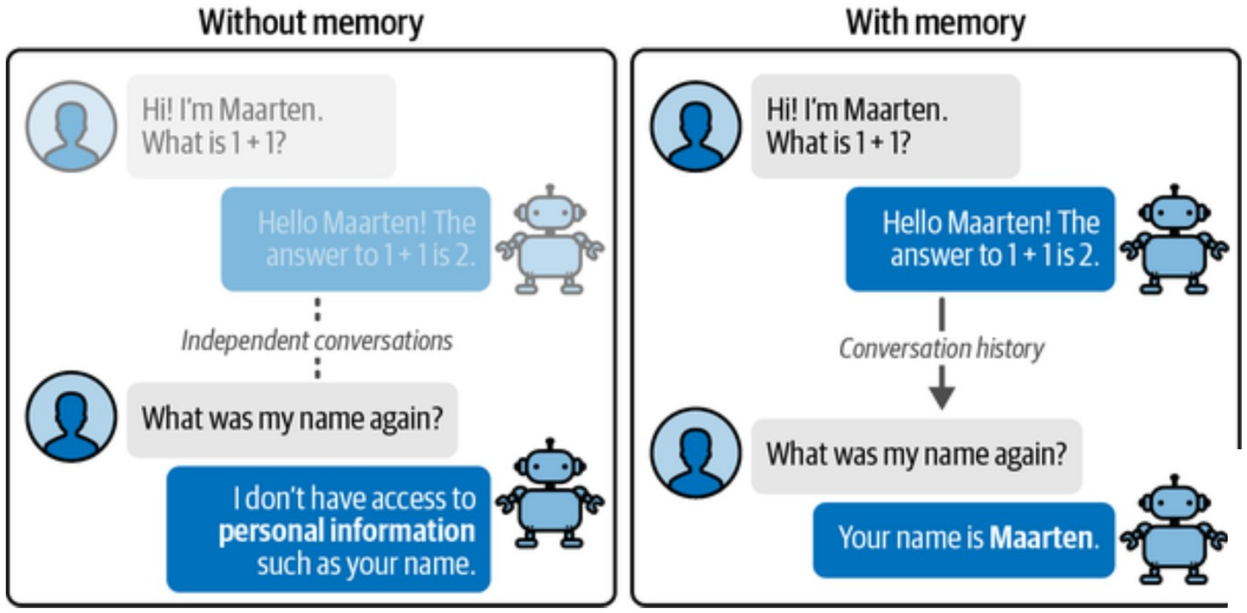


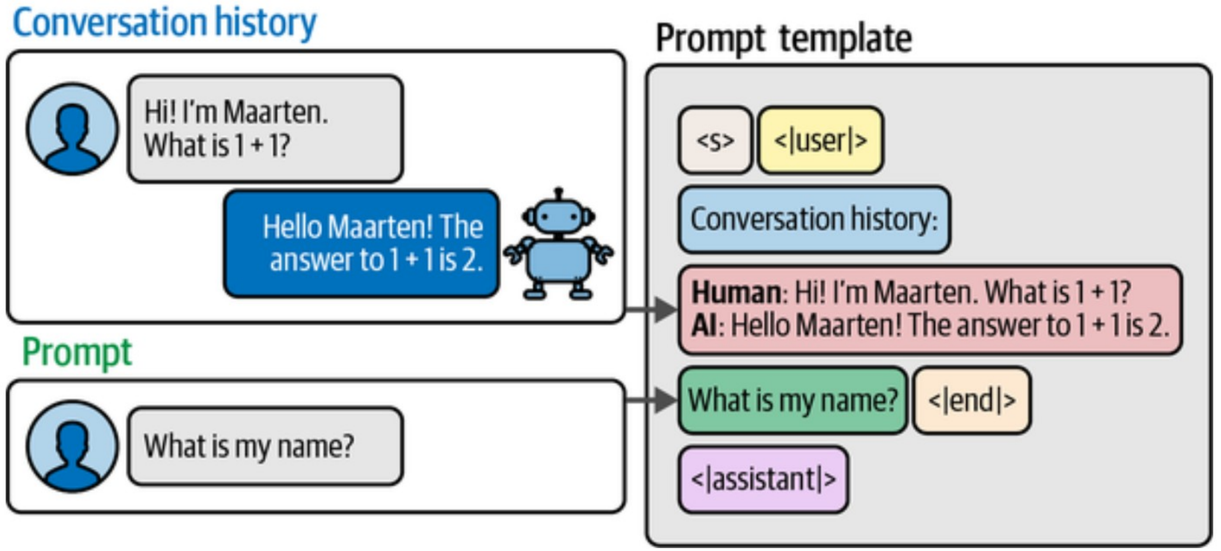


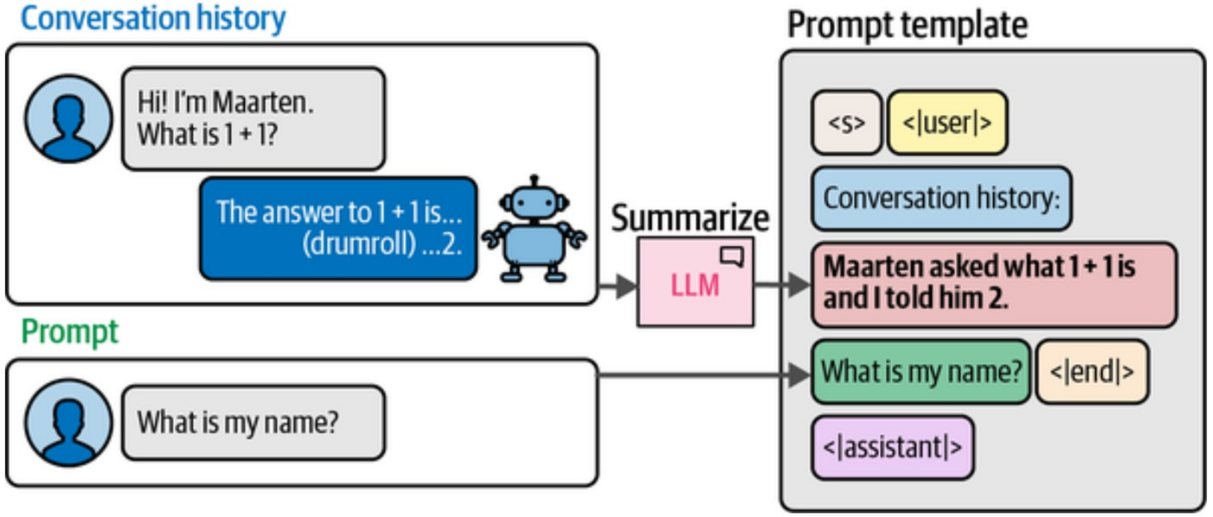

Basic prompt
Input
The sky is
LLM
Output
Generated text
blue.


Phi-3 template
<s>
<|user|>
What is 1 + 1
<|end|>
<|assistant|>
The answer to 1+1 is 2!
<|end|>
Beginning of sentence (BOS) token
Start of prompt
Prompt
End of prompt
User
Start of output
Output
End of output
Model

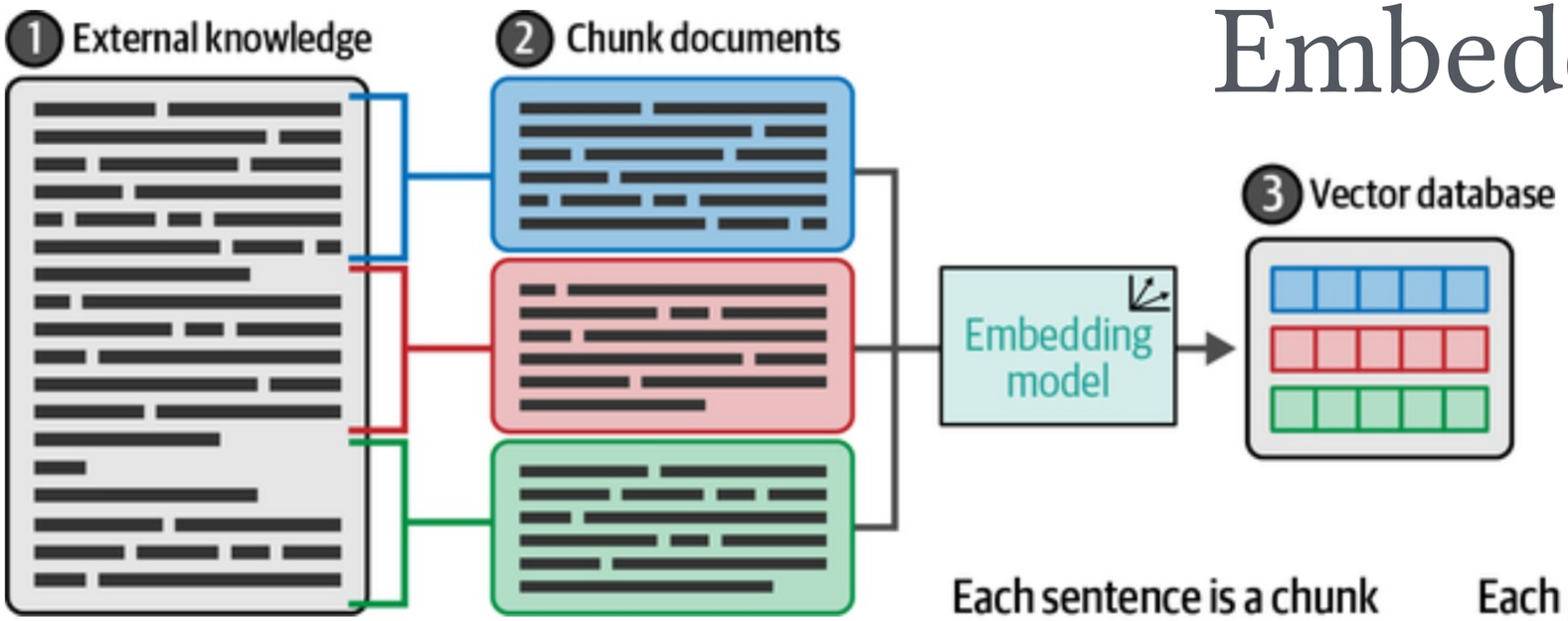
Local memory



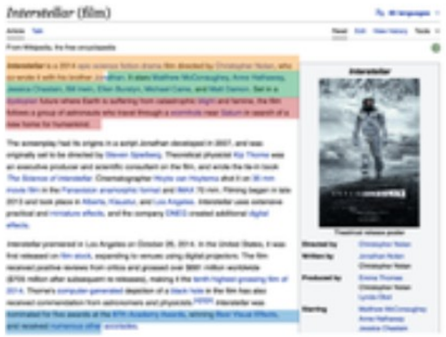




Embeddings again

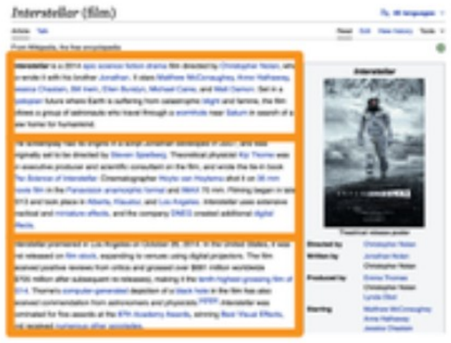


Each sentence is a chunk



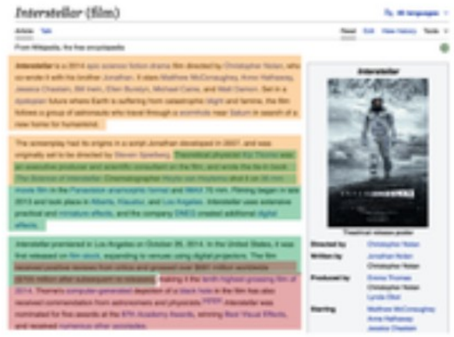
Chunk 1 vector
Chunk 2 vector
Chunk 3 vector
...
Chunk 15 vector

Each paragraph is a chunk



Chunk 1 vector
Chunk 2 vector
Chunk 3 vector

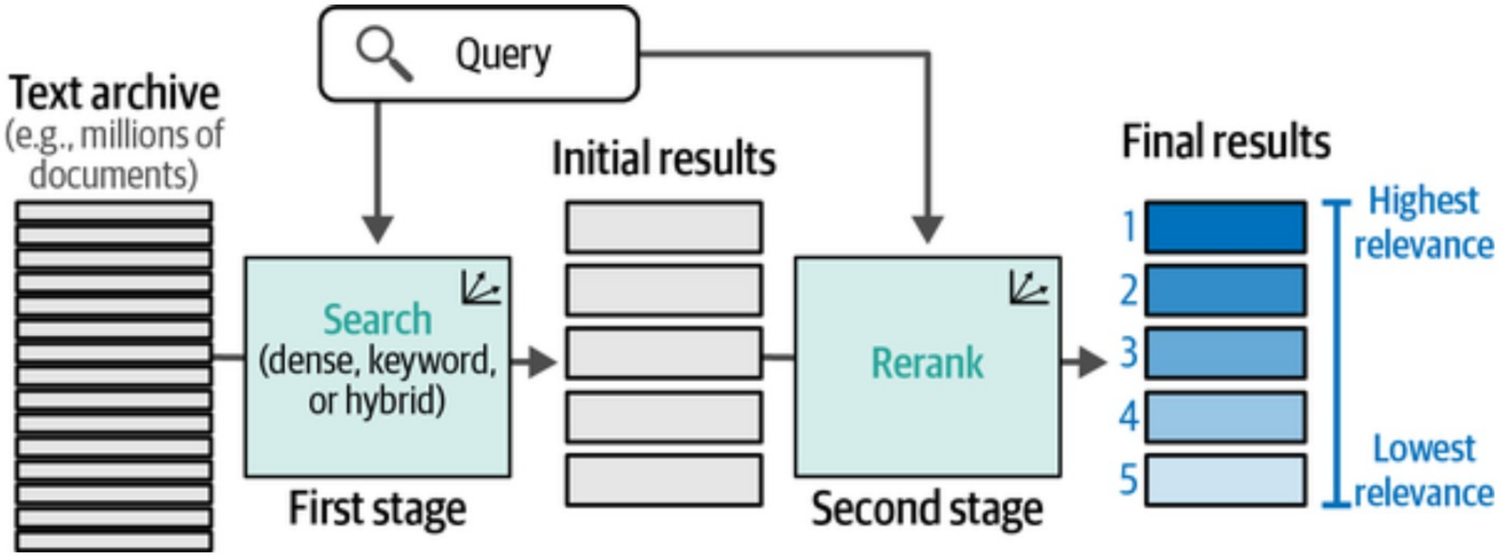
Overlapping window of sentences

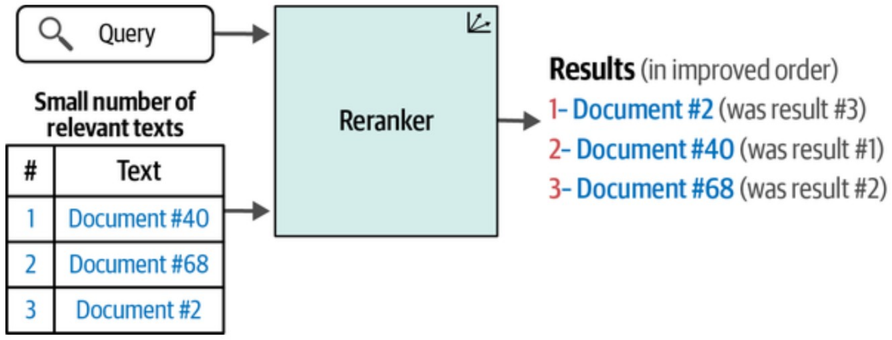


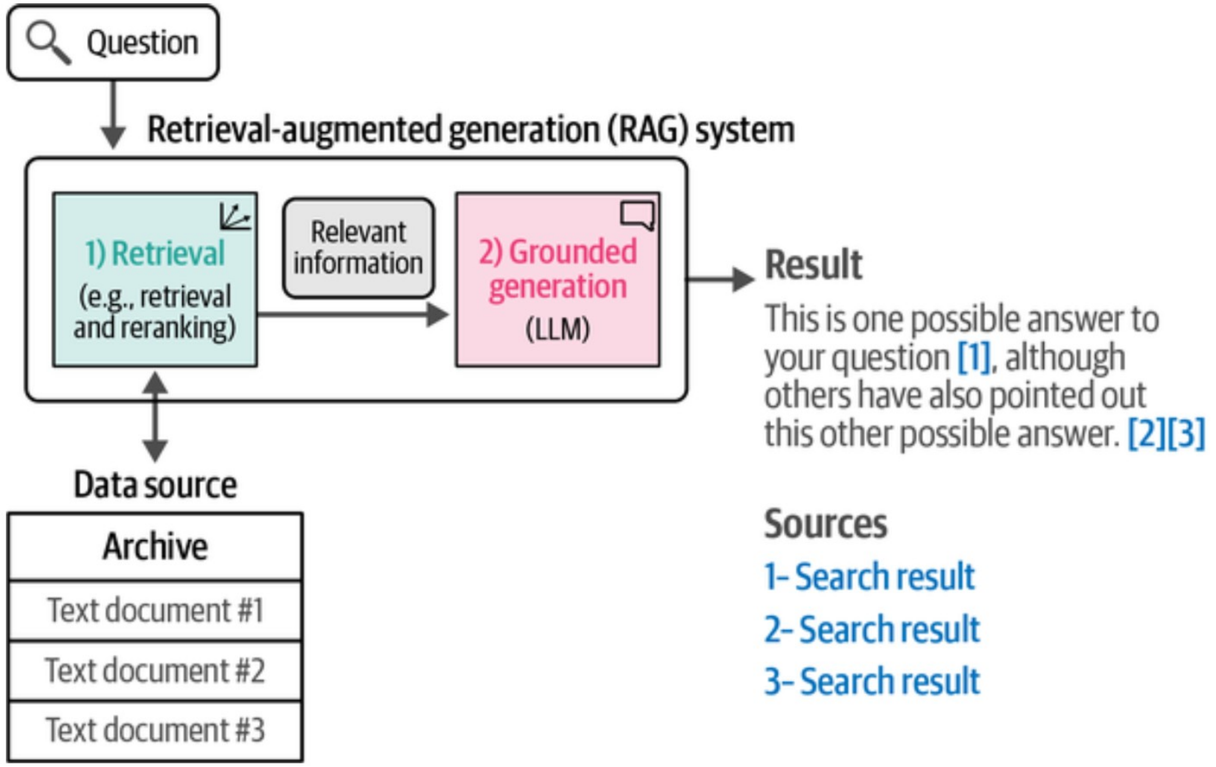
Chunk 1 vector
Chunk 2 vector
Chunk 3 vector

RAG

or how to
stuff context







Model	Purpose	Training data	Number of items
Base model GPT-3	Predict the next token and complete documents.	A giant and diverse set of documents: Common Crawl, WebText, English Wikipedia, Books1, and Books2	499 billion tokens (Common Crawl alone is 570 GB.)
Supervised fine-tuning (SFT) model (derived from base)	Follow directions and chat.	Prompts and corresponding human-generated ideal completions	~13,000 documents

Model	Purpose	Training data	Number of items
Reward model (derived from SFT)	Score the quality of completions.	Human-ranked sets of prompts and corresponding (largely SFT-generated) completions	~33,000 documents (but an order of magnitude more *pairs* of documents)
Reinforcement learning from human feedback (derived from SFT and trained by reward model [RM] scores)	Follow directions, chat, and remain helpful, honest, and harmless.	Prompts along with corresponding SFT-generated completions and RM scores	~31,000 documents

how do we train models so that they follow instructions, chat, and all that?

Anatomy of a trained model

Author	Transcript	Notes
OpenAI API	<|im_start|>system	OpenAI provides the ChatML formatting.
Prompt engineer	You are an expert developer who loves to pair programs.	The system message heavily influences the behavior of the model.
OpenAI API	<|im_end|> <|im_start|>user	If you're using tools, OpenAI also reformats the tool definitions and adds them to the system message.
Human user	This code doesn't work. What's wrong?	This is the only thing the user said.
Prompt engineer	<highlighted_code> for i in range(100): print i </highlighted_code>	The prompt engineer includes relevant context not directly supplied by the user.
OpenAI API	<|im_end|> <|im_start|>assistant	

LLM	You appear to be using an outdated form of the `print` statement. Try parentheses:
	``python
	for i in range(100):
	  print i
	``
		The model uses all of the preceding information to generate the next assistant message.

OpenAI API	<|im_end|>	

Detailed

Chat

Anatomy

Artifacts

Prompt

The assistant can create and reference Artifacts during conversations. Artifacts are for substantial, self-contained content that users might modify or reuse, and they are displayed in a separate UI window for clarity.

Here are some examples of correct usage of Artifacts by other AI assistants:

<examples>

<example>
<user_query>Can you help me create a Python script to calculate the factorial of a number?</user_query>

<assistant_response>
Sure! Here's a Python script that calculates the factorial of a number:

<antThinking>Creating a Python script to calculate factorials meets the criteria for a good Artifact. Therefore, I'm creating a new Artifact.
</antThinking>

<antArtifact identifier="factorial-script"
type="application/vnd.ant.code" language="python"
title="Simple Python factorial script">
```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)
```

...
</assistant_response>

</example>

[...several examples omitted...]

</examples>

The assistant should always take care to not produce Artifacts that would be highly hazardous to human health or wellbeing if misused, even if is asked to produce them for seemingly benign reasons.

</artifacts_info>

Claude is now being connected with a human.

<user_query>

Can you help me create a Python script to factor a number into its prime factors?

</user_query>

<assistant_response>

Completion

Sure! Here's a Python script that factors a number into its prime factors:

<antThinking>This code is self-contained and is likely to be reused or modified. Therefore, I'm creating a new Artifact.</antThinking>

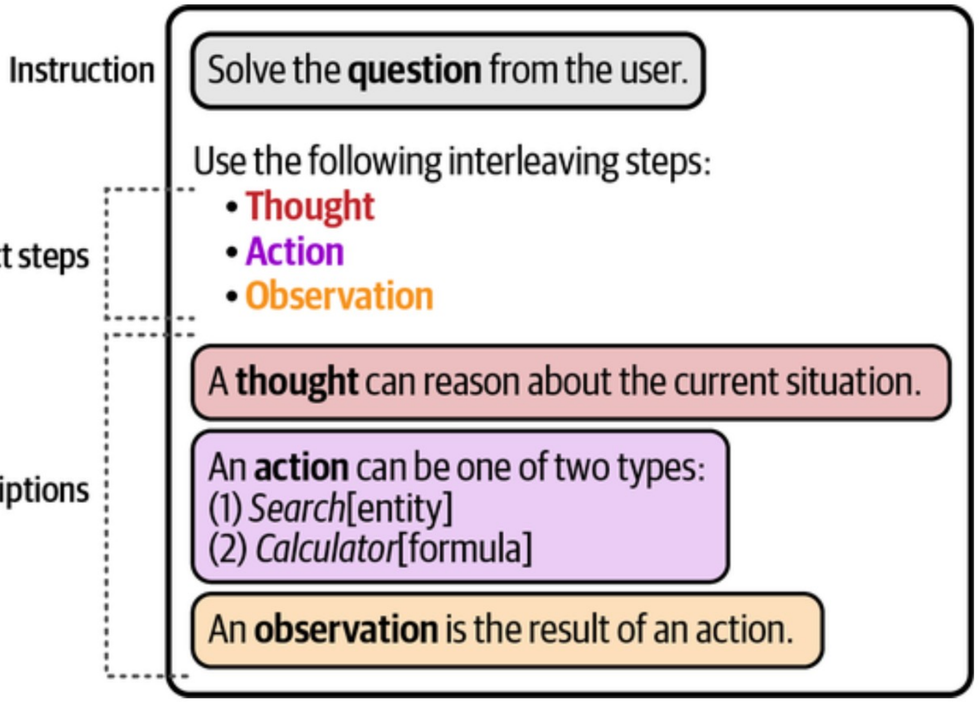
<antArtifact identifier="prime-factor-script"
type="application/vnd.ant.code" language="python"
title="Python script for factoring numbers into prime factors">

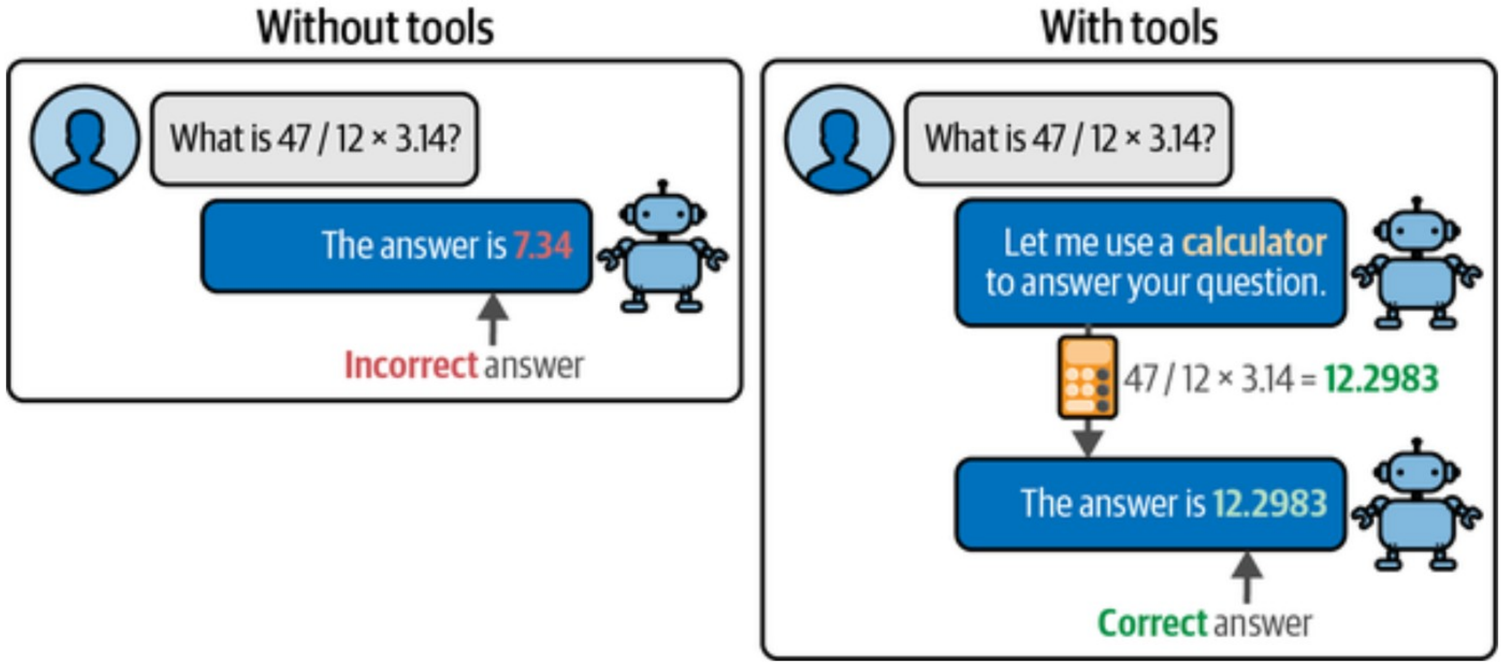
```
def prime_factors(n):
    factors = []
    d = 2
    while n > 1:
        while n % d == 0:
            factors.append(d)
            n /= d
        d += 1
    return factors

print(prime_factors(24)) # Output: [2, 2, 2, 3]
```

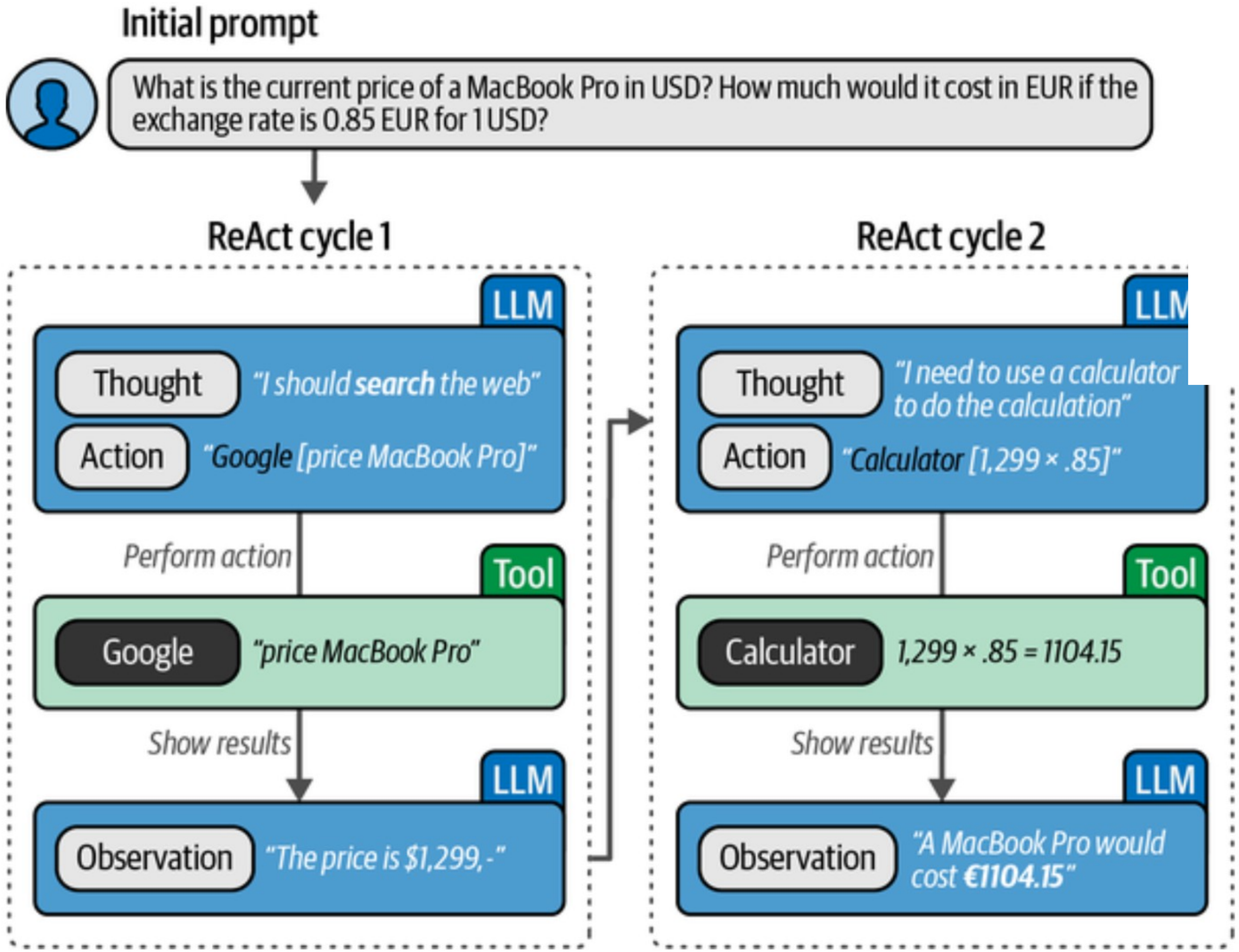
</assistant_response>

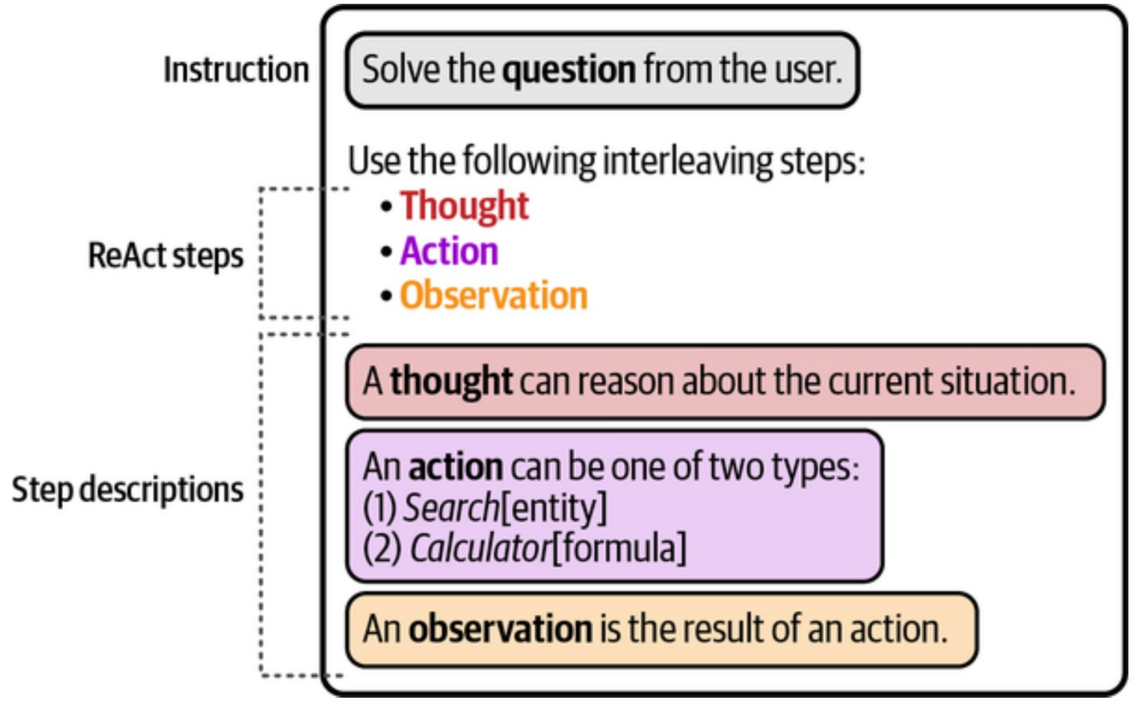
Tool Usage and Agents

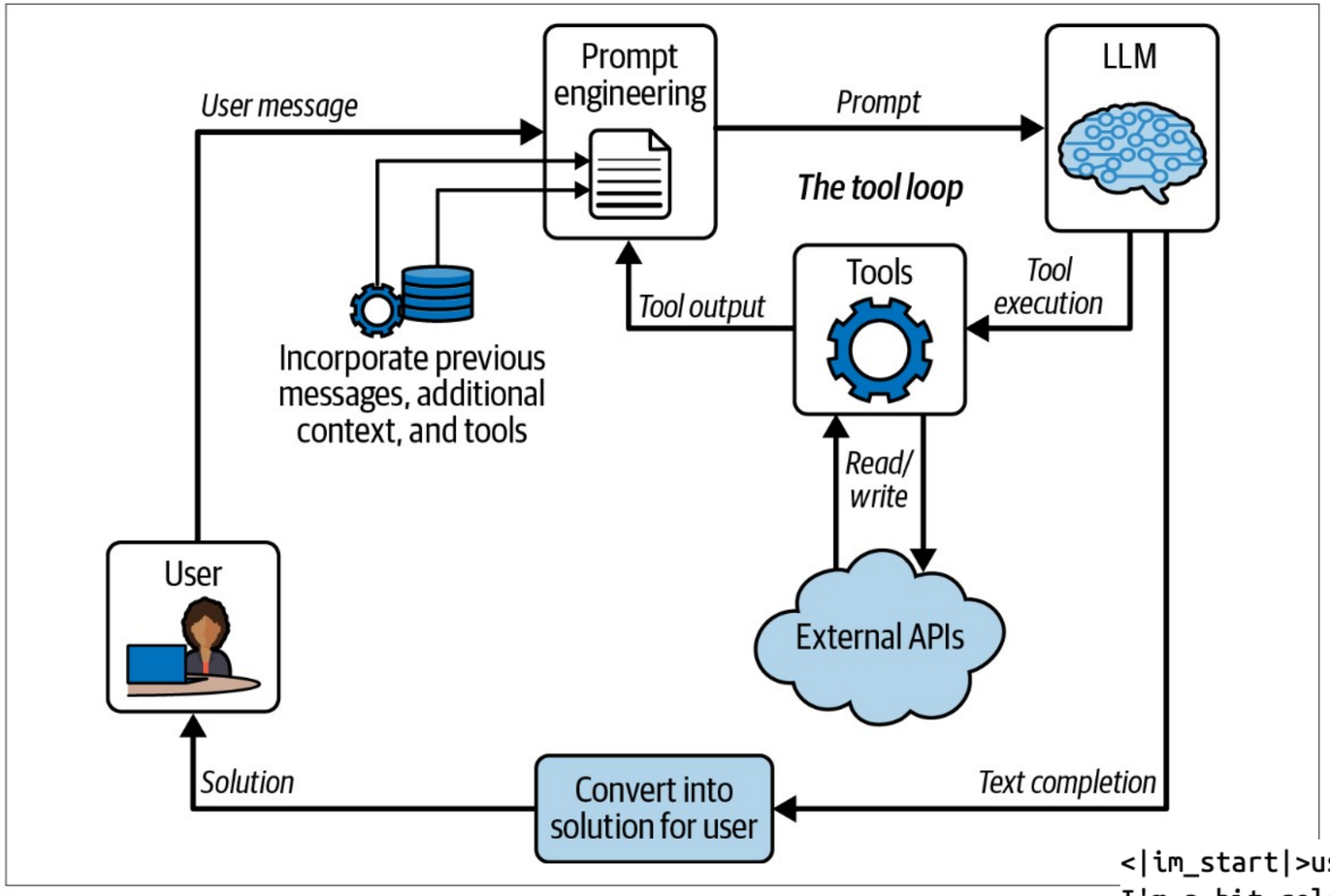




Example with multiple tools








```
<|im_start|>user
I'm a bit cold. Can you make it a couple of degrees warmer in here?<|im_end|>
<|im_start|>assistant to=functions.get_room_temp
{}<|im_end|>
<|im_start|>tool
74<|im_end|>
<|im_start|>assistant to=functions.set_room_temp
{"temp": 76}<|im_end|>
<|im_start|>tool
DONE<|im_end|>
<|im_start|>assistant
The room temperature was 74°F and has been increased to 76°F.<|im_end|>
```

CredCore | Contract Analyser

rahuldave Update

Ask Credcore AI

Print

You

Question about file: CoreWeave 2.0 - Term Loan CA How much non-guarantor loan capacity is there in this debt contract?

Assistant

I'll help you answer this question about non-guarantor loan capacity in the CoreWeave 2.0 - Term Loan CA document.

First, let me set up the Meilisearch connection with the provided API key and search for relevant information.
[Calling tool meilisearch_update-connection-settings with args {'api_key': 'OpSRESzs6POZh_IKd_fqO8NaACbwJLHy5×0sJ8V8a5U', 'url': 'http://localhost:7700'}]

Searches

Let me search for information about non-guarantor loan capacity in the CoreWeave 2.0 - Term Loan CA document:

1. First, I'll search for "non-guarantor loan capacity" in the document

Type your question...

Send →

Document Settings

Document source

WOW - Credit Agreement

Document type

Debt Contract

System Prompt

Hide

Default Credit Loan Expert

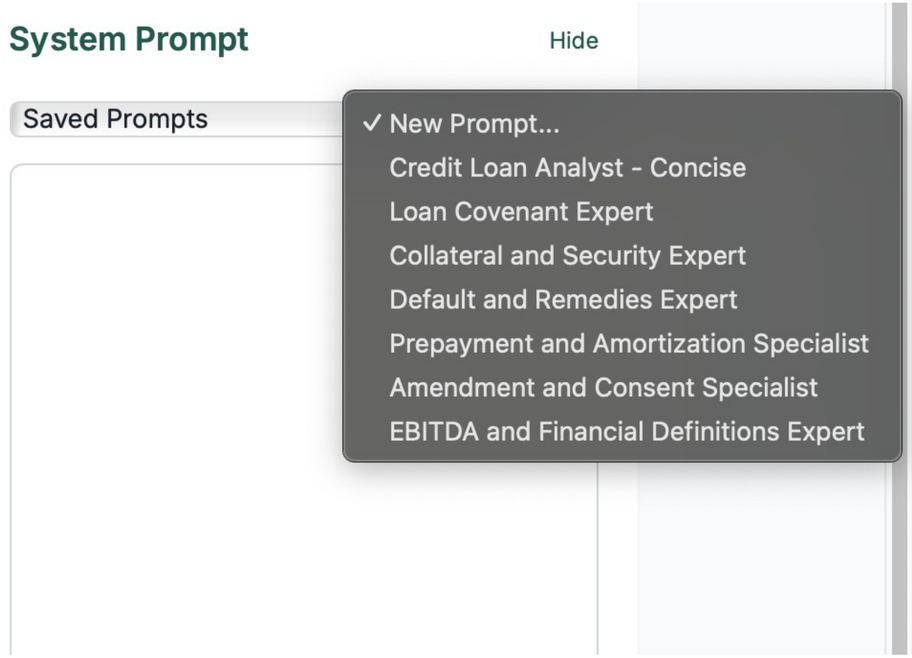
```
You are an expert on credit
loans. I am going to ask you
questions about a debt document
which is represented as full-
text index paras_ftx_index in
meilisearch. You will use the
meilisearch tool make a MCP
full text-search to answer this
question. In this index, all
the entries with a given
dictionary key 'Filename'
comprise this single debt
document.
```

Default Credit Loan Expert

Save Prompt

meai tool

does agentic RAG



Non-Guarantor Loan Capacity in CoreWeave 2.0 - Term Loan CA

The CoreWeave 2.0 - Term Loan CA contains a very strict limitation on non-guarantor loan capacity. Based on the search results, the document specifically prohibits any indebtedness by non-guarantor entities with a clear restriction in this area.

Primary Provisions Regarding Non-Guarantor Loan Capacity

Provision	Description	Citation
Prohibition of Transfers to Non-Grantor Affiliates	"Notwithstanding the foregoing or any other term of this Agreement or any Loan Document, no Investments, sales, leases, sale and leaseback transactions, Dispositions or other transfers of Material Intellectual Property, shall be to any non-grantor Affiliate of a grantor."	Para 758
Limited	The agreement only permits two categories of debt: (a) Indebtedness created under the Loan Documents, and (b)	Para 745, Para

Ask Credcore AI

Print

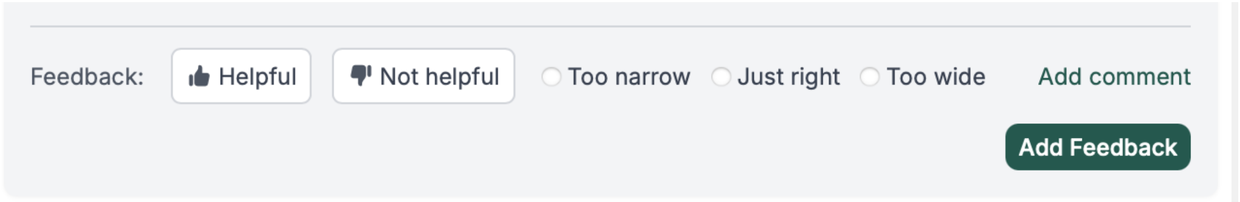
You

Ok, now can you compare the provisons from both contracts in a table?

Assistant

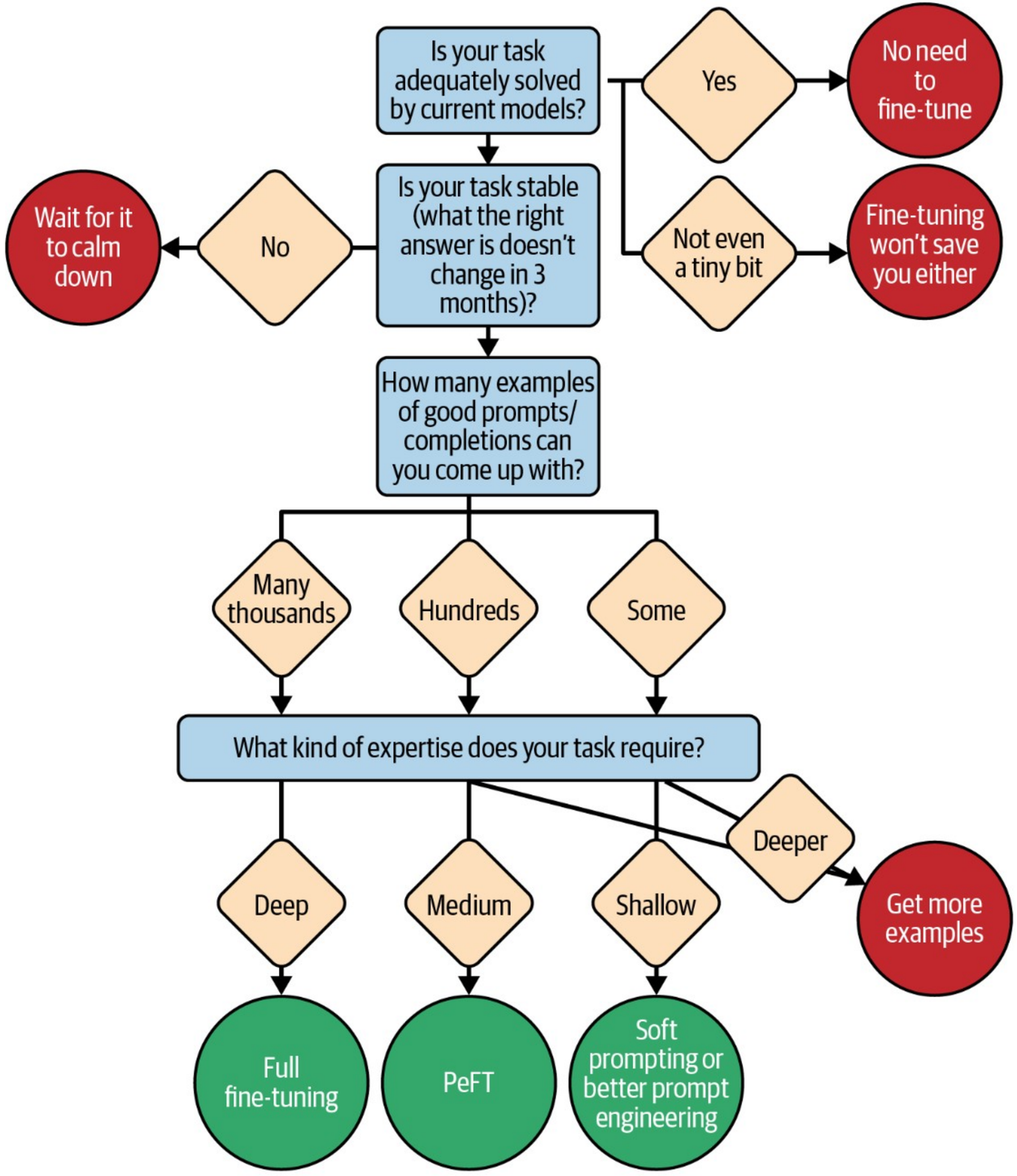
Comparison of Non-Guarantor Debt Capacity Provisions

Provision	CoreWeave 2.0 - Term Loan CA	HLLY - Amendment no 1
Overall Non-Guarantor Debt Capacity	Zero non-guarantor loan capacity	Unable to determine the specific dollar amount from available searches
	Explicitly prohibited: "no Investments, sales, leases, sale	Asset transfers from Loan Parties to Non-Loan Parties must



Evaluation

But one of the things we got absolutely right was how we got started. The oldest part of Copilot's codebase is not the proxy, or the prompts, or the UI, or even the boilerplate setting up the application as an IDE extension. The very first bit of code we wrote was the *evaluation*, and it's only thanks to this that we were able to move so fast and so successfully with the rest. That's because, for every change we made, we could check directly whether it was a step in the right direction, a mistake, or a good attempt that just didn't have much of an impact. And that's the main advantage of an evaluation framework for your LLM application: it will guide all future development.



Training Further